C CampusFlow

CampusFlow UI Document

Screenshots, file structure and source code of every interface

React.js · JavaScript · Axios · React Router · plain CSS
Backend: Java 17 · Spring Boot · Spring Security · JWT · MySQL 8

All names and numbers in the screenshots are demo data.

How to use this document

1. Open the **frontend**/ folder from the package in VS Code: it is the complete UI, ready to run (`npm install`, then `npm run dev`).
2. Each interface below shows its screenshot, route, role, the data it needs, and the page source code.
3. The backend only has to provide the endpoints and field names listed for each screen. The frontend is not redesigned.
4. Shared code (components, layouts, services, styles) is in Appendix A.

Interface index

#	Interface	Route	Role
1	Login	`/login`	Public
2	Register (student)	`/register`	Public
3	Register (faculty, role-specific fields)	`/register`	Public
4	Student dashboard	`/student`	STUDENT
5	Courses (student view)	`/student/courses`	STUDENT
6	Assignments (student)	`/student/assignments`	STUDENT
7	Attendance (student)	`/student/attendance`	STUDENT
8	Results	`/student/results`	STUDENT
9	Skills and learning path	`/student/skills`	STUDENT
10	Placements (student)	`/student/placements`	STUDENT
11	Campus events (list)	`/student/events`	STUDENT, FACULTY, PLACEMENT_OFFICER, ADMIN (view); register is STUDENT
12	Campus events (calendar view)	`/student/events`	Same as events list
13	Event details	`/student/events/:id`	All logged-in roles
14	Notifications	`/student/notifications`	All logged-in roles
15	AI College Assistant	`/student/assistant`	All logged-in roles
16	Profile	`/student/profile`	All logged-in roles
17	Global search results	`/student/search?q=software`	All logged-in roles
18	Faculty dashboard	`/faculty`	FACULTY
19	Assignments (faculty)	`/faculty/assignments`	FACULTY
20	Record attendance (faculty)	`/faculty/attendance`	FACULTY
21	Placement officer dashboard	`/placement`	PLACEMENT_OFFICER
22	Placement drive management	`/placement/drives`	PLACEMENT_OFFICER
23	Administrator dashboard	`/admin`	ADMIN
24	Users and approvals	`/admin/users`	ADMIN
25	Course management (add course dialog)	`/admin/courses`	ADMIN, FACULTY (manage mode)
26	Manage events (admin)	`/admin/events`	ADMIN, FACULTY (manage mode)
27	Venue management	`/admin/venues`	ADMIN
28	Student dashboard on a phone (responsive)	`/student`	STUDENT

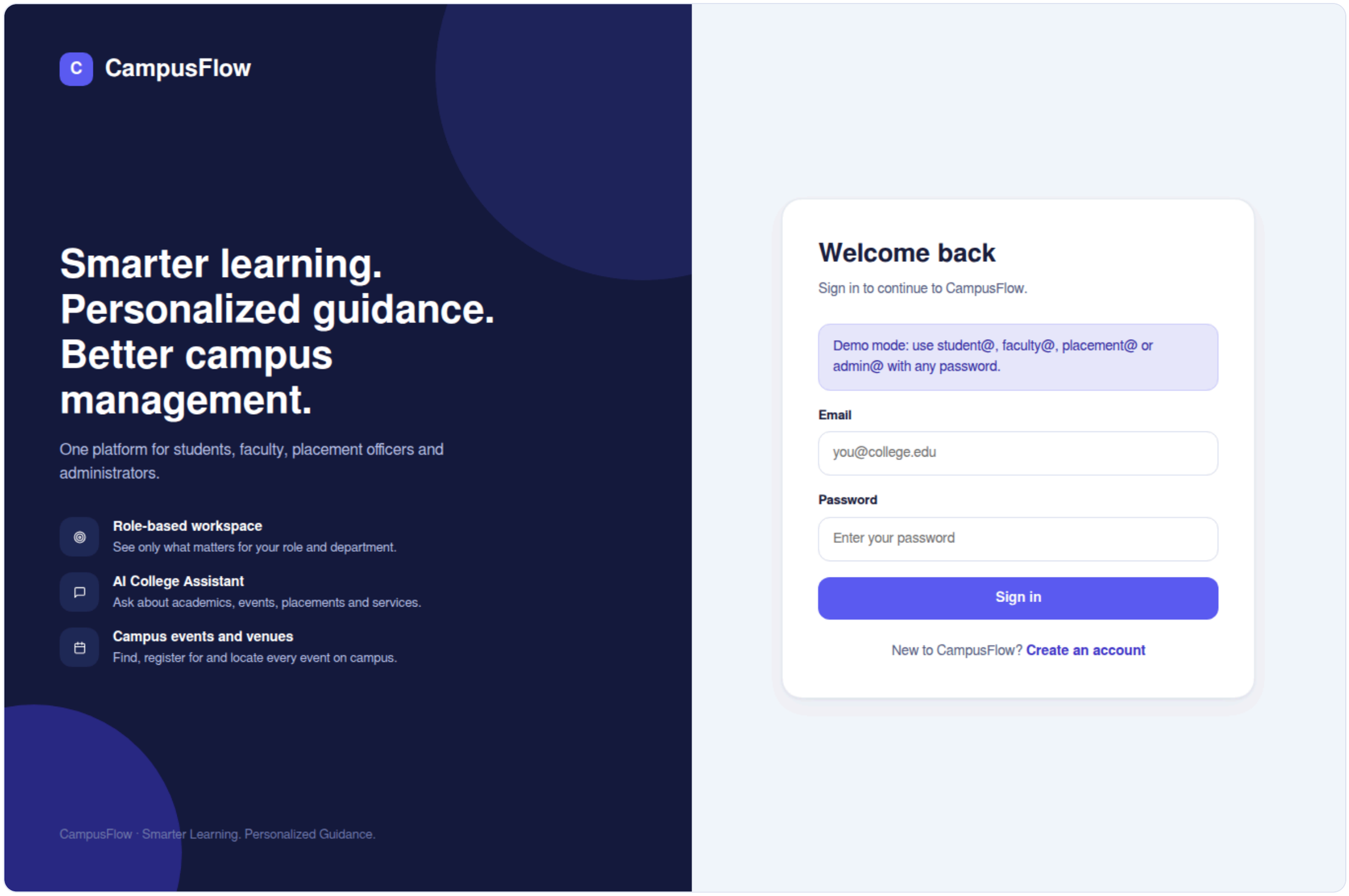
File structure

```
frontend/
├── public/
│   └── .gitkeep
├── src/
│   ├── assets/
│   │   └── .gitkeep
│   ├── components/
│   │   ├── Alert.jsx
│   │   ├── Avatar.jsx
│   │   ├── Badge.jsx
│   │   ├── BarChart.jsx
│   │   ├── Card.jsx
│   │   ├── DataTable.jsx
│   │   ├── EmptyState.jsx
│   │   ├── EventCard.jsx
│   │   ├── Field.jsx
│   │   ├── Hero.jsx
│   │   ├── MeterRow.jsx
│   │   ├── Modal.jsx
│   │   ├── PageHeader.jsx
│   │   ├── ProgressBar.jsx
│   │   ├── RingChart.jsx
│   │   ├── SearchInput.jsx
│   │   ├── Skeleton.jsx
│   │   ├── StatCard.jsx
│   │   └── Tabs.jsx
│   ├── context/
│   │   └── AuthContext.jsx
```

```
│   ├── data/
│   │   └── demoData.js
│   ├── hooks/
│   │   └── .gitkeep
│   ├── layouts/
│   │   ├── AppLayout.jsx
│   │   ├── AuthLayout.jsx
│   │   ├── Sidebar.jsx
│   │   └── Topbar.jsx
│   ├── pages/
│   │   ├── admin/
│   │   │   ├── AdminDashboard.jsx
│   │   │   ├── UserManagement.jsx
│   │   │   └── Venues.jsx
│   │   ├── auth/
│   │   │   ├── Login.jsx
│   │   │   └── Register.jsx
│   │   ├── faculty/
│   │   │   ├── FacultyDashboard.jsx
│   │   │   └── ManageAttendance.jsx
│   │   ├── placement/
│   │   │   ├── DriveManagement.jsx
│   │   │   └── PlacementDashboard.jsx
│   │   ├── shared/
│   │   │   ├── AIAssistant.jsx
│   │   │   ├── Assignments.jsx
│   │   │   ├── Courses.jsx
│   │   │   ├── EventDetails.jsx
│   │   │   ├── Events.jsx
│   │   │   ├── Notifications.jsx
│   │   │   ├── Profile.jsx
│   │   │   └── SearchResults.jsx
│   │   └── student/
│   │       ├── Attendance.jsx
│   │       ├── Placements.jsx
│   │       ├── Results.jsx
│   │       ├── Skills.jsx
│   │       └── StudentDashboard.jsx
│   ├── routes/
│   │   ├── ProtectedRoute.jsx
│   │   └── routes.jsx
│   ├── services/
│   │   ├── api.js
│   │   ├── assistantService.js
│   │   ├── authService.js
│   │   └── searchService.js
│   ├── styles/
│   │   ├── auth.css
│   │   ├── components.css
│   │   ├── global.css
│   │   ├── layout.css
│   │   └── pages.css
│   ├── utils/
│   │   ├── format.js
│   │   └── navConfig.js
│   ├── App.jsx
│   └── main.jsx
├── .env.example
├── .gitignore
├── index.html
├── package.json
└── vite.config.js
```

Login

Route	`/login`
Role	Public
Page file	`frontend/src/pages/auth/Login.jsx`
Data needed	`POST /api/auth/login -> { token, user:{id,name,email,role,department} }`
Behaviour	Validates email and password, shows API errors and the session-expired notice, then redirects by role (STUDENT /student, FACULTY /faculty, PLACEMENT_OFFICER /placement, ADMIN /admin).



Source code

`frontend/src/pages/auth/Login.jsx`

```
import { useState } from 'react';
import { Link, useNavigate, useSearchParams } from 'react-router-dom';
import { FiMail, FiLock } from 'react-icons/fi';
import { useAuth } from '../../context/AuthContext';
import { isDemoMode } from '../../services/authService';
import { ROLE_HOME, getErrorMessage } from '../../utils/format';
import Field from '../../components/Field';
import Alert from '../../components/Alert';

export default function Login() {
  const { login } = useAuth();
  const navigate = useNavigate();
  const [params] = useSearchParams();
  const [form, setForm] = useState({ email: '', password: '' });
  const [errors, setErrors] = useState({});
  const [apiError, setApiError] = useState('');
  const [loading, setLoading] = useState(false);

  const update = (key) => (e) => setForm((f) => ({ ...f, [key]: e.target.value }));

  const validate = () => {
    const next = {};
    if (!/^\S+@\S+\.\S+$/.test(form.email)) next.email = 'Enter a valid email address.';
    if (!form.password) next.password = 'Enter your password.';
    setErrors(next);
    return Object.keys(next).length === 0;
  };

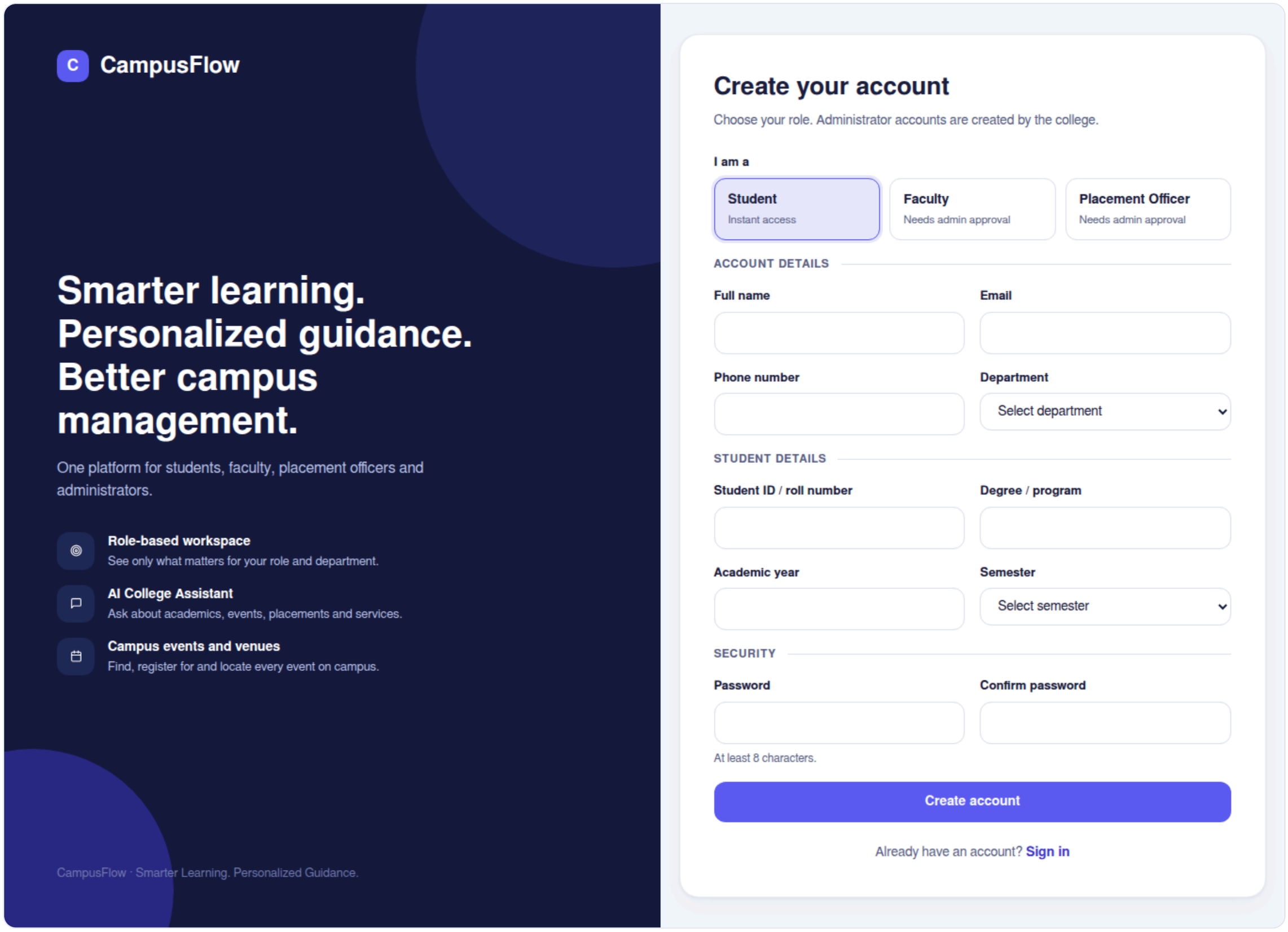
  const onSubmit = async (e) => {
    e.preventDefault();
    setApiError('');
    if (!validate()) return;
    setLoading(true);
    try {
```

```
      const user = await login(form.email, form.password);
      navigate(ROLE_HOME[user.role], { replace: true });
    } catch (err) {
      setApiError(getErrorMessage(err));
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="auth-card">
      <h1>Welcome back</h1>
      <p className="lead">Sign in to continue to CampusFlow.</p>
      <form className="auth-form" onSubmit={onSubmit} noValidate>
        {params.get('expired') && <Alert tone="warning">Your session expired. Please sign in again.</Alert>}
        {apiError && <Alert tone="error">{apiError}</Alert>}
        {isDemoMode && (
          <Alert tone="info">Demo mode: use student@, faculty@, placement@ or admin@ with any password.</Alert>
        )}
        <Field label="Email" id="login-email" error={errors.email}>
          <input className="input" type="email" autoComplete="email" placeholder="you@college.edu" value={form.email} onChange={update('email')} />
        </Field>
        <Field label="Password" id="login-password" error={errors.password}>
          <input className="input" type="password" autoComplete="current-password" placeholder="Enter your password" value={form.password} onChange={update('password')}
/>
        </Field>
        <button type="submit" className="btn btn-primary btn-block" disabled={loading}>
          {loading ? 'Signing in...' : 'Sign in'}
        </button>
      </form>
      <p className="auth-switch">
        New to CampusFlow? <Link to="/register">Create an account</Link>
      </p>
    </div>
  );
}
```

Register (student)

Route	`/register`
Role	Public
Page file	`frontend/src/pages/auth/Register.jsx`
Data needed	`POST /api/auth/register`
Behaviour	Public self-registration for STUDENT only. Fields: name, email, phone, department, password, studentId, program, academicYear, semester. Duplicate email and student ID must be rejected by the backend.



Source code

`frontend/src/pages/auth/Register.jsx`

```
import { useState } from 'react';
import { Link } from 'react-router-dom';
import { registerRequest } from '../../services/authService';
import { getErrorMessage } from '../../utils/format';
import Field from '../../components/Field';
import Alert from '../../components/Alert';

// Public registration is limited to these roles. ADMIN accounts are provisioned by the system.
const ROLES = [
  { value: 'STUDENT', label: 'Student', hint: 'Instant access' },
  { value: 'FACULTY', label: 'Faculty', hint: 'Needs admin approval' },
  { value: 'PLACEMENT_OFFICER', label: 'Placement Officer', hint: 'Needs admin approval' },
];

const COMMON = { name: '', email: '', phone: '', department: '', password: '', confirmPassword: '' };
const EXTRA = {
  STUDENT: { studentId: '', program: '', academicYear: '', semester: '' },
  FACULTY: { employeeId: '', designation: '' },
  PLACEMENT_OFFICER: { employeeId: '', placementDepartment: '', designation: '' },
};
const LABELS = {
  studentId: 'Student ID / roll number', program: 'Degree / program', academicYear: 'Academic year', semester: 'Semester',
  employeeId: 'Employee ID', designation: 'Designation', placementDepartment: 'Placement department',
};

export default function Register() {
  const [role, setRole] = useState('STUDENT');
  const [form, setForm] = useState({ ...COMMON, ...EXTRA.STUDENT });
  const [errors, setErrors] = useState({});
  const [apiError, setApiError] = useState('');
```

```
const [success, setSuccess] = useState('');
const [loading, setLoading] = useState(false);

const chooseRole = (value) => {
  setRole(value);
  setForm({ ...COMMON, ...EXTRA[value], name: form.name, email: form.email, phone: form.phone, department: form.department });
  setErrors({});
};
const update = (key) => (e) => setForm((f) => ({ ...f, [key]: e.target.value }));

const validate = () => {
  const next = {};
  if (form.name.trim().length < 2) next.name = 'Enter your full name.';
  if (!/^\S+@\S+\.\S+$/.test(form.email)) next.email = 'Enter a valid email address.';
  if (!/^[0-9+\s-]{8,15}$/.test(form.phone)) next.phone = 'Enter a valid phone number.';
  if (!form.department) next.department = 'Select your department.';
  if (form.password.length < 8) next.password = 'Use at least 8 characters.';
  if (form.confirmPassword !== form.password) next.confirmPassword = 'Passwords do not match.';
  Object.keys(EXTRA[role]).forEach((key) => {
    if (!String(form[key]).trim()) next[key] = 'This field is required.';
  });
  setErrors(next);
  return Object.keys(next).length === 0;
};

const onSubmit = async (e) => {
  e.preventDefault();
  setApiError('');
  setSuccess('');
  if (!validate()) return;
  setLoading(true);
  try {
    const { confirmPassword, ...payload } = form;
    await registerRequest({ ...payload, role });
    setSuccess(
      role === 'STUDENT'
        ? 'Account created. You can sign in now.'
        : 'Request submitted. An administrator must approve your account before you can sign in.'
    );
  } catch (err) {
    setApiError(getErrorMessage(err));
  } finally {
    setLoading(false);
  }
};

const text = (key, props = {}) => (
  <Field label={LABELS[key]} id={`reg-${key}`} error={errors[key]} key={key}>
    <input className="input" value={form[key]} onChange={update(key)} {...props} />
  </Field>
);

return (
  <div className="auth-card auth-card--wide">
    <h1>Create your account</h1>
    <p className="lead">Choose your role. Administrator accounts are created by the college.</p>
    <form className="auth-form" onSubmit={onSubmit} noValidate>
      {success && <Alert tone="success">{success}</Alert>}
      {apiError && <Alert tone="error">{apiError}</Alert>}

      <div className="field">
        <span className="fw-600" id="role-label" style={{ fontSize: 13 }}>I am a</span>
        <div className="role-picker" role="radiogroup" aria-labelledby="role-label">
          {ROLES.map((r) => (
            <button type="button" key={r.value} role="radio" aria-checked={role === r.value} className={`role-option ${role === r.value ? 'active' : ''}`} onClick={()
=> chooseRole(r.value)}>
              {r.label}
              <small>{r.hint}</small>
            </button>
          ))}
        </div>
      </div>

      <div className="divider-label">Account details</div>
      <div className="form-grid">
        <Field label="Full name" id="reg-name" error={errors.name}>
          <input className="input" autoComplete="name" value={form.name} onChange={update('name')} />
        </Field>
        <Field label="Email" id="reg-email" error={errors.email}>
          <input className="input" type="email" autoComplete="email" value={form.email} onChange={update('email')} />
        </Field>
        <Field label="Phone number" id="reg-phone" error={errors.phone}>
          <input className="input" type="tel" autoComplete="tel" value={form.phone} onChange={update('phone')} />
        </Field>
        <Field label="Department" id="reg-department" error={errors.department}>
          <select className="select" value={form.department} onChange={update('department')}>
            <option value="">Select department</option>
            <option>Computer Science</option>
            <option>AI & ML</option>
            <option>Electronics</option>
            <option>Mechanical</option>
            <option>Training & Placement</option>
          </select>
        </Field>
      </div>

      <div className="divider-label">{role === 'STUDENT' ? 'Student details' : role === 'FACULTY' ? 'Faculty details' : 'Placement officer details'}</div>
      <div className="form-grid">
        {Object.keys(EXTRA[role]).map((key) =>
          key === 'semester' ? (
            <Field label={LABELS[key]} id="reg-semester" error={errors.semester} key={key}>
              <select className="select" value={form.semester} onChange={update('semester')}>
                <option value="">Select semester</option>
                {[1, 2, 3, 4, 5, 6, 7, 8].map((n) => <option key={n} value={n}>Semester {n}</option>)}
              </select>
            </Field>
          ) : (
            text(key)
          )
        )}
      </div>

      <div className="divider-label">Security</div>
```

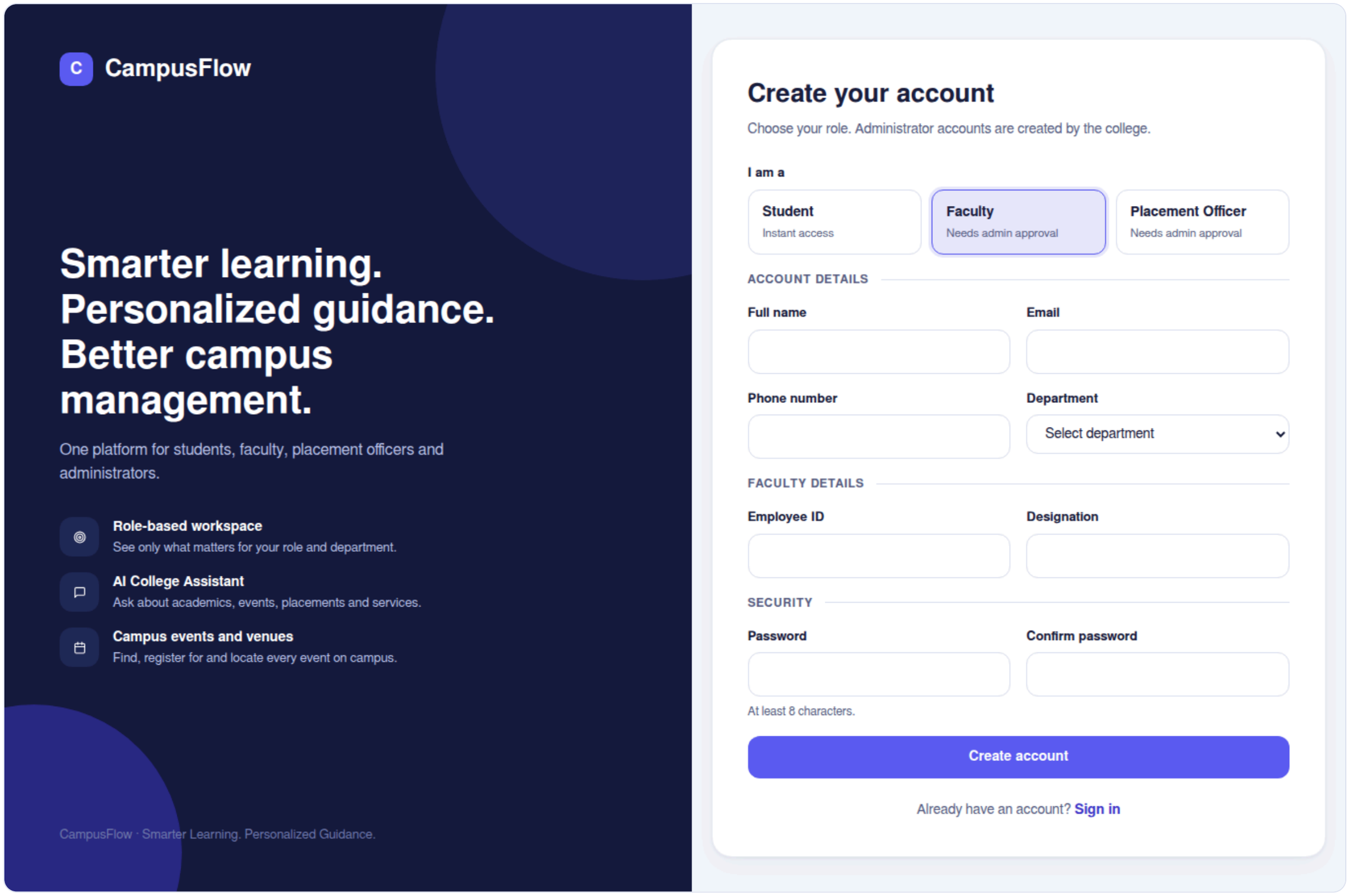
```
        <div className="form-grid">
          <Field label="Password" id="reg-password" error={errors.password} hint="At least 8 characters.">
            <input className="input" type="password" autoComplete="new-password" value={form.password} onChange={update('password')} />
          </Field>
          <Field label="Confirm password" id="reg-confirm" error={errors.confirmPassword}>
            <input className="input" type="password" autoComplete="new-password" value={form.confirmPassword} onChange={update('confirmPassword')} />
          </Field>
        </div>

        <button type="submit" className="btn btn-primary btn-block" disabled={loading}>
          {loading ? 'Creating account...' : 'Create account'}
        </button>
      </form>
      <p className="auth-switch">
        Already have an account? <Link to="/login">Sign in</Link>
      </p>
    </div>
  );
}
```

Interface 3 of 28

Register (faculty, role-specific fields)

Route	`/register`
Role	Public
Page file	`frontend/src/pages/auth/Register.jsx`
Data needed	`POST /api/auth/register`
Behaviour	Faculty and Placement Officer registrations are created as PENDING and cannot sign in until an admin approves them. Role-specific fields change with the selected role. There is no ADMIN option.



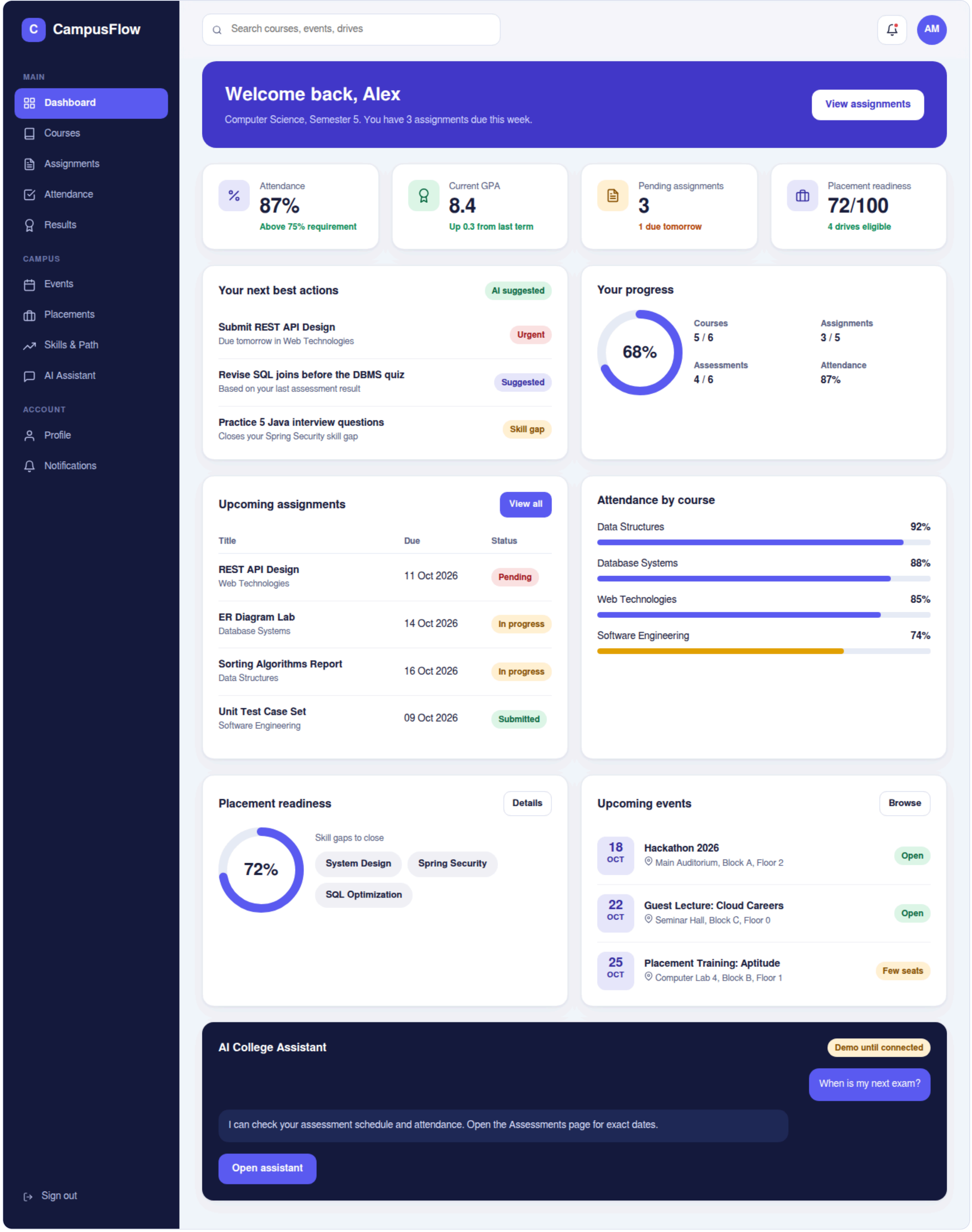
Source code

Same file as an earlier interface: `frontend/src/pages/auth/Register.jsx` (rendered with a different role, props or state).

Interface 4 of 28

Student dashboard

Route	`/student`
Role	STUDENT
Page file	`frontend/src/pages/student/StudentDashboard.jsx`
Data needed	`GET /api/student/dashboard -> summary (attendance %, GPA, pending assignments, readiness score), nextActions[], progress, upcoming assignments, attendance by course, skill gaps, upcoming events`
Behaviour	All numbers are computed by the backend from stored data. Show empty states when a student has no records.



Source code

frontend/src/pages/student/StudentDashboard.jsx

```
import { Link, useNavigate } from 'react-router-dom';
import { FiPercent, FiAward, FiFileText, FiBriefcase, FiMapPin } from 'react-icons/fi';
import Hero from '../../components/Hero';
import StatCard from '../../components/StatCard';
import Card from '../../components/Card';
import Badge from '../../components/Badge';
import DataTable from '../../components/DataTable';
import MeterRow from '../../components/MeterRow';
import RingChart from '../../components/RingChart';
import { useAuth } from '../../context/AuthContext';
import { firstName, formatDate, dayOf, monthShort } from '../../utils/format';
import { student, assignments, attendanceByCourse, nextActions, progress, skillGaps, events } from '../../data/demoData';

const STATUS = {
  PENDING: { tone: 'danger', label: 'Pending' },
```

```
  IN_PROGRESS: { tone: 'warning', label: 'In progress' },
  SUBMITTED: { tone: 'success', label: 'Submitted' },
  GRADED: { tone: 'info', label: 'Graded' },
};

export default function StudentDashboard() {
  const { user } = useAuth();
  const navigate = useNavigate();
  const upcoming = assignments.filter((a) => a.status !== 'GRADED').slice(0, 4);

  return (
    <>
      <Hero
        title={`Welcome back, ${firstName(user?.name)}`}
        subtitle={`${student.department}, Semester ${student.semester}. You have 3 assignments due this week.`}
        action={<button type="button" className="btn btn-light" onClick={() => navigate('/student/assignments')}>View assignments</button>}
      />

      <div className="stat-grid">
        <StatCard icon={FiPercent} label="Attendance" value="87%" hint="Above 75% requirement" />
        <StatCard icon={FiAward} tone="success" label="Current GPA" value="8.4" hint="Up 0.3 from last term" />
        <StatCard icon={FiFileText} tone="warning" label="Pending assignments" value="3" hint="1 due tomorrow" trend="down" />
        <StatCard icon={FiBriefcase} label="Placement readiness" value="72/100" hint="4 drives eligible" />
      </div>

      <div className="grid-2">
        <Card title="Your next best actions" action={<Badge tone="success">AI suggested</Badge>}>
          {nextActions.map((a) => (
        <div className="list-row" key={a.id}>
          <div className="grow">
            <div className="fw-700">{a.title}</div>
            <div className="muted text-sm">{a.hint}</div>
          </div>
          <Badge tone={a.tone}>{a.badge}</Badge>
        </div>
      ))}
    </Card>
    <Card title="Your progress">
      <div className="flex center gap-16 wrap">
        <RingChart value={68} label="Overall completion 68 percent" />
        <dl className="kv grow" style={{ gridTemplateColumns: '1fr 1fr', margin: 0 }}>
          {progress.map((p) => (
            <div key={p.label}>
              <dt>{p.label}</dt>
              <dd>{p.value}</dd>
            </div>
          ))}
        </dl>
      </div>
    </Card>
  </div>

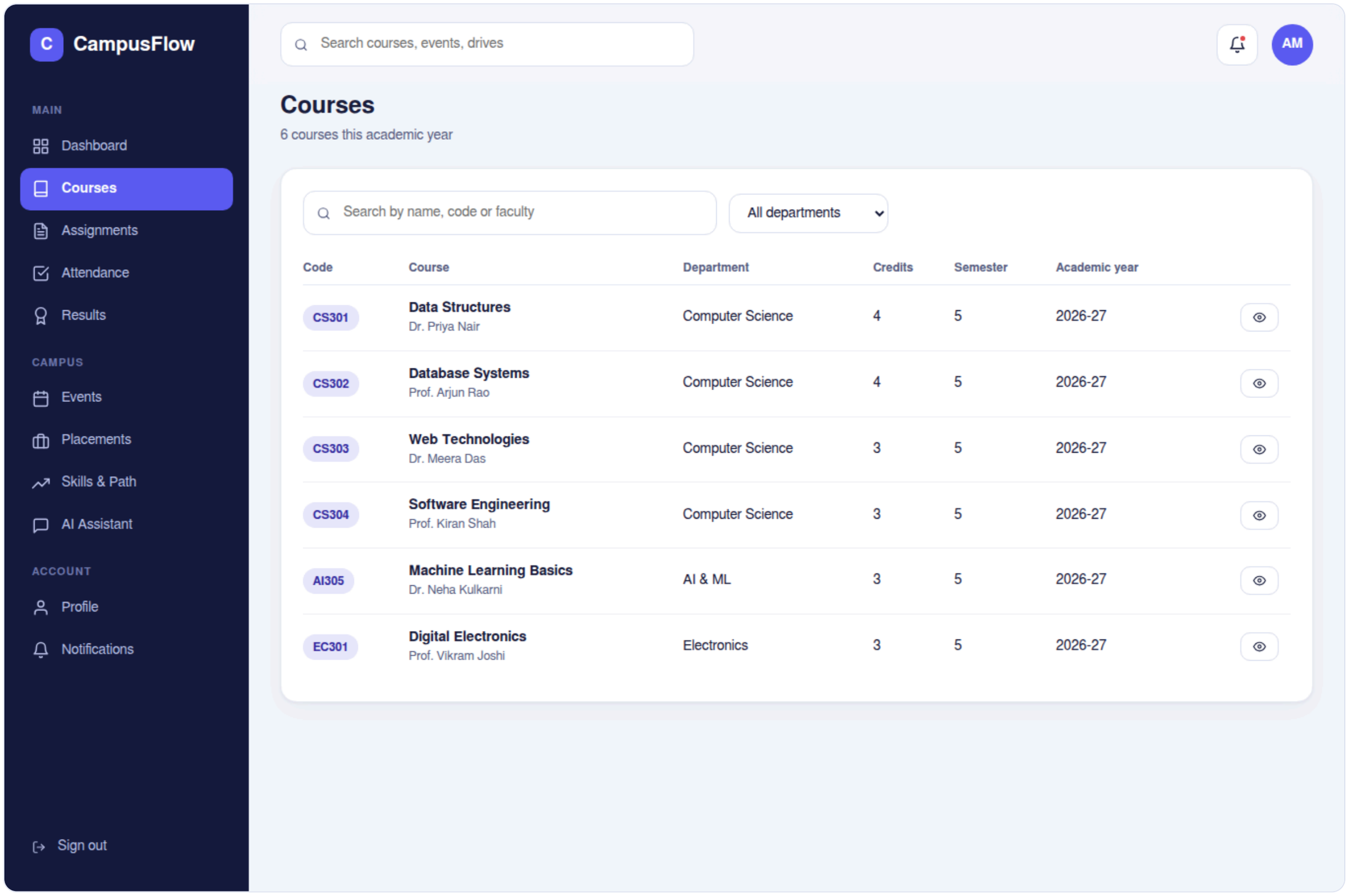
  <div className="grid-2">
    <Card title="Upcoming assignments" action={<Link className="btn btn-primary btn-sm" to="/student/assignments">View all</Link>}>
      <DataTable
        columns={[
          { key: 'title', header: 'Title', render: (r) => <div><div className="fw-700">{r.title}</div><div className="muted text-sm">{r.course}</div></div> },
          { key: 'due', header: 'Due', render: (r) => formatDate(r.due) },
          { key: 'status', header: 'Status', render: (r) => <Badge tone={STATUS[r.status].tone}>{STATUS[r.status].label}</Badge> },
        ]}
        rows={upcoming}
      />
    </Card>
    <Card title="Attendance by course">
      {attendanceByCourse.slice(0, 4).map((c) => (
        <MeterRow key={c.course} label={c.course} value={c.percent} />
      ))}
    </Card>
  </div>

  <div className="grid-2">
    <Card title="Placement readiness" action={<Link className="btn btn-outline btn-sm" to="/student/placements">Details</Link>}>
      <div className="flex center gap-16 wrap">
        <RingChart value={72} label="Placement readiness 72 percent" />
        <div className="grow">
          <p className="muted text-sm" style={{ marginBottom: 8 }}>Skill gaps to close</p>
          <div className="chip-list">
            {skillGaps.map((s) => <span className="chip" key={s}>{s}</span>)}
          </div>
        </div>
      </div>
    </Card>
    <Card title="Upcoming events" action={<Link className="btn btn-outline btn-sm" to="/student/events">Browse</Link>}>
      {events.slice(0, 3).map((e) => (
        <div className="list-row" key={e.id}>
          <div className="date-tile">{dayOf(e.date)}<small>{monthShort(e.date)}</small></div>
          <div className="grow">
            <div className="fw-700">{e.title}</div>
            <div className="muted text-sm"><FiMapPin aria-hidden="true" /> {e.venue}, {e.building}, Floor {e.floor}</div>
          </div>
          <Badge tone={e.total - e.registered <= 10 ? 'warning' : 'success'}>{e.total - e.registered <= 10 ? 'Few seats' : 'Open'}</Badge>
        </div>
      ))}
    </Card>
  </div>

  <Card className="card--dark ai-card" title="AI College Assistant" action={<Badge tone="warning">Demo until connected</Badge>}>
    <div className="chat">
      <div className="bubble me">When is my next exam?</div>
      <div className="bubble">I can check your assessment schedule and attendance. Open the Assessments page for exact dates.</div>
    </div>
    <button type="button" className="btn btn-primary mt-16" onClick={() => navigate('/student/assistant')}>Open assistant</button>
  </Card>
 </>
 );
}
```

Courses (student view)

Route	`/student/courses`
Role	STUDENT
Page file	`frontend/src/pages/shared/Courses.jsx`
Data needed	`GET /api/courses?search=&department=`
Behaviour	Read-only for students (no Add/Edit/Delete buttons).



Source code

`frontend/src/pages/shared/Courses.jsx`

```
import { useMemo, useState } from 'react';
import { FiPlus, FiEdit2, FiTrash2, FiEye } from 'react-icons/fi';
import PageHeader from '../../components/PageHeader';
import Card from '../../components/Card';
import Badge from '../../components/Badge';
import DataTable from '../../components/DataTable';
import SearchInput from '../../components/SearchInput';
import Modal from '../../components/Modal';
import Field from '../../components/Field';
import { courses as demoCourses } from '../../data/demoData';

const EMPTY = { code: '', name: '', department: '', credits: '', semester: '', academicYear: '', faculty: '', description: '' };

// manage=true shows Add / Edit / Delete (faculty and admin). Backend must enforce the same.
export default function Courses({ manage = false }) {
  const [rows, setRows] = useState(demoCourses); // replace with GET /api/courses
  const [query, setQuery] = useState('');
  const [dept, setDept] = useState('');
  const [modal, setModal] = useState(null); // null | 'add' | { delete: course }
  const [form, setForm] = useState(EMPTY);
  const [errors, setErrors] = useState({});

  const departments = [...new Set(rows.map((c) => c.department))];
  const filtered = useMemo(
    () => rows.filter((c) =>
      (!dept || c.department === dept) &&
      `${c.name} ${c.code} ${c.faculty}`.toLowerCase().includes(query.toLowerCase())),
    [rows, query, dept]
  );

  const update = (key) => (e) => setForm((f) => ({ ...f, [key]: e.target.value }));
  const save = () => {
    const next = {};
    if (!form.code.trim()) next.code = 'Course code is required.';
    if (!form.name.trim()) next.name = 'Course name is required.';
```

```
    if (!form.department) next.department = 'Select a department.';
    if (!(Number(form.credits) > 0)) next.credits = 'Enter credits.';
    setErrors(next);
    if (Object.keys(next).length) return;
    setRows((r) => [...r, { ...form, id: Date.now(), credits: Number(form.credits), semester: Number(form.semester) || 1 }]); // replace with POST /api/courses
    setForm(EMPTY);
    setModal(null);
  };
  const remove = () => {
    setRows((r) => r.filter((c) => c.id !== modal.delete.id)); // replace with DELETE /api/courses/{id}
    setModal(null);
  };

  const columns = [
    { key: 'code', header: 'Code', render: (r) => <Badge tone="info">{r.code}</Badge> },
    { key: 'name', header: 'Course', render: (r) => <div><div className="fw-700">{r.name}</div><div className="muted text-sm">{r.faculty}</div></div> },
    { key: 'department', header: 'Department' },
    { key: 'credits', header: 'Credits' },
    { key: 'semester', header: 'Semester' },
    { key: 'academicYear', header: 'Academic year' },
    { key: 'actions', header: '', className: 'actions', render: (r) => (
      <>
        <button type="button" className="btn btn-outline btn-sm" aria-label={`View ${r.name}`}><FiEye /></button>
        {manage && <button type="button" className="btn btn-outline btn-sm" aria-label={`Edit ${r.name}`}><FiEdit2 /></button>}
        {manage && <button type="button" className="btn btn-danger btn-sm" aria-label={`Delete ${r.name}`} onClick={() => setModal({ delete: r })}><FiTrash2 />
</button>}
      </>
    ) },
  ];

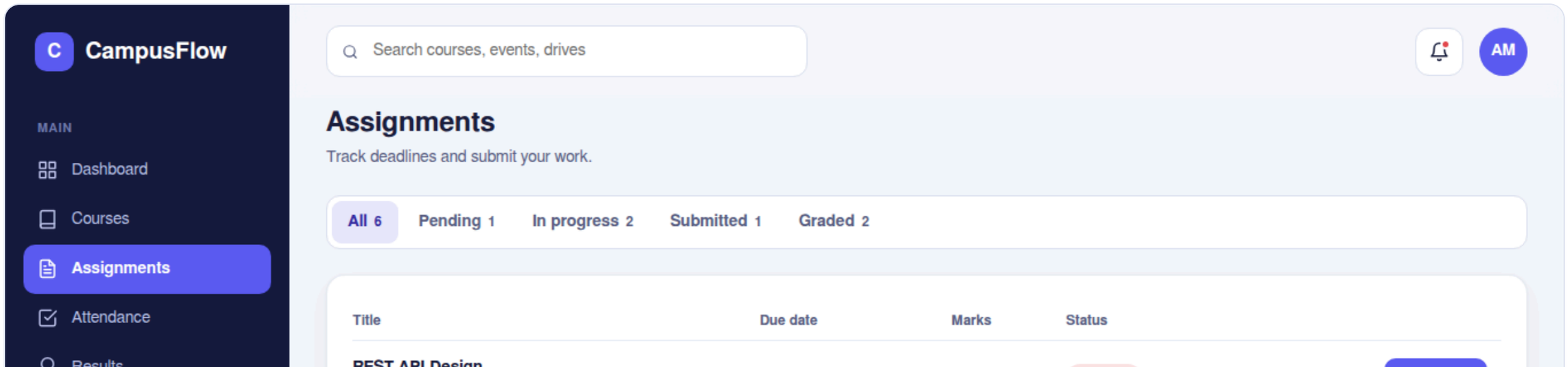
  return (
    <>
      <PageHeader
        title={manage ? 'Course management' : 'Courses'}
        subtitle={`${rows.length} courses this academic year`}
        actions={manage && <button type="button" className="btn btn-primary" onClick={() => setModal('add')}><FiPlus /> Add course</button>}
      />
      <Card>
        <div className="toolbar" style={{ marginBottom: 16 }}>
          <SearchInput value={query} onChange={setQuery} placeholder="Search by name, code or faculty" label="Search courses" />
          <select className="select" value={dept} onChange={(e) => setDept(e.target.value)} aria-label="Filter by department">
            <option value="">All departments</option>
            {departments.map((d) => <option key={d}>{d}</option>)}
          </select>
        </div>
        <DataTable columns={columns} rows={filtered} emptyTitle="No courses found" emptyText="Try a different search or filter." />
      </Card>

      <Modal
        open={modal === 'add'}
        title="Add course"
        onClose={() => setModal(null)}
        footer={<><button type="button" className="btn btn-outline" onClick={() => setModal(null)}>Cancel</button><button type="button" className="btn btn-primary"
onClick={save}>Save course</button></>}
      >
        <div className="form-grid">
          <Field label="Course code" id="c-code" error={errors.code}><input className="input" value={form.code} onChange={update('code')} /></Field>
          <Field label="Course name" id="c-name" error={errors.name}><input className="input" value={form.name} onChange={update('name')} /></Field>
          <Field label="Department" id="c-dept" error={errors.department}>
            <select className="select" value={form.department} onChange={update('department')}>
              <option value="">Select department</option>
              {departments.map((d) => <option key={d}>{d}</option>)}
            </select>
          </Field>
          <Field label="Credits" id="c-credits" error={errors.credits}><input className="input" type="number" min="1" value={form.credits} onChange={update('credits')}
/></Field>
          <Field label="Semester" id="c-sem"><input className="input" type="number" min="1" max="8" value={form.semester} onChange={update('semester')} /></Field>
          <Field label="Academic year" id="c-year"><input className="input" placeholder="2026-27" value={form.academicYear} onChange={update('academicYear')} /></Field>
          <Field label="Assigned faculty" id="c-faculty"><input className="input" value={form.faculty} onChange={update('faculty')} /></Field>
        </div>
        <div style={{ marginTop: 16 }}>
          <Field label="Description" id="c-desc"><textarea className="textarea" value={form.description} onChange={update('description')} /></Field>
        </div>
      </Modal>

      <Modal
        open={Boolean(modal?.delete)}
        title="Delete course"
        onClose={() => setModal(null)}
        footer={<><button type="button" className="btn btn-outline" onClick={() => setModal(null)}>Cancel</button><button type="button" className="btn btn-danger"
onClick={remove}>Delete</button></>}
      >
        <p>Delete <b>{modal?.delete?.name}</b>? This cannot be undone.</p>
      </Modal>
    </>
  );
}
```

Assignments (student)

Route	`/student/assignments`
Role	STUDENT
Page file	`frontend/src/pages/shared/Assignments.jsx`
Data needed	`GET /api/assignments/me` `POST /api/assignments/{id}/submissions`
Behaviour	Tabs filter by status. Submit opens the submission flow for pending or in-progress work.



Source code

`frontend/src/pages/shared/Assignments.jsx`

```
import { useState } from 'react';
import { FiPlus, FiUpload } from 'react-icons/fi';
import PageHeader from '../../components/PageHeader';
import Card from '../../components/Card';
import Badge from '../../components/Badge';
import Tabs from '../../components/Tabs';
import DataTable from '../../components/DataTable';
import ProgressBar from '../../components/ProgressBar';
import { formatDate } from '../../utils/format';
import { assignments, facultyStats } from '../../data/demoData';

const STATUS = {
  PENDING: { tone: 'danger', label: 'Pending' },
  IN_PROGRESS: { tone: 'warning', label: 'In progress' },
  SUBMITTED: { tone: 'success', label: 'Submitted' },
  GRADED: { tone: 'info', label: 'Graded' },
};

// mode: 'student' (view and submit) | 'faculty' (create and review)
export default function Assignments({ mode = 'student' }) {
  const [tab, setTab] = useState('ALL');
  const faculty = mode === 'faculty';
  const rows = tab === 'ALL' ? assignments : assignments.filter((a) => a.status === tab);
  const count = (s) => assignments.filter((a) => a.status === s).length;

  const studentColumns = [
    { key: 'title', header: 'Title', render: (r) => <div><div className="fw-700">{r.title}</div><div className="muted text-sm">{r.course}</div></div> },
    { key: 'due', header: 'Due date', render: (r) => formatDate(r.due) },
    { key: 'maxMarks', header: 'Marks', render: (r) => (r.marks !== null ? `${r.marks} / ${r.maxMarks}` : `- / ${r.maxMarks}`) },
    { key: 'status', header: 'Status', render: (r) => <Badge tone={STATUS[r.status].tone}>{STATUS[r.status].label}</Badge> },
    { key: 'action', header: '', className: 'actions', render: (r) => (
      ['PENDING', 'IN_PROGRESS'].includes(r.status)
        ? <button type="button" className="btn btn-primary btn-sm"><FiUpload /> Submit</button>
        : <button type="button" className="btn btn-outline btn-sm">View</button>
```

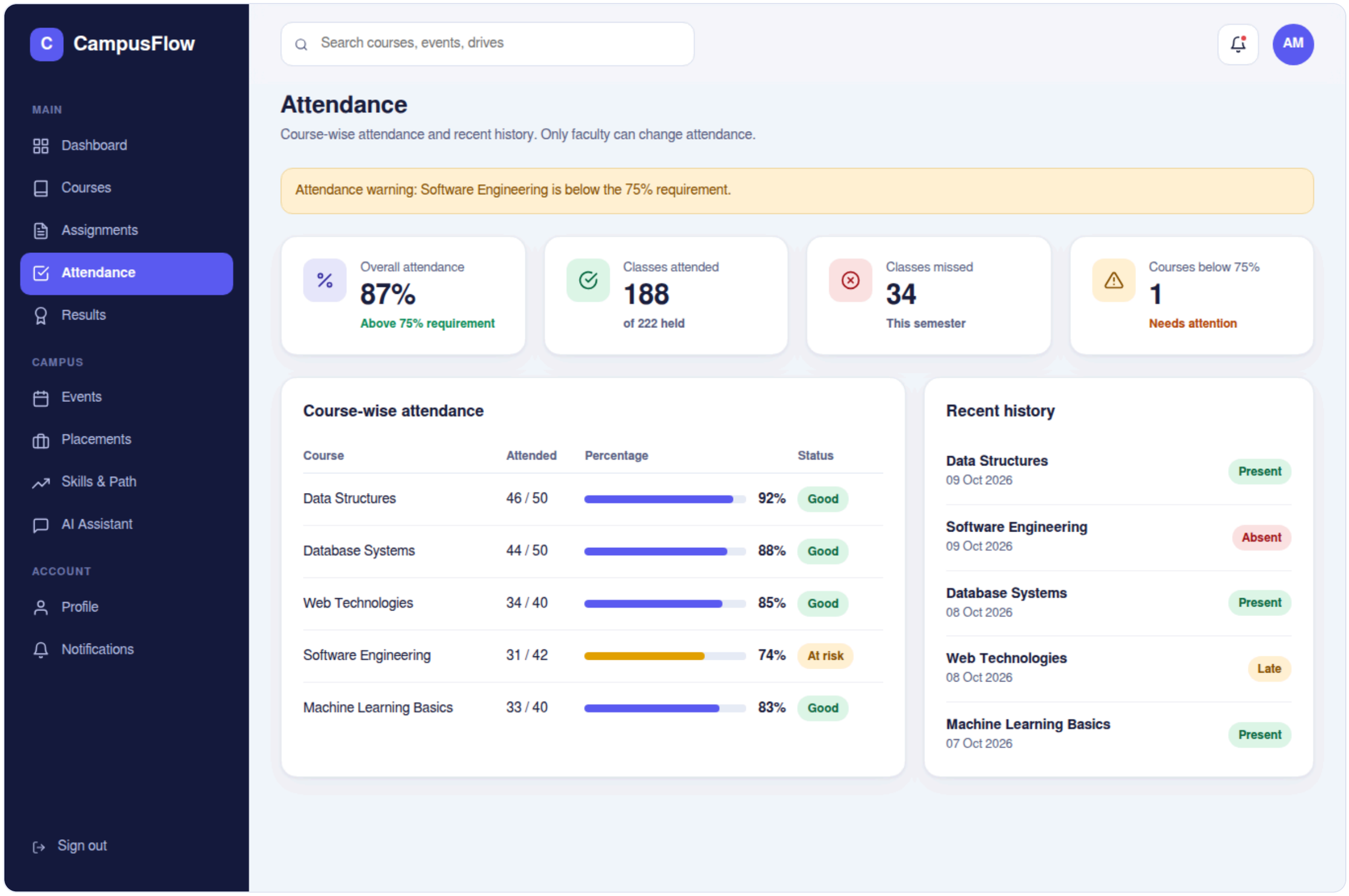
```
    ) },
  ];

  const facultyColumns = [
    { key: 'title', header: 'Assignment', render: (r) => <div><div className="fw-700">{r.title}</div><div className="muted text-sm">{r.course}</div></div> },
    { key: 'due', header: 'Due date', render: (r) => formatDate(r.due) },
    { key: 'submitted', header: 'Submissions', render: (r) => (
      <div className="flex center gap-12"><div style={{ minWidth: 110 }}><ProgressBar value={Math.round((r.submitted / r.total) * 100)} warnBelow={50} label=
{`${r.title} submissions`} /></div><b>{r.submitted}/{r.total}</b></div>
    ) },
    { key: 'maxMarks', header: 'Max marks' },
    { key: 'action', header: '', className: 'actions', render: () => (
      <><button type="button" className="btn btn-outline btn-sm">Edit</button><button type="button" className="btn btn-primary btn-sm">Review</button></>
    ) },
  ];

  return (
    <>
      <PageHeader
        title="Assignments"
        subtitle={faculty ? 'Create assignments and review student submissions.' : 'Track deadlines and submit your work.'}
        actions={faculty && <button type="button" className="btn btn-primary"><FiPlus /> Create assignment</button>}
      />
      <Tabs
        value={tab}
        onChange={setTab}
        tabs={[
          { value: 'ALL', label: 'All', count: assignments.length },
          { value: 'PENDING', label: 'Pending', count: count('PENDING') },
          { value: 'IN_PROGRESS', label: 'In progress', count: count('IN_PROGRESS') },
          { value: 'SUBMITTED', label: 'Submitted', count: count('SUBMITTED') },
          { value: 'GRADED', label: 'Graded', count: count('GRADED') },
        ]}
      />
      <Card>
        <DataTable columns={faculty ? facultyColumns : studentColumns} rows={rows} emptyTitle="No assignments" emptyText="Nothing matches this filter." />
      </Card>
      {faculty && (
        <Card title="Submissions waiting for review">
          <DataTable
            columns={[
              { key: 'student', header: 'Student' },
              { key: 'assignment', header: 'Assignment', render: (r) => <div><div className="fw-700">{r.assignment}</div><div className="muted text-sm">{r.course}</div>
</div> },
              { key: 'submitted', header: 'Submitted' },
              { key: 'action', header: '', className: 'actions', render: () => <button type="button" className="btn btn-primary btn-sm">Grade</button> },
            ]}
            rows={facultyStats.toReview}
          />
        </Card>
      )}
    </>
  );
}
```

Attendance (student)

Route	`/student/attendance`
Role	STUDENT
Page file	`frontend/src/pages/student/Attendance.jsx`
Data needed	`GET /api/attendance/me/summary` `GET /api/attendance/me/history`
Behaviour	View only. Students can never edit attendance. Courses under 75% show the warning banner.



Source code

`frontend/src/pages/student/Attendance.jsx`

```
import { FiPercent, FiCheckCircle, FiXCircle, FiAlertTriangle } from 'react-icons/fi';
import PageHeader from '../../components/PageHeader';
import StatCard from '../../components/StatCard';
import Card from '../../components/Card';
import Badge from '../../components/Badge';
import Alert from '../../components/Alert';
import DataTable from '../../components/DataTable';
import ProgressBar from '../../components/ProgressBar';
import { formatDate } from '../../utils/format';
import { attendanceByCourse, attendanceHistory } from '../../data/demoData';

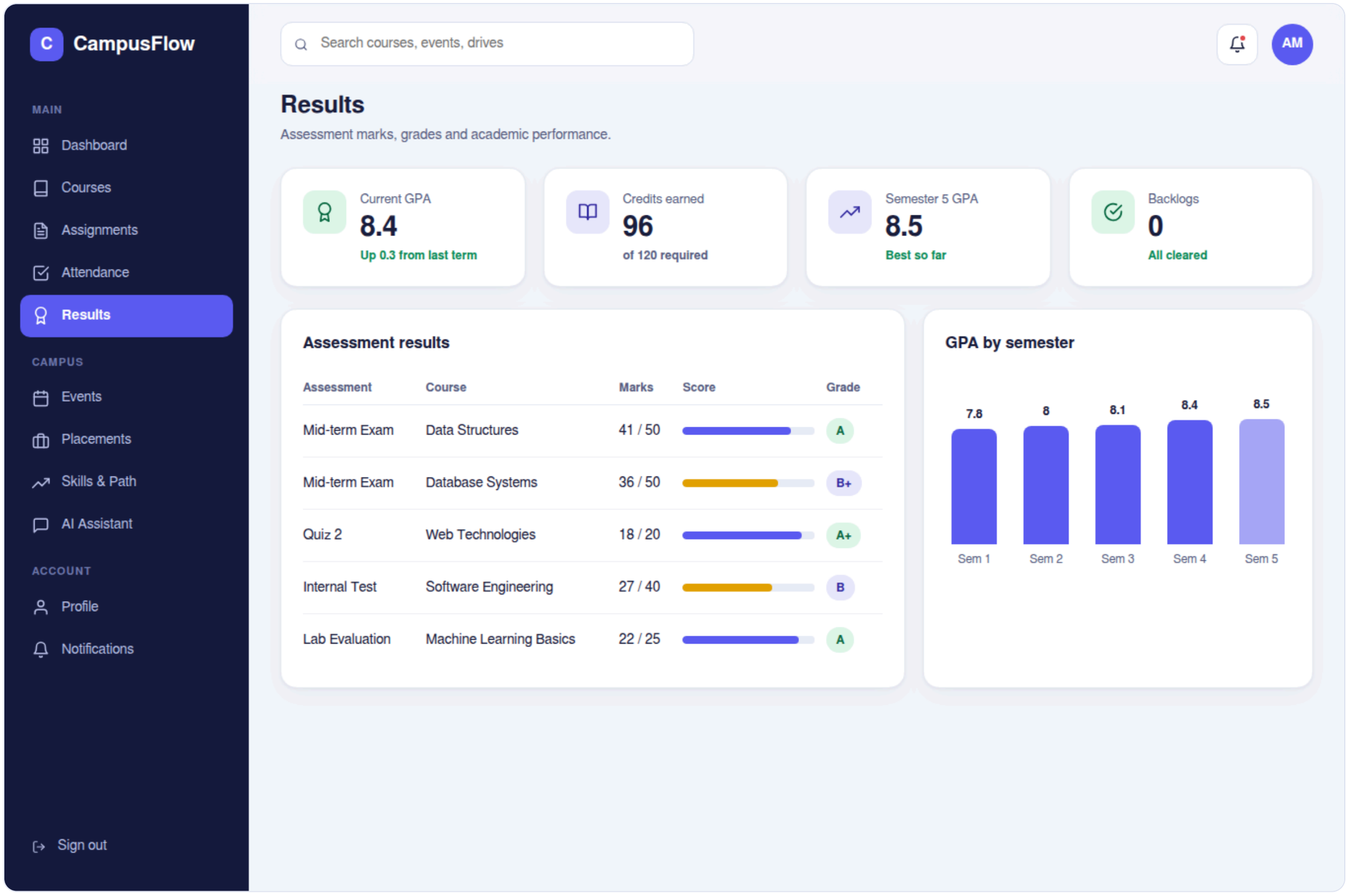
const TONE = { Present: 'success', Absent: 'danger', Late: 'warning' };

export default function Attendance() {
  const low = attendanceByCourse.filter((c) => c.percent < 75);
  return (
    <>
      <PageHeader title="Attendance" subtitle="Course-wise attendance and recent history. Only faculty can change attendance." />
      {low.length > 0 && (
        <Alert tone="warning">Attendance warning: {low.map((c) => c.course).join(', ')} is below the 75% requirement.</Alert>
      )}
      <div className="stat-grid">
        <StatCard icon={FiPercent} label="Overall attendance" value="87%" hint="Above 75% requirement" />
        <StatCard icon={FiCheckCircle} tone="success" label="Classes attended" value="188" hint="of 222 held" trend="flat" />
        <StatCard icon={FiXCircle} tone="danger" label="Classes missed" value="34" hint="This semester" trend="flat" />
        <StatCard icon={FiAlertTriangle} tone="warning" label="Courses below 75%" value={low.length} hint="Needs attention" trend="down" />
      </div>
      <div className="grid-main">
        <Card title="Course-wise attendance">
          <DataTable
            columns={[
              { key: 'course', header: 'Course' },
              { key: 'attended', header: 'Attended', render: (r) => `${r.attended} / ${r.total}` },
              { key: 'percent', header: 'Percentage', render: (r) => (
```

```
              <div className="flex center gap-12"><div className="grow" style={{ minWidth: 120 }}><ProgressBar value={r.percent} label={`${r.course} attendance`} />
</div><b>{r.percent}%</b></div>
              ) },
              { key: 'status', header: 'Status', render: (r) => <Badge tone={r.percent < 75 ? 'warning' : 'success'}>{r.percent < 75 ? 'At risk' : 'Good'}</Badge> },
            ]}
            rows={attendanceByCourse}
            rowKey="course"
          />
        </Card>
        <Card title="Recent history">
          {attendanceHistory.map((h) => (
            <div className="list-row" key={h.id}>
              <div className="grow">
                <div className="fw-700">{h.course}</div>
                <div className="muted text-sm">{formatDate(h.date)}</div>
              </div>
              <Badge tone={TONE[h.status]}>{h.status}</Badge>
            </div>
          ))}
        </Card>
      </div>
    </>
  );
}
```

Results

Route	`/student/results`
Role	STUDENT
Page file	`frontend/src/pages/student/Results.jsx`
Data needed	`GET /api/results/me` `GET /api/results/me/gpa-by-semester`
Behaviour	View only.



Source code

`frontend/src/pages/student/Results.jsx`

```
import { FiAward, FiBookOpen, FiTrendingUp, FiCheckCircle } from 'react-icons/fi';
import PageHeader from '../../components/PageHeader';
import StatCard from '../../components/StatCard';
import Card from '../../components/Card';
import Badge from '../../components/Badge';
import DataTable from '../../components/DataTable';
import BarChart from '../../components/BarChart';
import ProgressBar from '../../components/ProgressBar';
import { results, semesterGpa } from '../../data/demoData';

const gradeTone = (g) => (g.startsWith('A') ? 'success' : g.startsWith('B') ? 'info' : 'warning');

export default function Results() {
  return (
    <>
      <PageHeader title="Results" subtitle="Assessment marks, grades and academic performance." />
      <div className="stat-grid">
        <StatCard icon={FiAward} tone="success" label="Current GPA" value="8.4" hint="Up 0.3 from last term" />
        <StatCard icon={FiBookOpen} label="Credits earned" value="96" hint="of 120 required" trend="flat" />
        <StatCard icon={FiTrendingUp} label="Semester 5 GPA" value="8.5" hint="Best so far" />
        <StatCard icon={FiCheckCircle} tone="success" label="Backlogs" value="0" hint="All cleared" />
      </div>
      <div className="grid-main">
        <Card title="Assessment results">
          <DataTable
            columns={[
              { key: 'assessment', header: 'Assessment' },
              { key: 'course', header: 'Course' },
              { key: 'marks', header: 'Marks', render: (r) => `${r.marks} / ${r.total}` },
              { key: 'percent', header: 'Score', render: (r) => <div style={{ minWidth: 110 }}><ProgressBar value={Math.round((r.marks / r.total) * 100)} label=
{`${r.assessment} score`} /></div> },
              { key: 'grade', header: 'Grade', render: (r) => <Badge tone={gradeTone(r.grade)}>{r.grade}</Badge> },
            ]}
            rows={results}
```

```
          />
        </Card>
        <Card title="GPA by semester">
          <BarChart data={semesterGpa} max={10} />
        </Card>
      </div>
    </>
  );
}
```

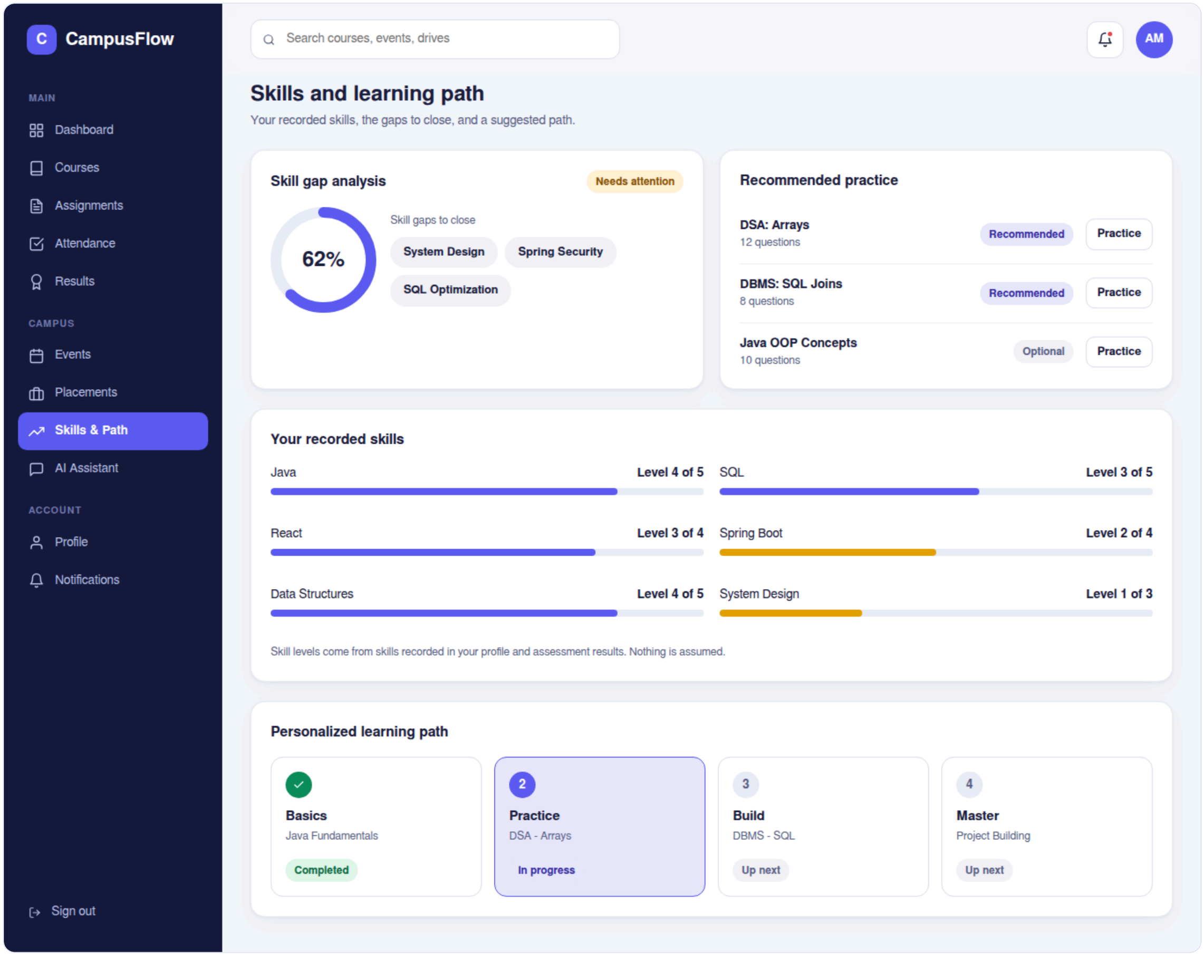
Interface 9 of 28

Skills and learning path

Route	`/student/skills`
Role	STUDENT
Page file	`frontend/src/pages/student/Skills.jsx`

Data needed	`GET /api/skills/me`
	`GET /api/skills/me/gaps`
	`GET /api/learning-path/me`
	`GET /api/recommendations/practice`

Behaviour	Rule-based Java logic only. Shows only recorded skills, never invented achievements.



Source code

frontend/src/pages/student/Skills.jsx

```
import { FiCheck } from 'react-icons/fi';
import PageHeader from '../../components/PageHeader';
import Card from '../../components/Card';
import Badge from '../../components/Badge';
import MeterRow from '../../components/MeterRow';
import RingChart from '../../components/RingChart';
import { skills, skillGaps, recommendedPractice, learningPath } from '../../data/demoData';

export default function Skills() {
  return (
    <>
      <PageHeader title="Skills and learning path" subtitle="Your recorded skills, the gaps to close, and a suggested path." />
      <div className="grid-2">
        <Card title="Skill gap analysis" action={<Badge tone="warning">Needs attention</Badge>}>
          <div className="flex center gap-16 wrap">
            <RingChart value={62} label="Overall skill completion 62 percent" />
            <div className="grow">
              <p className="muted text-sm" style={{ marginBottom: 8 }}>Skill gaps to close</p>
              <div className="chip-list">{skillGaps.map((s) => <span className="chip" key={s}>{s}</span>)}</div>
            </div>
          </div>
        </Card>
```

```
      <Card title="Recommended practice">
        {recommendedPractice.map((p) => (
          <div className="list-row" key={p.id}>
            <div className="grow">
              <div className="fw-700">{p.title}</div>
              <div className="muted text-sm">{p.hint}</div>
            </div>
            <Badge tone={p.badge === 'Recommended' ? 'info' : 'neutral'}>{p.badge}</Badge>
            <button type="button" className="btn btn-outline btn-sm">Practice</button>
          </div>
        ))}
      </Card>
    </div>

    <Card title="Your recorded skills">
      <div className="grid-2">
        {skills.map((s) => (
          <MeterRow key={s.name} label={s.name} value={Math.round((s.level / s.target) * 100)} display={`Level ${s.level} of ${s.target}`} warnBelow={60} />
        ))}
      </div>
      <p className="muted text-sm mt-16">Skill levels come from skills recorded in your profile and assessment results. Nothing is assumed.</p>
    </Card>

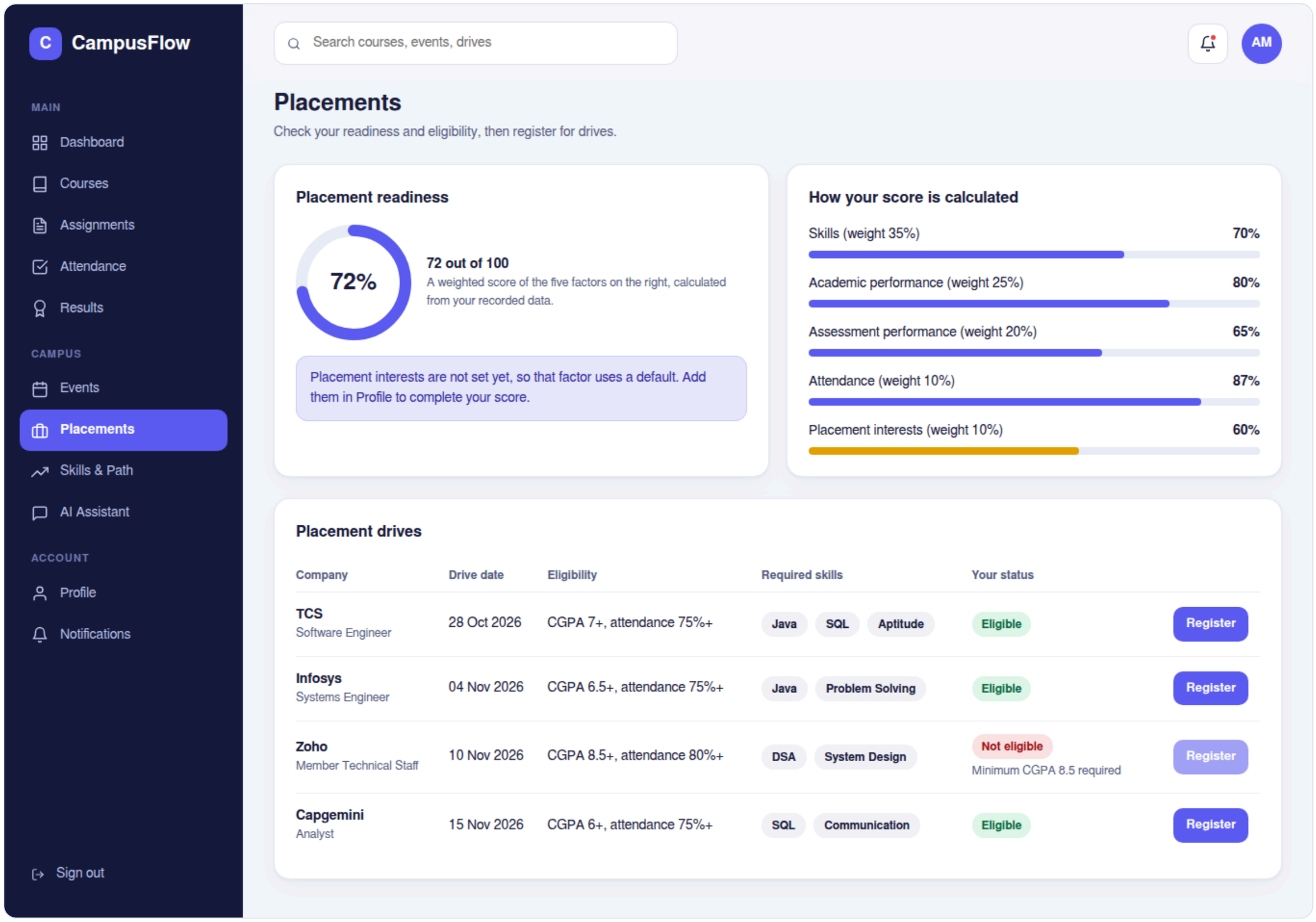
    <Card title="Personalized learning path">
      <div className="steps">
        {learningPath.map((s, i) => (
          <div className={`step ${s.state === 'done' ? 'done' : s.state === 'current' ? 'current' : ''}`} key={s.step}>
            <div className="step__no">{s.state === 'done' ? <FiCheck aria-hidden="true" /> : i + 1}</div>
            <h4>{s.step}</h4>
            <p className="muted text-sm">{s.topic}</p>
            <div className="mt-16">
              <Badge tone={s.state === 'done' ? 'success' : s.state === 'current' ? 'info' : 'neutral'}>
                {s.state === 'done' ? 'Completed' : s.state === 'current' ? 'In progress' : 'Up next'}
              </Badge>
            </div>
          </div>
        ))}
      </div>
    </Card>
  </>
  );
}
```

Placements (student)

Route	`/student/placements`
Role	STUDENT
Page file	`frontend/src/pages/student/Placements.jsx`

Data needed	`GET /api/placements/readiness/me -> score, factors[{label,value,weight}], missingData[]`
	`GET /api/placements/drives -> each drive includes eligible:boolean and reason`
	`POST /api/placements/drives/{id}/register`

Behaviour	The card "How your score is calculated" must reflect the real weights used by the backend and list missing data.



Source code

`frontend/src/pages/student/Placements.jsx`

```
import { useState } from 'react';
import PageHeader from '../../components/PageHeader';
import Card from '../../components/Card';
import Badge from '../../components/Badge';
import Alert from '../../components/Alert';
import DataTable from '../../components/DataTable';
import MeterRow from '../../components/MeterRow';
import RingChart from '../../components/RingChart';
import { formatDate } from '../../utils/format';
import { drives, readinessFactors } from '../../data/demoData';

export default function Placements() {
  const [registered, setRegistered] = useState([]);
  const register = (id) => setRegistered((r) => [...r, id]); // replace with POST /api/placements/drives/{id}/register

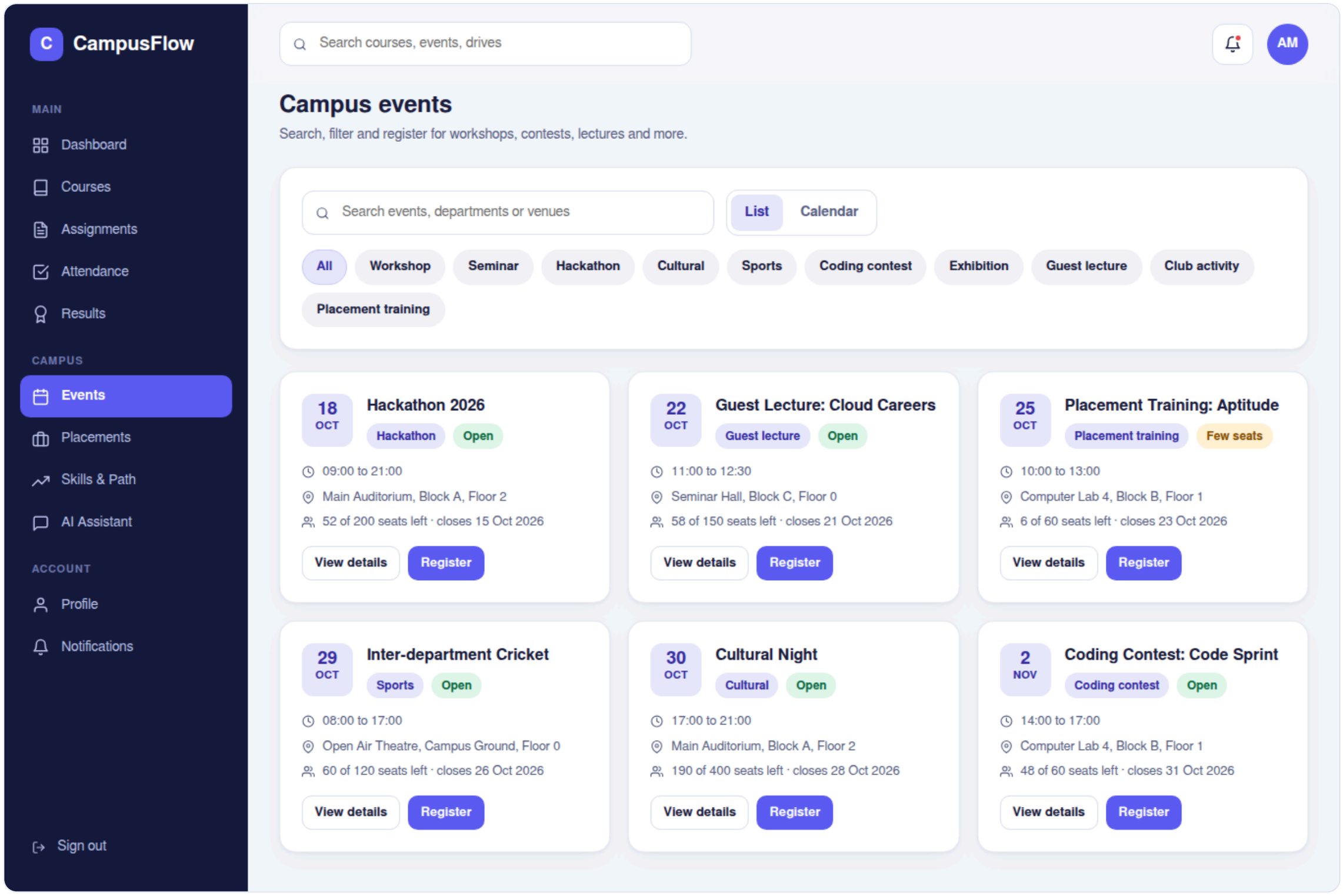
  return (
    <>
      <PageHeader title="Placements" subtitle="Check your readiness and eligibility, then register for drives." />
      <div className="grid-2">
        <Card title="Placement readiness">
          <div className="flex center gap-16 wrap">
            <RingChart value={72} label="Placement readiness 72 percent" />
            <div className="grow">
              <p className="fw-700">72 out of 100</p>
              <p className="muted text-sm">A weighted score of the five factors on the right, calculated from your recorded data.</p>
            </div>
          </div>
          <div className="mt-16">
            <Alert tone="info">Placement interests are not set yet, so that factor uses a default. Add them in Profile to complete your score.</Alert>
          </div>
```

```
        </Card>
        <Card title="How your score is calculated">
          {readinessFactors.map((f) => (
            <MeterRow key={f.label} label={`${f.label} (weight ${f.weight}%)`} value={f.value} warnBelow={65} />
          ))}
        </Card>
      </div>

      <Card title="Placement drives">
        <DataTable
          columns={[
            { key: 'company', header: 'Company', render: (r) => <div><div className="fw-700">{r.company}</div><div className="muted text-sm">{r.role}</div></div> },
            { key: 'date', header: 'Drive date', render: (r) => formatDate(r.date) },
            { key: 'criteria', header: 'Eligibility', render: (r) => `CGPA ${r.minCgpa}+, attendance ${r.minAttendance}%+` },
            { key: 'skills', header: 'Required skills', render: (r) => <div className="chip-list">{r.skills.map((s) => <span className="chip" key={s} style={{ padding:
'3px 10px', fontSize: 12 }}>{s}</span>)}</div> },
            { key: 'status', header: 'Your status', render: (r) => r.eligible ? <Badge tone="success">Eligible</Badge> : <div><Badge tone="danger">Not eligible</Badge>
<div className="muted text-sm" style={{ marginTop: 4 }}>{r.reason}</div></div> },
            { key: 'action', header: '', className: 'actions', render: (r) => registered.includes(r.id)
              ? <Badge tone="info">Registered</Badge>
              : <button type="button" className="btn btn-primary btn-sm" disabled={!r.eligible} onClick={() => register(r.id)}>Register</button> },
          ]}
          rows={drives}
        />
      </Card>
    </>
  );
}
```

Campus events (list)

Route	`/student/events`
Role	STUDENT, FACULTY, PLACEMENT_OFFICER, ADMIN (view); register is STUDENT
Page file	`frontend/src/pages/shared/Events.jsx`
Data needed	`GET /api/events?search=&category=` `POST /api/events/{id}/register`
Behaviour	Search box, category chips and List/Calendar toggle. Backend blocks duplicate registration, registration after the deadline, and full events.



Source code

`frontend/src/pages/shared/Events.jsx`

```
import { useMemo, useState } from 'react';
import { useLocation } from 'react-router-dom';
import { FiPlus } from 'react-icons/fi';
import PageHeader from '../../components/PageHeader';
import Card from '../../components/Card';
import Tabs from '../../components/Tabs';
import EventCard from '../../components/EventCard';
import EmptyState from '../../components/EmptyState';
import SearchInput from '../../components/SearchInput';
import Alert from '../../components/Alert';
import { events as demoEvents, eventCategories } from '../../data/demoData';
import { parseDate } from '../../utils/format';

const DOW = ['Sun', 'Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat'];

function MonthCalendar({ year, month, events }) {
  const first = new Date(year, month, 1).getDay();
  const days = new Date(year, month + 1, 0).getDate();
  const cells = [...Array(first).fill(null), ...Array.from({ length: days }, (_, i) => i + 1)];
  const byDay = (d) => events.filter((e) => { const p = parseDate(e.date); return p.year === year && p.month === month && p.day === d; });
  return (
    <div className="calendar" role="grid" aria-label="Event calendar">
      {DOW.map((d) => <div className="calendar__dow" key={d}>{d}</div>)}
      {cells.map((d, i) => (
        <div className={`calendar__cell ${d ? '' : 'blank'}`} key={i}>
          {d && <span className="calendar__day">{d}</span>}
          {d && byDay(d).map((e) => <span className="calendar__event" key={e.id} title={e.title}>{e.title}</span>)}
        </div>
      ))}
    </div>
  );
}

// manage=true (faculty/admin) adds Create event, Approve and Edit actions.
```

```
export default function Events({ manage = false }) {
  const { pathname } = useLocation();
  const basePath = '/' + pathname.split('/')[1];
  const [view, setView] = useState('list');
  const [query, setQuery] = useState('');
  const [category, setCategory] = useState('All');
  const [registered, setRegistered] = useState([]);

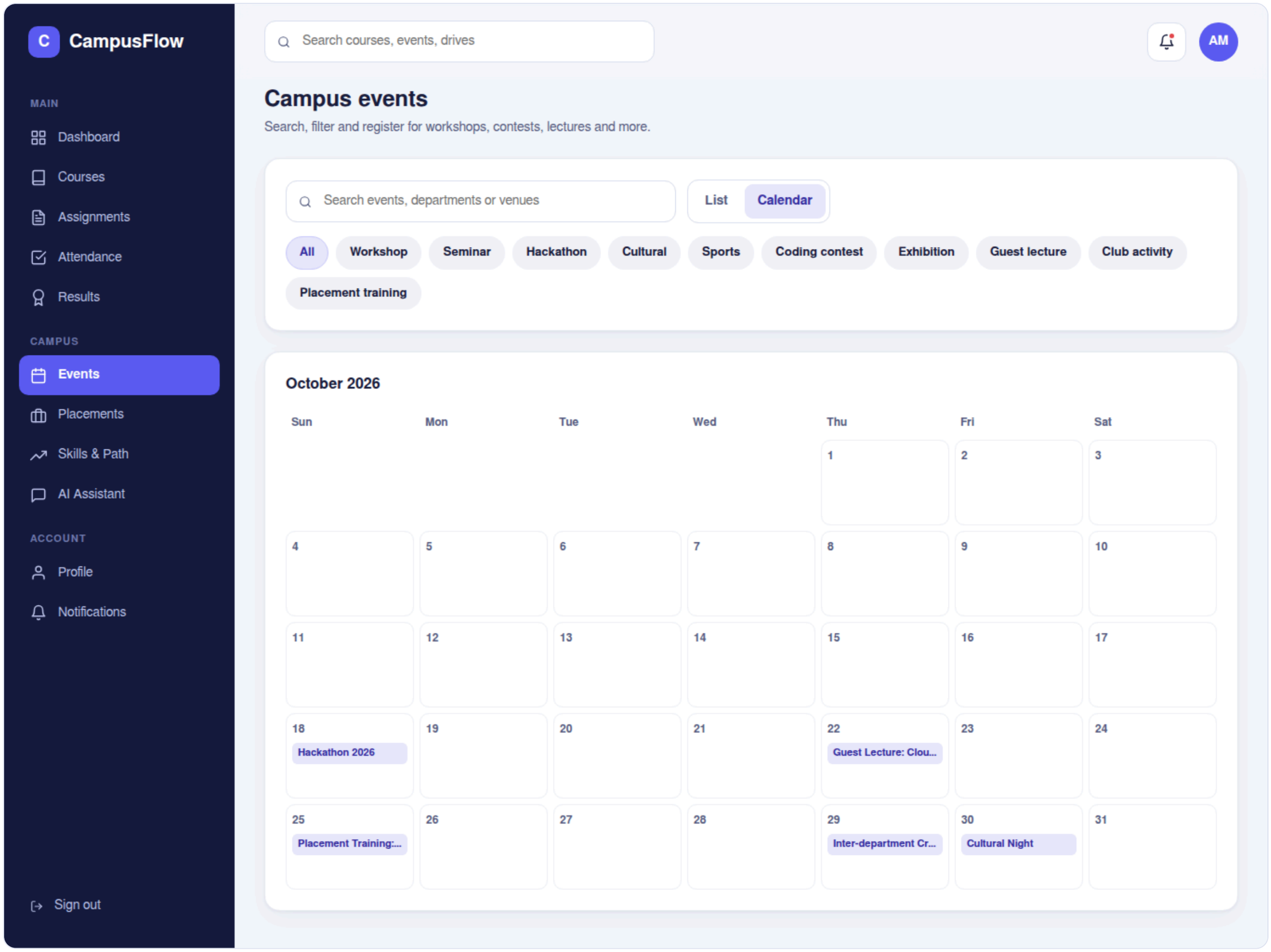
  const filtered = useMemo(
    () => demoEvents.filter((e) =>
      (category === 'All' || e.category.toLowerCase() === category.toLowerCase()) &&
      `${e.title} ${e.department} ${e.venue}`.toLowerCase().includes(query.toLowerCase())), // replace with GET /api/events?search=&category=
    [query, category]
  );

  return (
    <>
      <PageHeader
        title={manage ? 'Manage events' : 'Campus events'}
        subtitle="Search, filter and register for workshops, contests, lectures and more."
        actions={manage && <button type="button" className="btn btn-primary"><FiPlus /> Create event</button>}
      />
      {registered.length > 0 && <Alert tone="success">You are registered for {registered.length} event{registered.length > 1 ? 's' : ''}. Reminders will appear in
Notifications.</Alert>}
      <Card>
        <div className="toolbar">
          <SearchInput value={query} onChange={setQuery} placeholder="Search events, departments or venues" label="Search events" />
          <Tabs value={view} onChange={setView} tabs={[{ value: 'list', label: 'List' }, { value: 'calendar', label: 'Calendar' }]} />
        </div>
        <div className="chip-list" style={{ marginTop: 14 }}>
          {['All', ...eventCategories].map((c) => (
            <button type="button" key={c} className={`chip ${category === c ? 'active' : ''}`} onClick={() => setCategory(c)}>{c}</button>
          ))}
        </div>
      </Card>

      {view === 'calendar' ? (
        <Card title="October 2026"><MonthCalendar year={2026} month={9} events={filtered} /></Card>
      ) : filtered.length === 0 ? (
        <Card><EmptyState title="No events found" text="Try another category or clear the search." /></Card>
      ) : (
        <div className="event-grid">
          {filtered.map((e) => (
            <EventCard key={e.id} event={e} basePath={basePath} manage={manage} onRegister={(ev) => setRegistered((r) => [...new Set([...r, ev.id])])} />
          ))}
        </div>
      )}
    </>
  );
}
```

Campus events (calendar view)

Route	`/student/events`
Role	Same as events list
Page file	`frontend/src/pages/shared/Events.jsx`
Data needed	`GET /api/events?search=&category=`
Behaviour	Calendar view of the same data. Month is currently fixed to October 2026 in the demo: make it follow the current month with previous/next controls only if asked.



Source code

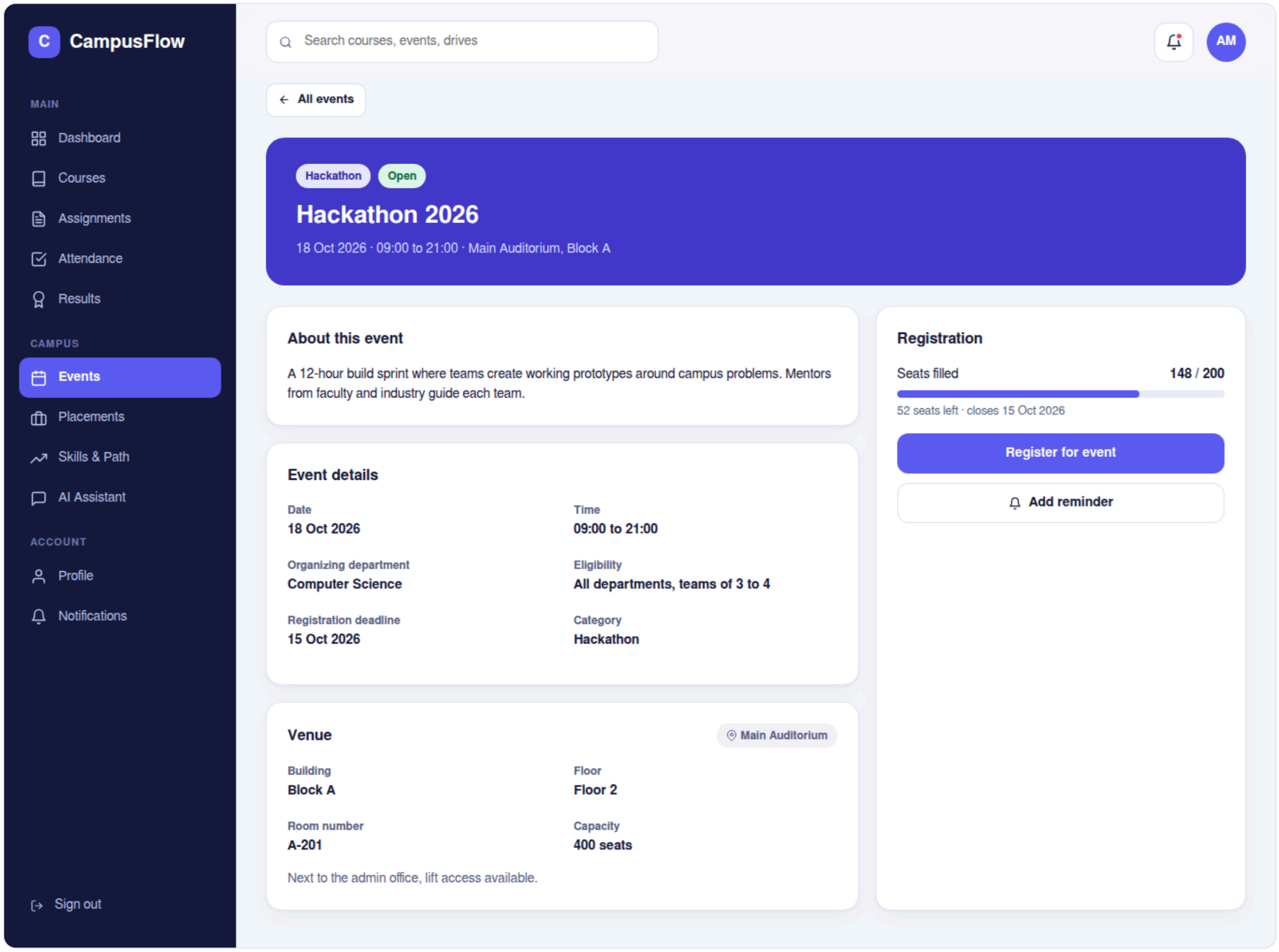
Same file as an earlier interface: `frontend/src/pages/shared/Events.jsx` (rendered with a different role, props or state).

Event details

Route	`/student/events/:id`
Role	All logged-in roles
Page file	`frontend/src/pages/shared/EventDetails.jsx`

Data needed	`GET /api/events/{id}`
	`POST /api/events/{id}/register`
	`DELETE /api/events/{id}/register`
	`POST /api/events/{id}/reminder`

Behaviour	Shows full venue breakdown (building, floor, room, capacity, description).



Source code

frontend/src/pages/shared/EventDetails.jsx

```
import { useState } from 'react';
import { Link, useLocation, useParams } from 'react-router-dom';
import { FiArrowLeft, FiBell, FiMapPin } from 'react-icons/fi';
import Card from '../../components/Card';
import Badge from '../../components/Badge';
import Alert from '../../components/Alert';
import ProgressBar from '../../components/ProgressBar';
import EmptyState from '../../components/EmptyState';
import { seatStatus } from '../../components/EventCard';
import { formatDate } from '../../utils/format';
import { events, venues } from '../../data/demoData';

export default function EventDetails() {
  const { id } = useParams();
  const { pathname } = useLocation();
  const basePath = '/' + pathname.split('/')[1];
  const event = events.find((e) => String(e.id) === String(id)); // replace with GET /api/events/{id}
  const [registered, setRegistered] = useState(false);
  const [reminder, setReminder] = useState(false);

  if (!event) return <Card><EmptyState title="Event not found" action={<Link className="btn btn-primary" to={`${basePath}/events`}>Back to events</Link>} /></Card>;

  const venue = venues.find((v) => v.name === event.venue);
  const seats = seatStatus(event);
  const pct = Math.round((event.registered / event.total) * 100);
```

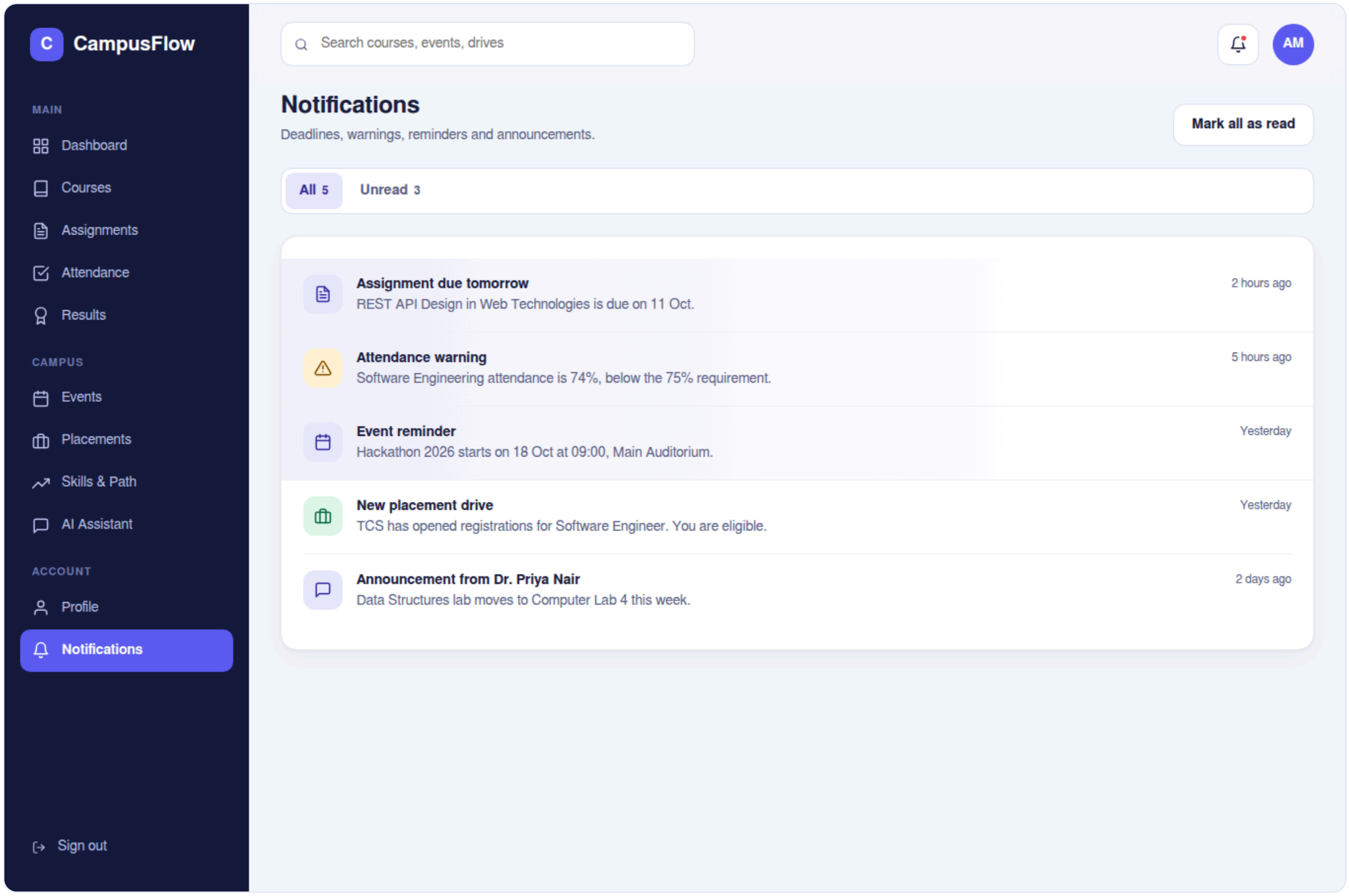
```
return (
  <>
    <Link to={`${basePath}/events`} className="btn btn-outline btn-sm" style={{ alignSelf: 'flex-start' }}><FiArrowLeft /> All events</Link>
    <section className="hero">
      <div>
        <div className="flex gap-8 wrap" style={{ marginBottom: 10 }}>
          <Badge tone="info">{event.category}</Badge>
          <Badge tone={seats.tone}>{seats.label}</Badge>
        </div>
        <h1>{event.title}</h1>
        <p>{formatDate(event.date)} · {event.start} to {event.end} · {event.venue}, {event.building}</p>
      </div>
    </section>

    <div className="grid-main">
      <div className="stack">
        <Card title="About this event">
          <p>{event.description}</p>
        </Card>
        <Card title="Event details">
          <dl className="kv">
            <div><dt>Date</dt><dd>{formatDate(event.date)}</dd></div>
            <div><dt>Time</dt><dd>{event.start} to {event.end}</dd></div>
            <div><dt>Organizing department</dt><dd>{event.department}</dd></div>
            <div><dt>Eligibility</dt><dd>{event.eligibility}</dd></div>
            <div><dt>Registration deadline</dt><dd>{formatDate(event.deadline)}</dd></div>
            <div><dt>Category</dt><dd>{event.category}</dd></div>
          </dl>
        </Card>
        <Card title="Venue" action={<Badge tone="neutral"><FiMapPin aria-hidden="true" /> {event.venue}</Badge>}>
          <dl className="kv">
            <div><dt>Building</dt><dd>{event.building}</dd></div>
            <div><dt>Floor</dt><dd>{event.floor === 0 ? 'Ground floor' : `Floor ${event.floor}`}</dd></div>
            <div><dt>Room number</dt><dd>{event.room}</dd></div>
            <div><dt>Capacity</dt><dd>{venue ? `${venue.capacity} seats` : '-'}</dd></div>
          </dl>
          {venue && <p className="muted" style={{ marginTop: 14 }}>{venue.description}</p>}
        </Card>
      </div>

      <Card title="Registration">
        {registered && <Alert tone="success">You are registered for this event.</Alert>}
        <div style={{ margin: '14px 0' }}>
          <div className="meter-row__top"><span>Seats filled</span><b>{event.registered} / {event.total}</b></div>
          <ProgressBar value={pct} warnBelow={0} tone={pct >= 90 ? 'danger' : pct >= 75 ? 'warning' : ''} label="Seats filled" />
          <p className="muted text-sm" style={{ marginTop: 6 }}>{seats.left} seats left · closes {formatDate(event.deadline)}</p>
        </div>
        <div className="stack" style={{ gap: 10 }}>
          {registered ? (
            <button type="button" className="btn btn-danger btn-block" onClick={() => setRegistered(false)}>Cancel registration</button>
          ) : (
            <button type="button" className="btn btn-primary btn-block" disabled={seats.left <= 0} onClick={() => setRegistered(true)}>Register for event</button>
          )}
          <button type="button" className="btn btn-outline btn-block" onClick={() => setReminder((v) => !v)}>
            <FiBell /> {reminder ? 'Reminder added' : 'Add reminder'}
          </button>
          </div>
        </Card>
      </div>
    </>
  );
}
```

Notifications

Route	`/student/notifications`
Role	All logged-in roles
Page file	`frontend/src/pages/shared/Notifications.jsx`
Data needed	`GET /api/notifications` `PATCH /api/notifications/read-all`
Behaviour	In-app notifications. Types: assignment, attendance, event, placement, announcement.



Source code

`frontend/src/pages/shared/Notifications.jsx`

```
import { useState } from 'react';
import { FiFileText, FiAlertTriangle, FiCalendar, FiBriefcase, FiMessageSquare, FiBell } from 'react-icons/fi';
import PageHeader from '../../components/PageHeader';
import Card from '../../components/Card';
import Tabs from '../../components/Tabs';
import EmptyState from '../../components/EmptyState';
import { notifications as demoNotifications } from '../../data/demoData';

const ICONS = {
  assignment: { icon: FiFileText, tone: 'tone-primary' },
  attendance: { icon: FiAlertTriangle, tone: 'tone-warning' },
  event: { icon: FiCalendar, tone: 'tone-primary' },
  placement: { icon: FiBriefcase, tone: 'tone-success' },
  announcement: { icon: FiMessageSquare, tone: 'tone-primary' },
};

export default function Notifications() {
  const [items, setItems] = useState(demoNotifications); // replace with GET /api/notifications
  const [tab, setTab] = useState('all');
  const unread = items.filter((n) => n.unread).length;
  const shown = tab === 'unread' ? items.filter((n) => n.unread) : items;

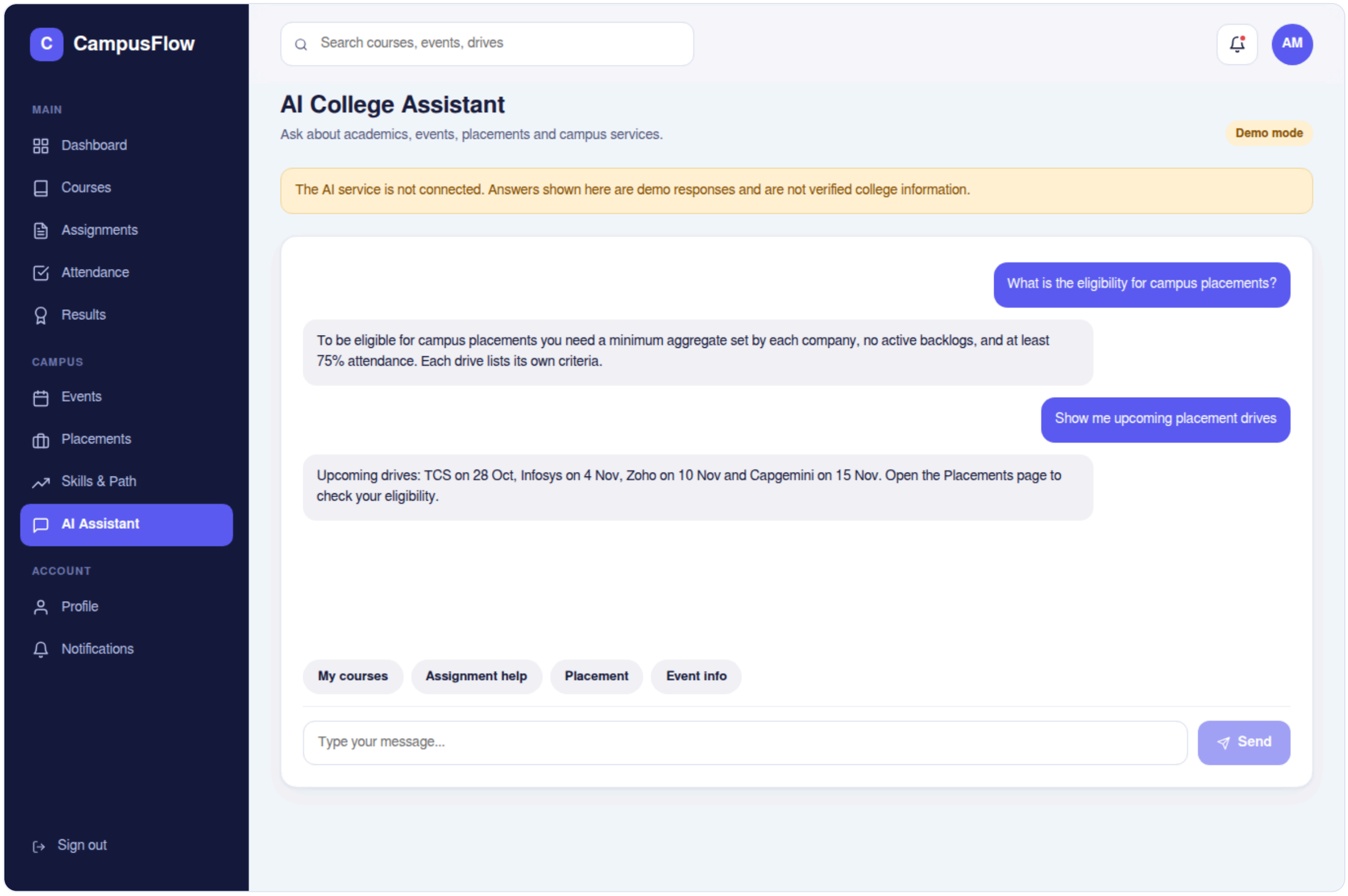
  const markAll = () => setItems((list) => list.map((n) => ({ ...n, unread: false }))); // PATCH /api/notifications/read-all

  return (
    <>
      <PageHeader
        title="Notifications"
        subtitle="Deadlines, warnings, reminders and announcements."
        actions={<button type="button" className="btn btn-outline" onClick={markAll} disabled={unread === 0}>Mark all as read</button>}
      />
      <Tabs value={tab} onChange={setTab} tabs={[{ value: 'all', label: 'All', count: items.length }, { value: 'unread', label: 'Unread', count: unread }]} />
      <Card>
        {shown.length === 0 ? (
```

```
          <EmptyState icon={FiBell} title="You are all caught up" text="New notifications will appear here." />
        ) : (
          shown.map((n) => {
            const { icon: Icon, tone } = ICONS[n.type];
            return (
              <div className={`notif ${n.unread ? 'unread' : ''}`} key={n.id}>
                <span className={`icon-tile ${tone}`}><Icon aria-hidden="true" /></span>
                <div>
                  <div className="notif__title">{n.title}</div>
                  <div className="muted">{n.message}</div>
                </div>
                <span className="notif__time">{n.time}</span>
              </div>
            );
          })
        )}
      </Card>
    </>
  );
}
```

AI College Assistant

Route	`/student/assistant`
Role	All logged-in roles
Page file	`frontend/src/pages/shared/AIAssistant.jsx`
Data needed	`POST /api/assistant/chat { message } -> { reply, mode: "LIVE" | "DEMO" }`
Behaviour	Starts empty when wired (remove chatSeed). When mode is DEMO the page shows a Demo badge and warning: never present demo replies as verified college information.



Source code

`frontend/src/pages/shared/AIAssistant.jsx`

```
import { useEffect, useRef, useState } from 'react';
import { FiSend } from 'react-icons/fi';
import PageHeader from '../../components/PageHeader';
import Card from '../../components/Card';
import Badge from '../../components/Badge';
import Alert from '../../components/Alert';
import { askAssistant } from '../../services/assistantService';
import { getErrorMessage } from '../../utils/format';
import { chatSeed } from '../../data/demoData';

const SUGGESTIONS = ['My courses', 'Assignment help', 'Placement', 'Event info'];

export default function AIAssistant() {
  const [messages, setMessages] = useState(chatSeed); // start with [] when wiring the backend
  const [text, setText] = useState('');
  const [loading, setLoading] = useState(false);
  const [mode, setMode] = useState('DEMO'); // LIVE when the backend reports a configured AI provider
  const endRef = useRef(null);

  useEffect(() => { endRef.current?.scrollIntoView({ block: 'end' }); }, [messages]);

  const send = async (value) => {
    const message = value.trim();
    if (!message || loading) return;
    setMessages((m) => [...m, { id: Date.now(), from: 'me', text: message }]);
    setText('');
    setLoading(true);
    try {
      const { reply, mode: replyMode } = await askAssistant(message);
      setMode(replyMode);
      setMessages((m) => [...m, { id: Date.now() + 1, from: 'bot', text: reply, mode: replyMode }]);
    } catch (err) {
      setMessages((m) => [...m, { id: Date.now() + 1, from: 'error', text: getErrorMessage(err) }]);
    } finally {
```

```
      setLoading(false);
    }
  };

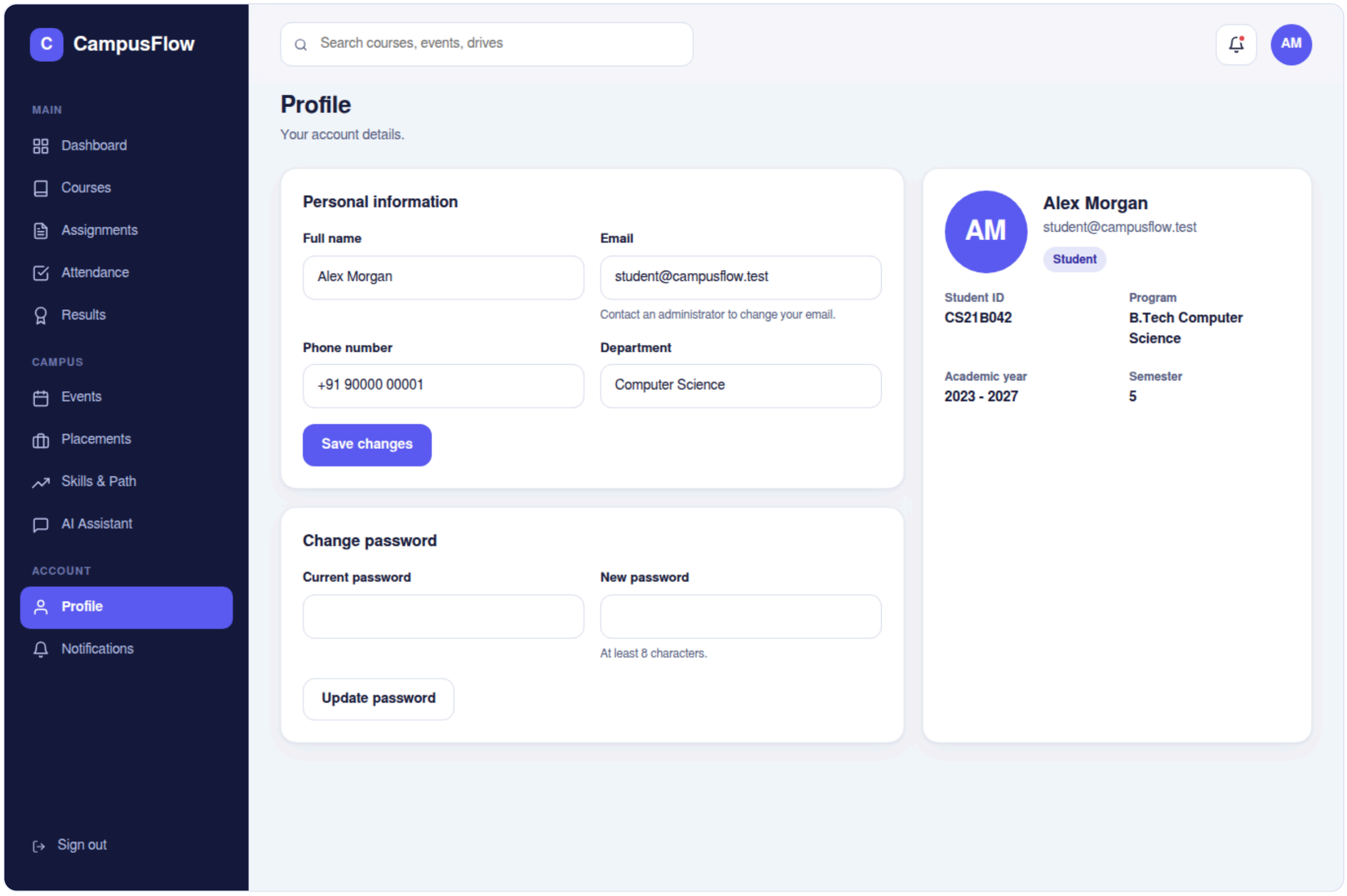
  return (
    <>
      <PageHeader title="AI College Assistant" subtitle="Ask about academics, events, placements and campus services." actions={<Badge tone={mode === 'LIVE' ? 'success'
: 'warning'}>{mode === 'LIVE' ? 'AI connected' : 'Demo mode'}</Badge>} />
      {mode !== 'LIVE' && <Alert tone="warning">The AI service is not connected. Answers shown here are demo responses and are not verified college information.
</Alert>}
      <Card className="chat-page">
        <div className="chat" aria-live="polite">
          {messages.map((m) => (
            <div key={m.id} className={`bubble ${m.from === 'me' ? 'me' : m.from === 'error' ? 'error' : ''}`}>
              {m.text}
              {m.from === 'bot' && m.mode === 'DEMO' && <small>Demo response</small>}
            </div>
          ))}
          {loading && <div className="bubble">Thinking...</div>}
          <div ref={endRef} />
        </div>
        <div className="chip-list" style={{ marginBottom: 12 }}>
          {SUGGESTIONS.map((s) => <button type="button" className="chip" key={s} onClick={() => send(s)}>{s}</button>)}
        </div>
        <form className="chat-input" onSubmit={(e) => { e.preventDefault(); send(text); }}>
          <input className="input" value={text} onChange={(e) => setText(e.target.value)} placeholder="Type your message..." aria-label="Message to the assistant" />
          <button type="submit" className="btn btn-primary" disabled={!text.trim() || loading}><FiSend /> Send</button>
        </form>
      </Card>
    </>
  );
}
```

Profile

Route	`/student/profile`
Role	All logged-in roles
Page file	`frontend/src/pages/shared/Profile.jsx`

Data needed	`GET /api/users/me`
	`PUT /api/users/me`
	`PUT /api/users/me/password`

Behaviour	Email is read-only. Student-only details appear for STUDENT.



Source code

`frontend/src/pages/shared/Profile.jsx`

```
import { useState } from 'react';
import PageHeader from '../../components/PageHeader';
import Card from '../../components/Card';
import Badge from '../../components/Badge';
import Avatar from '../../components/Avatar';
import Field from '../../components/Field';
import Alert from '../../components/Alert';
import { useAuth } from '../../context/AuthContext';
import { ROLE_LABELS } from '../../utils/format';
import { student } from '../../data/demoData';

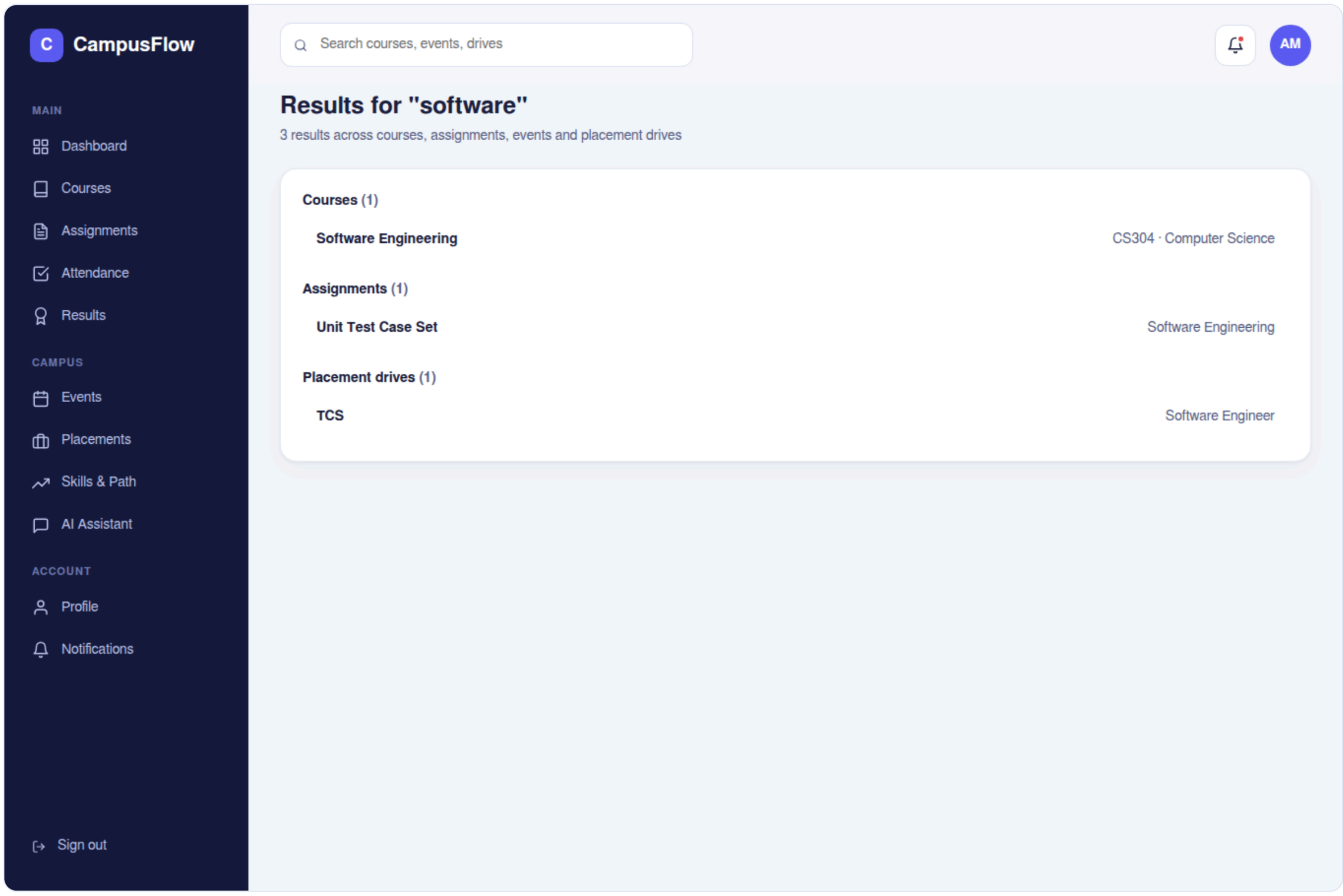
export default function Profile() {
  const { user } = useAuth();
  const [saved, setSaved] = useState(false);
  const [form, setForm] = useState({ name: user?.name || '', phone: student.phone, department: user?.department || '' }); // replace with GET /api/users/me
  const update = (key) => (e) => { setSaved(false); setForm((f) => ({ ...f, [key]: e.target.value })); };

  return (
    <>
      <PageHeader title="Profile" subtitle="Your account details." />
      <div className="grid-main">
        <div className="stack">
          <Card title="Personal information">
            {saved && <div style={{ marginBottom: 14 }}><Alert tone="success">Profile updated.</Alert></div>}
            <div className="form-grid">
              <Field label="Full name" id="p-name"><input className="input" value={form.name} onChange={update('name')} /></Field>
              <Field label="Email" id="p-email" hint="Contact an administrator to change your email."><input className="input" value={user?.email || ''} readOnly />
</Field>
              <Field label="Phone number" id="p-phone"><input className="input" value={form.phone} onChange={update('phone')} /></Field>
              <Field label="Department" id="p-dept"><input className="input" value={form.department} onChange={update('department')} /></Field>
            </div>
            <div className="flex gap-8 mt-16">
```

```
        <button type="button" className="btn btn-primary" onClick={() => setSaved(true)}>Save changes</button>
      </div>
    </Card>
    <Card title="Change password">
      <div className="form-grid">
        <Field label="Current password" id="pw-old"><input className="input" type="password" autoComplete="current-password" /></Field>
        <Field label="New password" id="pw-new" hint="At least 8 characters."><input className="input" type="password" autoComplete="new-password" /></Field>
      </div>
      <div className="mt-16"><button type="button" className="btn btn-outline">Update password</button></div>
    </Card>
  </div>
  <Card>
    <div className="flex center gap-16">
      <Avatar name={user?.name} size="lg" />
      <div>
        <div className="fw-700" style={{ fontSize: 18 }}>{user?.name}</div>
        <div className="muted">{user?.email}</div>
        <div style={{ marginTop: 8 }}><Badge tone="info">{ROLE_LABELS[user?.role]}</Badge></div>
      </div>
    </div>
    {user?.role === 'STUDENT' && (
      <dl className="kv mt-16" style={{ gridTemplateColumns: '1fr 1fr' }}>
        <div><dt>Student ID</dt><dd>{student.studentId}</dd></div>
        <div><dt>Program</dt><dd>{student.program}</dd></div>
        <div><dt>Academic year</dt><dd>{student.academicYear}</dd></div>
        <div><dt>Semester</dt><dd>{student.semester}</dd></div>
      </dl>
    )}
  </Card>
      </div>
    </>
  );
}
```

Global search results

Route	`/student/search?q=software`
Role	All logged-in roles
Page file	`frontend/src/pages/shared/SearchResults.jsx`
Data needed	`GET /api/search?q= -> { courses[], assignments[], events[], drives[] } each item {id,title,subtitle}`
Behaviour	Opened from the top-bar search box. Already has loading, error and no-results states. Results must respect the caller role.



Source code

frontend/src/pages/shared/SearchResults.jsx

```
import { useEffect, useState } from 'react';
import { Link, useLocation, useSearchParams } from 'react-router-dom';
import { FiSearch } from 'react-icons/fi';
import PageHeader from '../../components/PageHeader';
import Card from '../../components/Card';
import Alert from '../../components/Alert';
import EmptyState from '../../components/EmptyState';
import Skeleton from '../../components/Skeleton';
import { searchAll } from '../../services/searchService';
import { getErrorMessage } from '../../utils/format';

const GROUPS = [
  { key: 'courses', label: 'Courses', path: 'courses' },
  { key: 'assignments', label: 'Assignments', path: 'assignments' },
  { key: 'events', label: 'Events', path: 'events' },
  { key: 'drives', label: 'Placement drives', path: 'placements' },
];

export default function SearchResults() {
  const [params] = useSearchParams();
  const q = params.get('q') || '';
  const { pathname } = useLocation();
  const base = '/' + pathname.split('/')[1];
  const [state, setState] = useState({ loading: true, error: '', data: null });

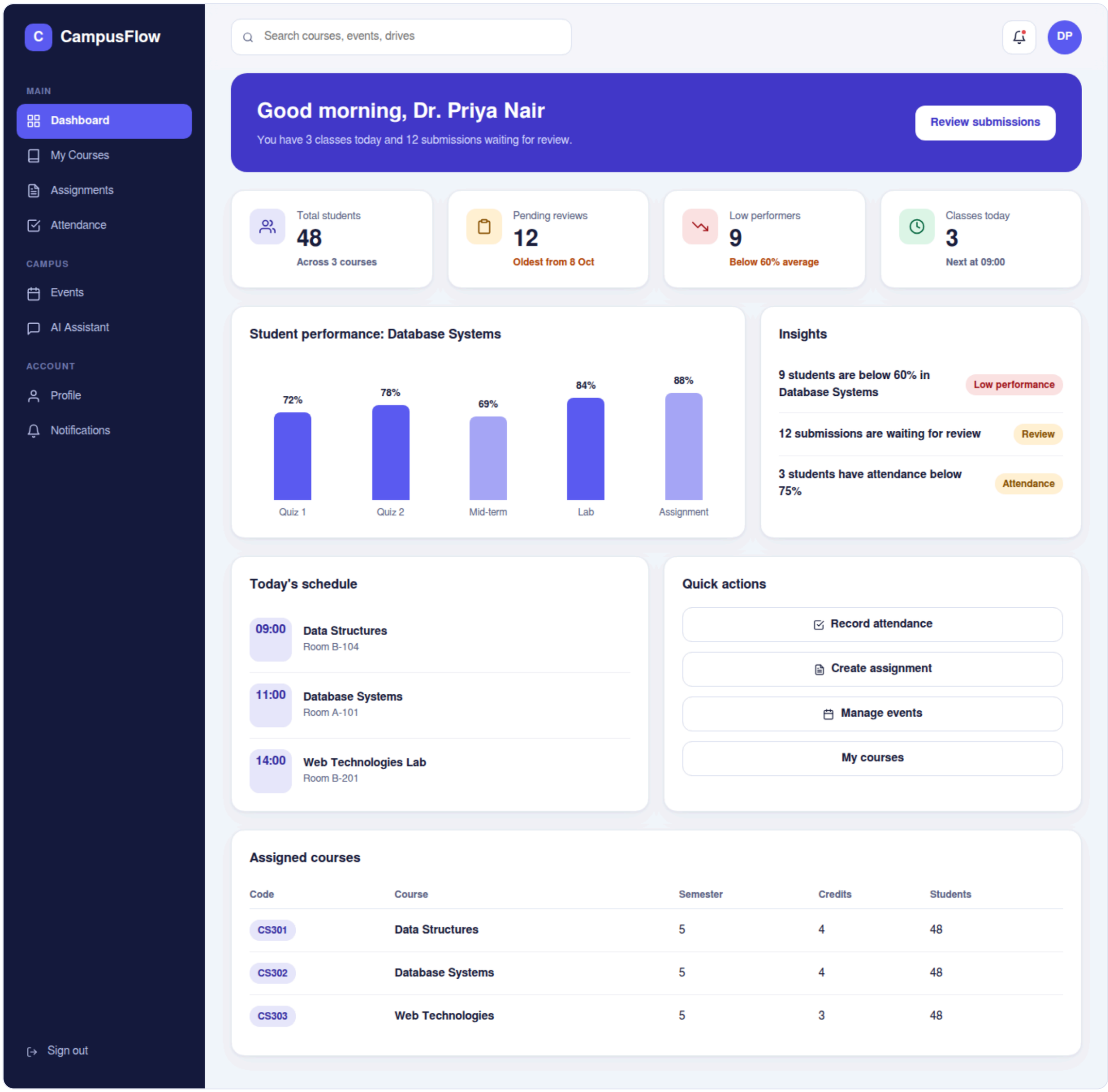
  useEffect(() => {
    let active = true;
    setState({ loading: true, error: '', data: null });
    searchAll(q)
      .then((data) => active && setState({ loading: false, error: '', data }))
      .catch((err) => active && setState({ loading: false, error: getErrorMessage(err), data: null }));
    return () => { active = false; };
  }, [q]);

  const total = state.data ? GROUPS.reduce((n, g) => n + (state.data[g.key]?.length || 0), 0) : 0;
```

```
  return (
    <>
      <PageHeader title={q ? `Results for "${q}"` : 'Search'} subtitle={state.data ? `${total} result${total === 1 ? '' : 's'} across courses, assignments, events and
placement drives` : 'Searching...'} />
      {state.error && <Alert tone="error">{state.error}</Alert>}
      {state.loading && <Card><div className="stack"><Skeleton height={18} width="40%" /><Skeleton height={14} /><Skeleton height={14} width="80%" /></div></Card>}
      {!state.loading && !state.error && total === 0 && (
        <Card><EmptyState icon={FiSearch} title="No results found" text="Try a different keyword, such as a course name or event title." /></Card>
      )}
      {state.data && total > 0 && (
        <Card>
          {GROUPS.filter((g) => state.data[g.key]?.length).map((g) => (
            <div className="result-group" key={g.key}>
              <h3 style={{ fontSize: 14, marginBottom: 6 }}>{g.label} <span className="muted">({state.data[g.key].length})</span></h3>
              {state.data[g.key].map((r) => (
                <Link className="result-link" key={r.id} to={g.key === 'events' ? `${base}/events/${r.id}` : `${base}/${g.path}`}>
                  <span className="fw-700">{r.title}</span>
                  <span className="muted">{r.subtitle}</span>
                </Link>
              ))}
            </div>
          ))}
        </Card>
      )}
    </>
  );
}
```

Faculty dashboard

Route	`/faculty`
Role	FACULTY
Page file	`frontend/src/pages/faculty/FacultyDashboard.jsx`
Data needed	`GET /api/faculty/dashboard`
Behaviour	Stats, performance chart, insights, schedule and assigned courses come from stored data.



Source code

frontend/src/pages/faculty/FacultyDashboard.jsx

```
import { useNavigate } from 'react-router-dom';
import { FiUsers, FiClipboard, FiTrendingDown, FiClock, FiCheckSquare, FiFileText, FiCalendar } from 'react-icons/fi';
import Hero from '../../components/Hero';
import StatCard from '../../components/StatCard';
import Card from '../../components/Card';
import Badge from '../../components/Badge';
import BarChart from '../../components/BarChart';
import DataTable from '../../components/DataTable';
import { useAuth } from '../../context/AuthContext';
import { courses, facultyStats } from '../../data/demoData';

export default function FacultyDashboard() {
```

```
const { user } = useAuth();
const navigate = useNavigate();
const mine = courses.slice(0, 3);

return (
  <>
    <Hero
      title={`Good morning, ${user?.name}`}
      subtitle="You have 3 classes today and 12 submissions waiting for review."
      action={<button type="button" className="btn btn-light" onClick={() => navigate('/faculty/assignments')}>Review submissions</button>}
    />
    <div className="stat-grid">
      <StatCard icon={FiUsers} label="Total students" value="48" hint="Across 3 courses" trend="flat" />
      <StatCard icon={FiClipboard} tone="warning" label="Pending reviews" value="12" hint="Oldest from 8 Oct" trend="down" />
      <StatCard icon={FiTrendingDown} tone="danger" label="Low performers" value="9" hint="Below 60% average" trend="down" />
      <StatCard icon={FiClock} tone="success" label="Classes today" value="3" hint="Next at 09:00" trend="flat" />
    </div>

    <div className="grid-main">
      <Card title="Student performance: Database Systems">
        <BarChart data={facultyStats.performance} max={100} unit="%" />
      </Card>
      <Card title="Insights">
        {facultyStats.insights.map((i) => (
          <div className="list-row" key={i.id}>
            <div className="grow fw-600">{i.text}</div>
            <Badge tone={i.tone}>{i.badge}</Badge>
          </div>
        ))}
      </Card>
    </div>

    <div className="grid-2">
      <Card title="Today's schedule">
        {facultyStats.schedule.map((s) => (
          <div className="list-row" key={s.id}>
            <div className="date-tile" style={{ fontSize: 15 }}>{s.time}</div>
            <div className="grow"><div className="fw-700">{s.course}</div><div className="muted text-sm">Room {s.room}</div></div>
          </div>
        ))}
      </Card>
      <Card title="Quick actions">
        <div className="grid-2" style={{ gap: 12 }}>
          <button type="button" className="btn btn-outline" onClick={() => navigate('/faculty/attendance')}><FiCheckSquare /> Record attendance</button>
          <button type="button" className="btn btn-outline" onClick={() => navigate('/faculty/assignments')}><FiFileText /> Create assignment</button>
          <button type="button" className="btn btn-outline" onClick={() => navigate('/faculty/events')}><FiCalendar /> Manage events</button>
          <button type="button" className="btn btn-outline" onClick={() => navigate('/faculty/courses')}>My courses</button>
        </div>
      </Card>
    </div>

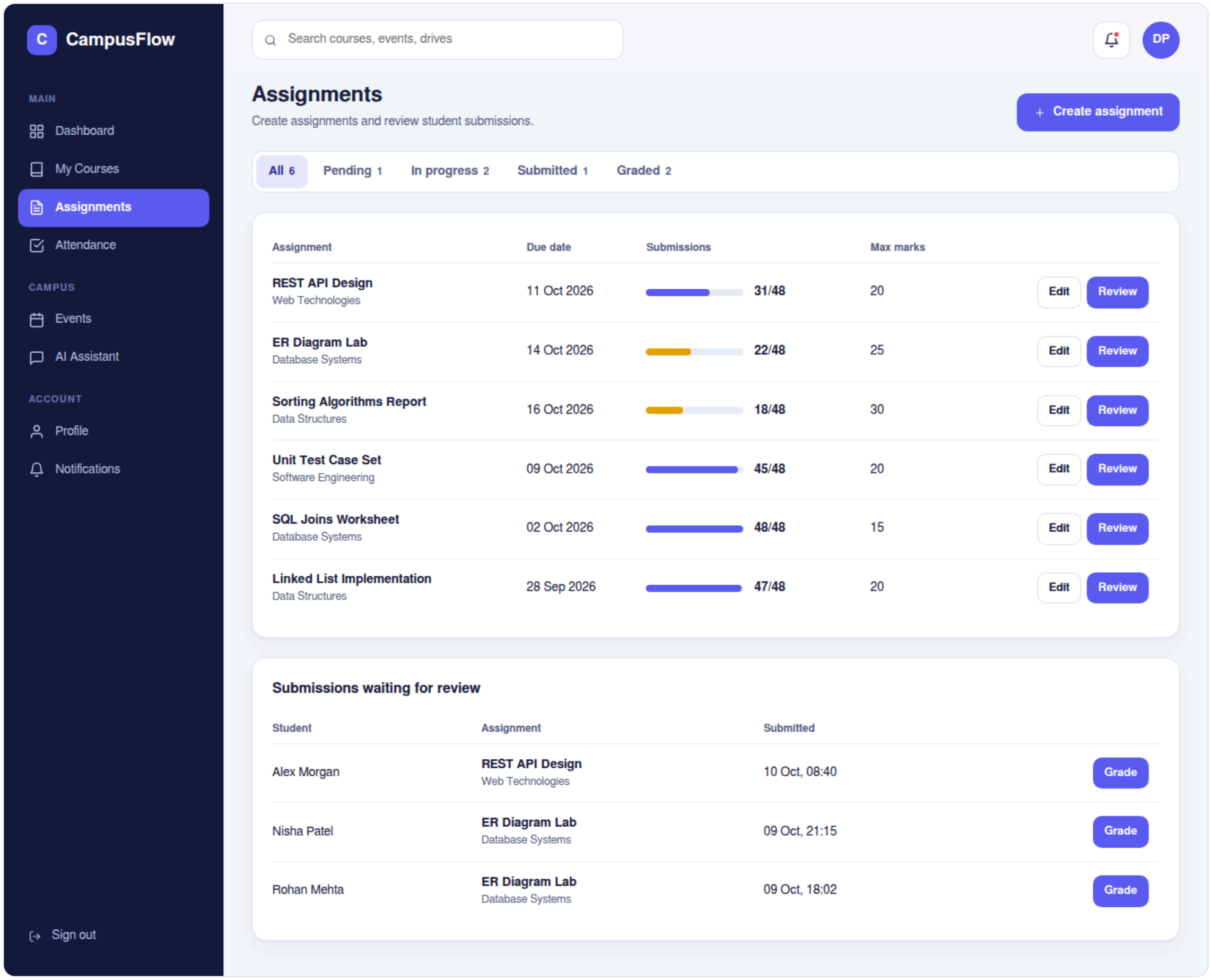
    <Card title="Assigned courses">
      <DataTable
        columns={[
          { key: 'code', header: 'Code', render: (r) => <Badge tone="info">{r.code}</Badge> },
          { key: 'name', header: 'Course', render: (r) => <span className="fw-700">{r.name}</span> },
          { key: 'semester', header: 'Semester' },
          { key: 'credits', header: 'Credits' },
          { key: 'students', header: 'Students', render: () => '48' },
        ]}
        rows={mine}
      />
    </Card>
  </>
);
}
```

Assignments (faculty)

Route	`/faculty/assignments`
Role	FACULTY
Page file	`frontend/src/pages/shared/Assignments.jsx`

Data needed	`GET /api/assignments?mine=true`
	`POST /api/assignments`
	`PUT /api/assignments/{id}`
	`GET /api/assignments/submissions?status=SUBMITTED`
	`PUT /api/submissions/{id}/grade`

Behaviour	Same page file as the student screen, rendered with mode="faculty".

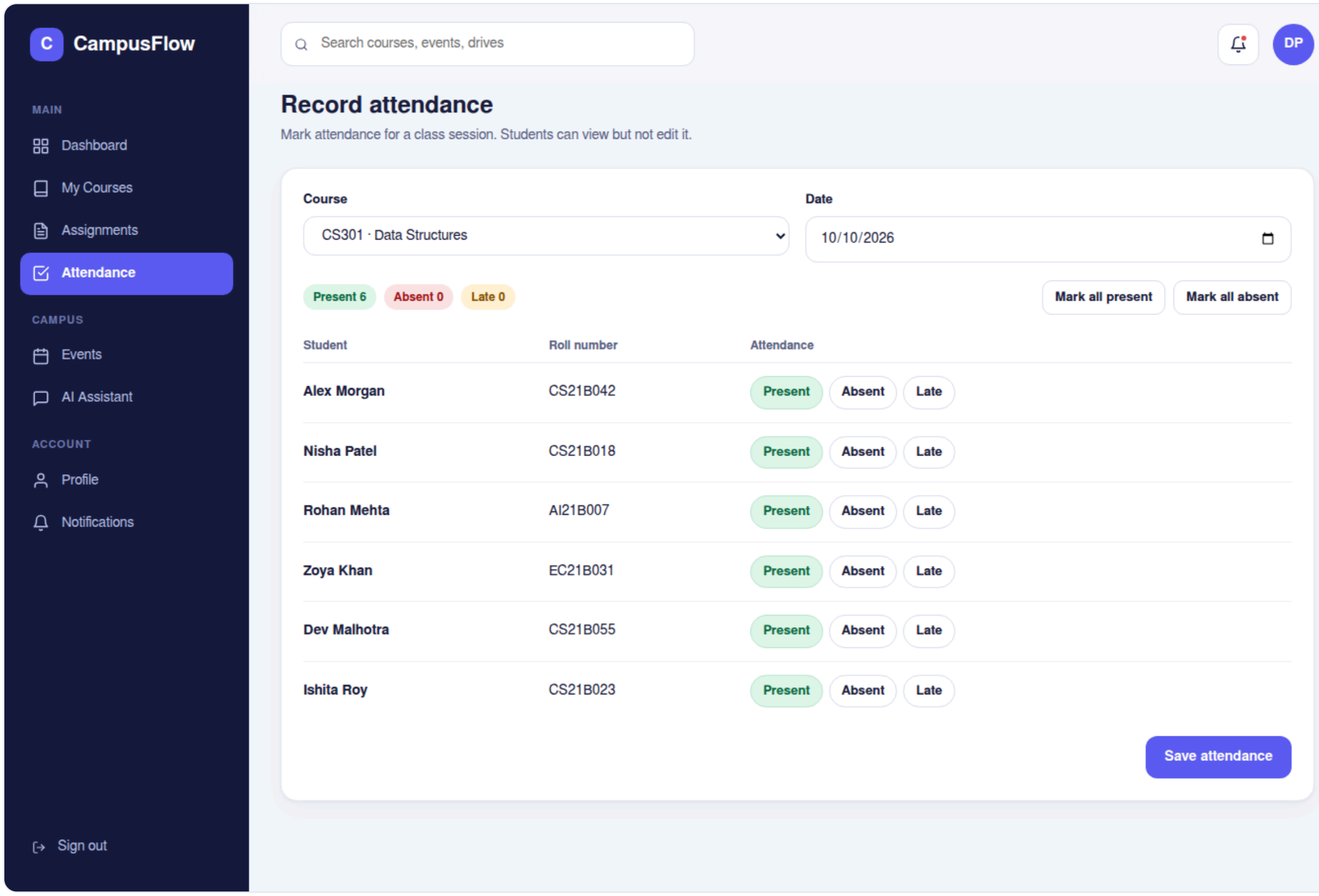


Source code

Same file as an earlier interface: `frontend/src/pages/shared/Assignments.jsx` (rendered with a different role, props or state).

Record attendance (faculty)

Route	`/faculty/attendance`
Role	FACULTY
Page file	`frontend/src/pages/faculty/ManageAttendance.jsx`
Data needed	`GET /api/courses/{id}/students` `POST /api/attendance { courseId, date, records:[{studentId,status}] }`
Behaviour	Only faculty of the course (or admin) may record attendance.



Source code

`frontend/src/pages/faculty/ManageAttendance.jsx`

```
import { useMemo, useState } from 'react';
import PageHeader from '../../components/PageHeader';
import Card from '../../components/Card';
import Badge from '../../components/Badge';
import Alert from '../../components/Alert';
import Field from '../../components/Field';
import DataTable from '../../components/DataTable';
import { courses, classRoster } from '../../data/demoData';

const OPTIONS = [
  { value: 'Present', cls: 'p' },
  { value: 'Absent', cls: 'a' },
  { value: 'Late', cls: 'l' },
];

export default function ManageAttendance() {
  const [course, setCourse] = useState(String(courses[0].id));
  const [date, setDate] = useState('2026-10-10');
  const [marks, setMarks] = useState(() => Object.fromEntries(classRoster.map((s) => [s.id, 'Present'])));
  const [saved, setSaved] = useState(false);

  const counts = useMemo(() => {
    const c = { Present: 0, Absent: 0, Late: 0 };
    Object.values(marks).forEach((v) => { c[v] += 1; });
    return c;
  }, [marks]);

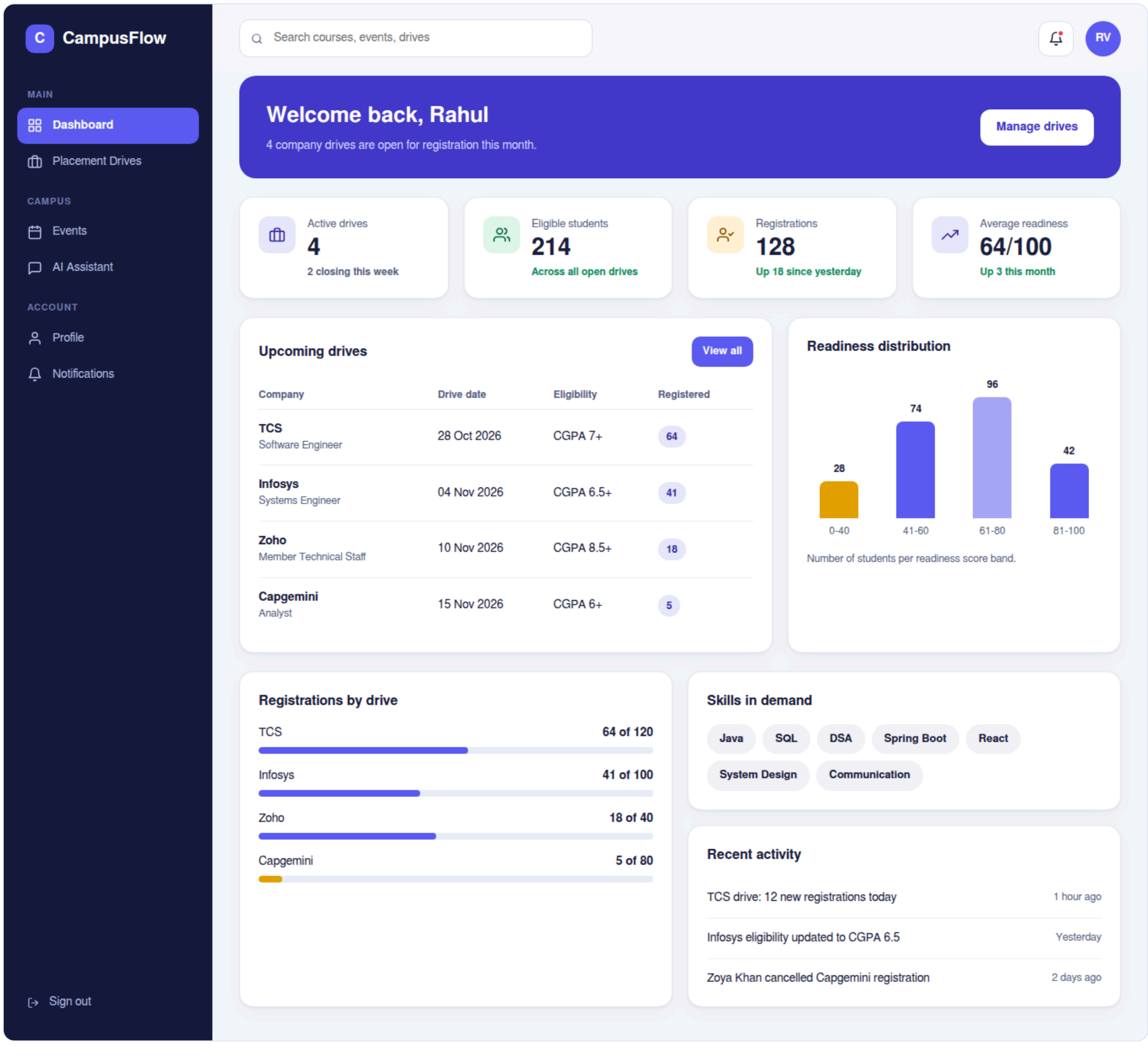
  const set = (id, value) => { setSaved(false); setMarks((m) => ({ ...m, [id]: value })); };
  const markAll = (value) => { setSaved(false); setMarks(Object.fromEntries(classRoster.map((s) => [s.id, value]))); };
  const save = () => setSaved(true); // replace with POST /api/attendance (courseId, date, records[])

  return (
    <>
      <PageHeader title="Record attendance" subtitle="Mark attendance for a class session. Students can view but not edit it." />
```

```
      {saved && <Alert tone="success">Attendance saved for {date}.</Alert>}
      <Card>
        <div className="form-grid" style={{ marginBottom: 18 }}>
          <Field label="Course" id="att-course">
            <select className="select" value={course} onChange={(e) => { setCourse(e.target.value); setSaved(false); }}>
              {courses.map((c) => <option key={c.id} value={c.id}>{c.code} · {c.name}</option>)}
            </select>
          </Field>
          <Field label="Date" id="att-date">
            <input className="input" type="date" value={date} onChange={(e) => { setDate(e.target.value); setSaved(false); }} />
          </Field>
        </div>
        <div className="flex center between wrap gap-12" style={{ marginBottom: 14 }}>
          <div className="flex gap-8 wrap">
            <Badge tone="success">Present {counts.Present}</Badge>
            <Badge tone="danger">Absent {counts.Absent}</Badge>
            <Badge tone="warning">Late {counts.Late}</Badge>
          </div>
          <div className="flex gap-8">
            <button type="button" className="btn btn-outline btn-sm" onClick={() => markAll('Present')}>Mark all present</button>
            <button type="button" className="btn btn-outline btn-sm" onClick={() => markAll('Absent')}>Mark all absent</button>
          </div>
        </div>
        <DataTable
          columns={[
            { key: 'name', header: 'Student', render: (r) => <span className="fw-700">{r.name}</span> },
            { key: 'roll', header: 'Roll number' },
            { key: 'status', header: 'Attendance', render: (r) => (
              <div className="attendance-input" role="radiogroup" aria-label={`Attendance for ${r.name}`}>
                {OPTIONS.map((o) => (
                  <label className="radio-pill" key={o.value}>
                    <input type="radio" name={`att-${r.id}`} checked={marks[r.id] === o.value} onChange={() => set(r.id, o.value)} />
                    <span className={o.cls}>{o.value}</span>
                  </label>
                ))}
              </div>
            ) },
          ]}
          rows={classRoster}
        />
        <div className="flex gap-8 mt-16" style={{ justifyContent: 'flex-end' }}>
          <button type="button" className="btn btn-primary" onClick={save}>Save attendance</button>
        </div>
      </Card>
    </>
  );
}
```

Placement officer dashboard

Route	`/placement`
Role	PLACEMENT_OFFICER
Page file	`frontend/src/pages/placement/PlacementDashboard.jsx`
Data needed	`GET /api/placements/dashboard`
Behaviour	Active drives, eligible students, registrations, readiness distribution, skills in demand, recent activity.



Source code

frontend/src/pages/placement/PlacementDashboard.jsx

```
import { Link, useNavigate } from 'react-router-dom';
import { FiBriefcase, FiUsers, FiUserCheck, FiTrendingUp } from 'react-icons/fi';
import Hero from '../../components/Hero';
import StatCard from '../../components/StatCard';
import Card from '../../components/Card';
import Badge from '../../components/Badge';
import BarChart from '../../components/BarChart';
import DataTable from '../../components/DataTable';
import MeterRow from '../../components/MeterRow';
import { useAuth } from '../../context/AuthContext';
import { formatDate, firstName } from '../../utils/format';
import { drives, placementStats } from '../../data/demoData';

export default function PlacementDashboard() {
  const { user } = useAuth();
  const navigate = useNavigate();
  return (
    <>
```

```
      <Hero
        title={`Welcome back, ${firstName(user?.name)}`}
        subtitle="4 company drives are open for registration this month."
        action={<button type="button" className="btn btn-light" onClick={() => navigate('/placement/drives')}>Manage drives</button>}
      />
      <div className="stat-grid">
        <StatCard icon={FiBriefcase} label="Active drives" value="4" hint="2 closing this week" trend="flat" />
        <StatCard icon={FiUsers} tone="success" label="Eligible students" value="214" hint="Across all open drives" />
        <StatCard icon={FiUserCheck} tone="warning" label="Registrations" value="128" hint="Up 18 since yesterday" />
        <StatCard icon={FiTrendingUp} label="Average readiness" value="64/100" hint="Up 3 this month" />
      </div>

      <div className="grid-main">
        <Card title="Upcoming drives" action={<Link className="btn btn-primary btn-sm" to="/placement/drives">View all</Link>}>
          <DataTable
            columns={[
              { key: 'company', header: 'Company', render: (r) => <div><div className="fw-700">{r.company}</div><div className="muted text-sm">{r.role}</div></div> },
              { key: 'date', header: 'Drive date', render: (r) => formatDate(r.date) },
              { key: 'criteria', header: 'Eligibility', render: (r) => `CGPA ${r.minCgpa}+` },
              { key: 'registered', header: 'Registered', render: (r) => <Badge tone="info">{r.registered}</Badge> },
            ]}
            rows={drives}
          />
        </Card>
        <Card title="Readiness distribution">
          <BarChart data={placementStats.readinessBands} />
          <p className="muted text-sm mt-16">Number of students per readiness score band.</p>
        </Card>
      </div>

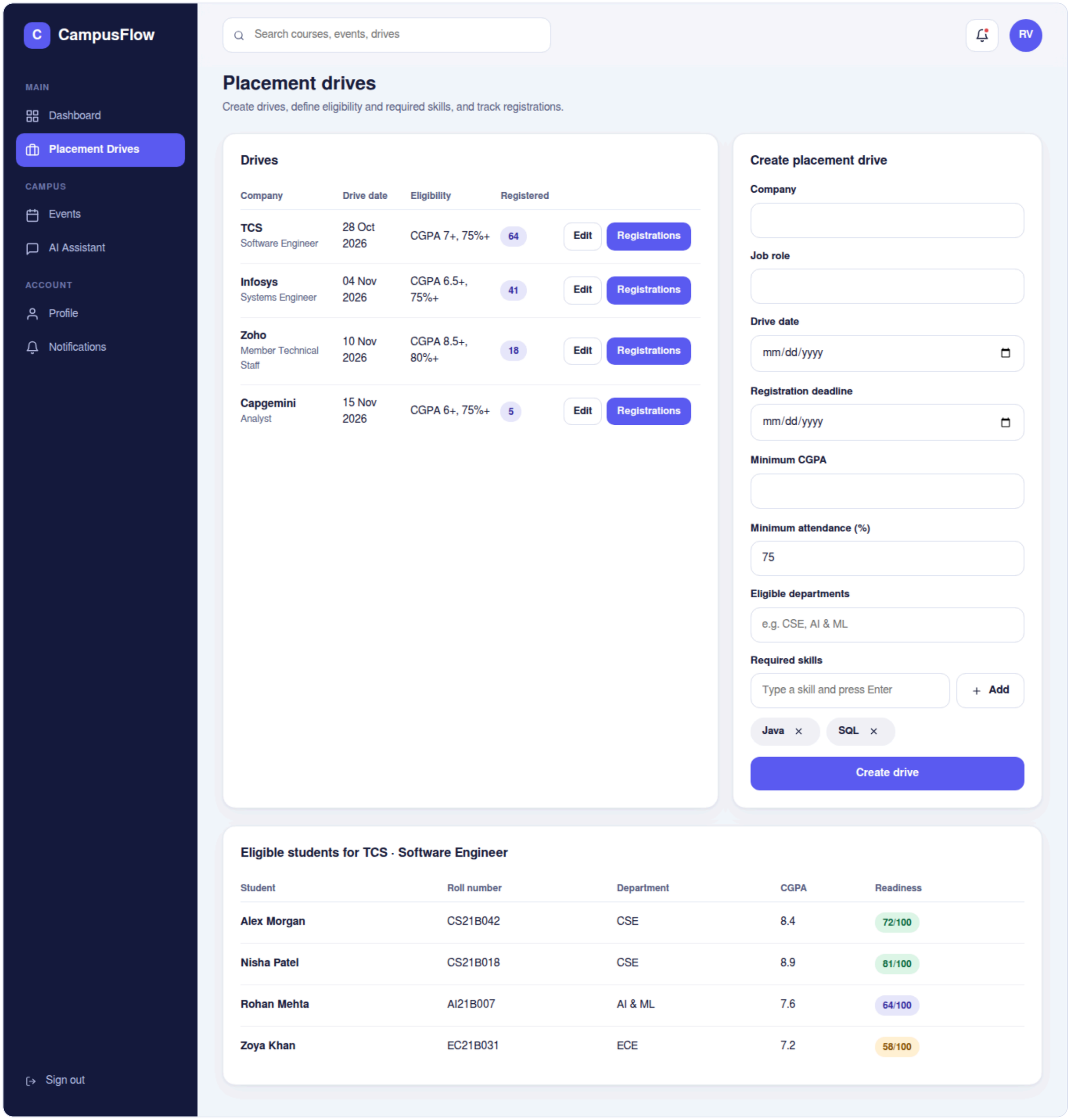

      <div className="grid-2">
        <Card title="Registrations by drive">
          {placementStats.registrationsByDrive.map((d) => (
            <MeterRow key={d.label} label={d.label} value={Math.round((d.value / d.of) * 100)} display={`${d.value} of ${d.of}`} warnBelow={30} />
          ))}
        </Card>
        <div className="stack">
          <Card title="Skills in demand">
            <div className="chip-list">{placementStats.skillDemand.map((s) => <span className="chip" key={s}>{s}</span>)}</div>
          </Card>
          <Card title="Recent activity">
            {placementStats.activity.map((a) => (
              <div className="list-row" key={a.id}>
                <div className="grow">{a.text}</div>
                <span className="muted text-sm">{a.time}</span>
              </div>
            ))}
          </Card>
        </div>
      </div>
    </>
  );
}
```

Placement drive management

Route	`/placement/drives`
Role	PLACEMENT_OFFICER
Page file	`frontend/src/pages/placement/DriveManagement.jsx`

Data needed	`GET /api/placements/drives`
	`POST /api/placements/drives`
	`PUT /api/placements/drives/{id}`
	`GET /api/placements/drives/{id}/eligible-students`
	`GET /api/placements/drives/{id}/registrations`

Behaviour	Create form validates the deadline is on or before the drive date.



Source code

`frontend/src/pages/placement/DriveManagement.jsx`

```
import { useState } from 'react';
import { FiPlus, FiX } from 'react-icons/fi';
```

```
import PageHeader from '../../components/PageHeader';
import Card from '../../components/Card';
import Badge from '../../components/Badge';
import Field from '../../components/Field';
import Alert from '../../components/Alert';
import DataTable from '../../components/DataTable';
import { formatDate } from '../../utils/format';
import { drives as demoDrives, eligibleStudents } from '../../data/demoData';

const EMPTY = { company: '', role: '', date: '', deadline: '', minCgpa: '', minAttendance: '75', departments: '' };

export default function DriveManagement() {
  const [rows, setRows] = useState(demoDrives); // replace with GET /api/placements/drives
  const [form, setForm] = useState(EMPTY);
  const [skills, setSkills] = useState(['Java', 'SQL']);
  const [skillInput, setSkillInput] = useState('');
  const [errors, setErrors] = useState({});
  const [saved, setSaved] = useState(false);

  const update = (key) => (e) => { setSaved(false); setForm((f) => ({ ...f, [key]: e.target.value })); };
  const addSkill = () => {
    const s = skillInput.trim();
    if (s && !skills.includes(s)) setSkills([...skills, s]);
    setSkillInput('');
  };

  const create = (e) => {
    e.preventDefault();
    const next = {};
    if (!form.company.trim()) next.company = 'Company name is required.';
    if (!form.role.trim()) next.role = 'Role is required.';
    if (!form.date) next.date = 'Choose the drive date.';
    if (!form.deadline) next.deadline = 'Choose the registration deadline.';
    if (form.date && form.deadline && form.deadline > form.date) next.deadline = 'Deadline must be on or before the drive date.';
    setErrors(next);
    if (Object.keys(next).length) return;
    setRows((r) => [{ ...form, id: Date.now(), skills, registered: 0, minCgpa: Number(form.minCgpa) || 0, minAttendance: Number(form.minAttendance) || 0 }, ...r]); //
POST /api/placements/drives
    setForm(EMPTY);
    setSaved(true);
  };

  return (
    <>
      <PageHeader title="Placement drives" subtitle="Create drives, define eligibility and required skills, and track registrations." />
      <div className="grid-main">
        <Card title="Drives">
          <DataTable
            columns={[
              { key: 'company', header: 'Company', render: (r) => <div><div className="fw-700">{r.company}</div><div className="muted text-sm">{r.role}</div></div> },
              { key: 'date', header: 'Drive date', render: (r) => formatDate(r.date) },
              { key: 'criteria', header: 'Eligibility', render: (r) => `CGPA ${r.minCgpa}+, ${r.minAttendance}%+` },
              { key: 'registered', header: 'Registered', render: (r) => <Badge tone="info">{r.registered}</Badge> },
              { key: 'actions', header: '', className: 'actions', render: () => <><button type="button" className="btn btn-outline btn-sm">Edit</button><button
type="button" className="btn btn-primary btn-sm">Registrations</button></> },
            ]}
            rows={rows}
          />
        </Card>

        <Card title="Create placement drive">
          <form className="stack" style={{ gap: 14 }} onSubmit={create} noValidate>
            {saved && <Alert tone="success">Drive created.</Alert>}
            <Field label="Company" id="d-company" error={errors.company}><input className="input" value={form.company} onChange={update('company')} /></Field>
            <Field label="Job role" id="d-role" error={errors.role}><input className="input" value={form.role} onChange={update('role')} /></Field>
            <div className="form-grid">
              <Field label="Drive date" id="d-date" error={errors.date}><input className="input" type="date" value={form.date} onChange={update('date')} /></Field>
              <Field label="Registration deadline" id="d-deadline" error={errors.deadline}><input className="input" type="date" value={form.deadline} onChange=
{update('deadline')} /></Field>
              <Field label="Minimum CGPA" id="d-cgpa"><input className="input" type="number" step="0.1" min="0" max="10" value={form.minCgpa} onChange=
{update('minCgpa')} /></Field>
              <Field label="Minimum attendance (%)" id="d-att"><input className="input" type="number" min="0" max="100" value={form.minAttendance} onChange=
{update('minAttendance')} /></Field>
            </div>
            <Field label="Eligible departments" id="d-dept"><input className="input" placeholder="e.g. CSE, AI & ML" value={form.departments} onChange=
{update('departments')} /></Field>
            <div className="field">
              <label htmlFor="d-skill">Required skills</label>
              <div className="flex gap-8">
                <input id="d-skill" className="input" value={skillInput} onChange={(e) => setSkillInput(e.target.value)} onKeyDown={(e) => { if (e.key === 'Enter') {
e.preventDefault(); addSkill(); } }} placeholder="Type a skill and press Enter" />
                <button type="button" className="btn btn-outline" onClick={addSkill}><FiPlus /> Add</button>
              </div>
              <div className="chip-list" style={{ marginTop: 6 }}>
                {skills.map((s) => (
                  <span className="chip" key={s}>{s}
                    <button type="button" aria-label={`Remove ${s}`} onClick={() => setSkills(skills.filter((x) => x !== s))} style={{ border: 0, background:
'transparent', marginLeft: 6, display: 'inline-flex' }}><FiX /></button>
                  </span>
                ))}
              </div>
            </div>
            <button type="submit" className="btn btn-primary btn-block">Create drive</button>
          </form>
        </Card>
      </div>

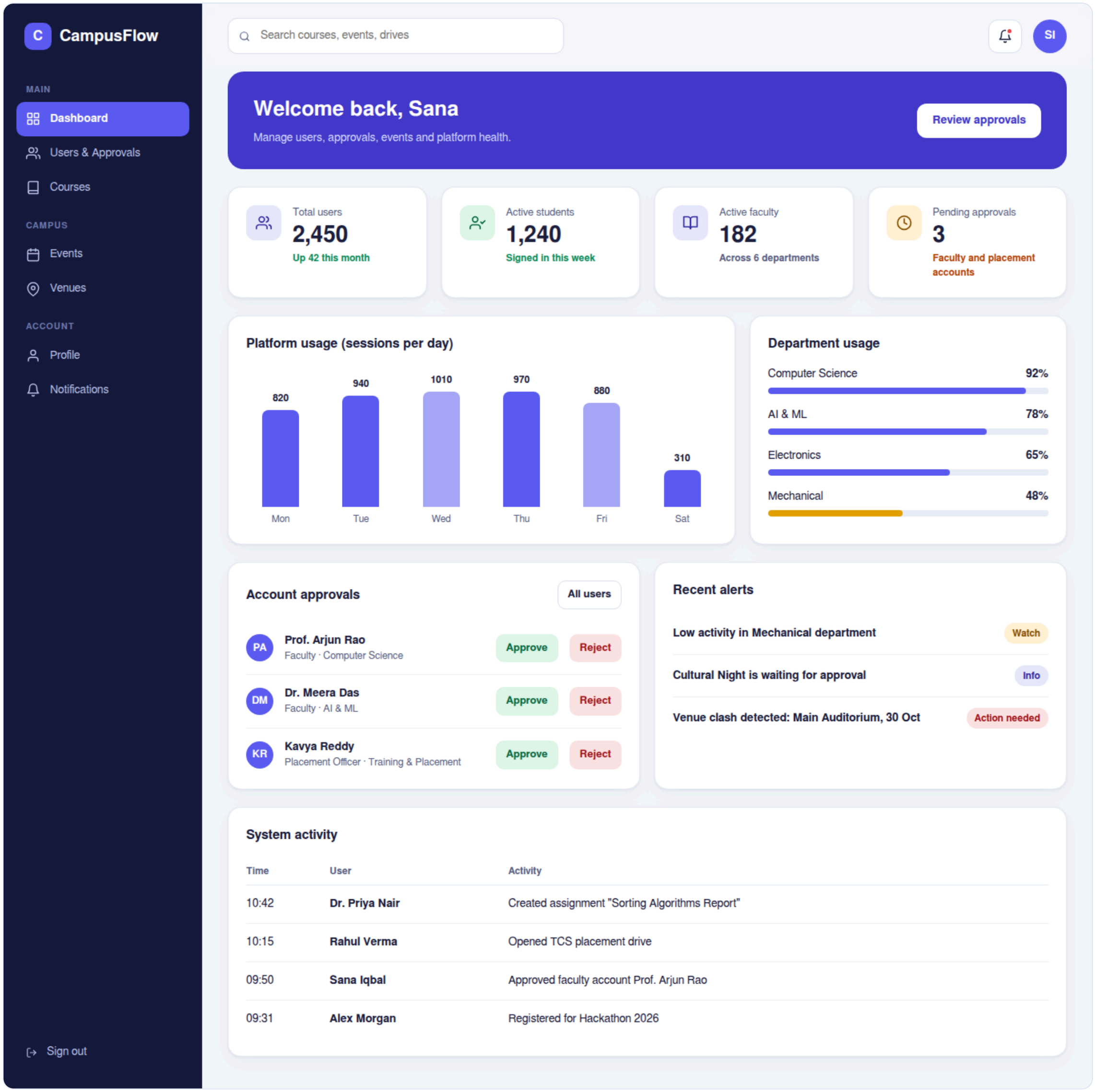
      <Card title="Eligible students for TCS · Software Engineer">
        <DataTable
          columns={[
            { key: 'name', header: 'Student', render: (r) => <span className="fw-700">{r.name}</span> },
            { key: 'roll', header: 'Roll number' },
            { key: 'department', header: 'Department' },
            { key: 'cgpa', header: 'CGPA' },
            { key: 'readiness', header: 'Readiness', render: (r) => <Badge tone={r.readiness >= 70 ? 'success' : r.readiness >= 60 ? 'info' : 'warning'}>
{r.readiness}/100</Badge> },
          ]}
          rows={eligibleStudents}
        />
      </Card>
    </>
  );
}
```

Administrator dashboard

Route	`/admin`
Role	ADMIN
Page file	`frontend/src/pages/admin/AdminDashboard.jsx`

Data needed	`GET /api/admin/dashboard`
	`PATCH /api/admin/users/{id}/approve`
	`PATCH /api/admin/users/{id}/reject`

Behaviour	Approve and Reject act on pending faculty and placement accounts.



Source code

`frontend/src/pages/admin/AdminDashboard.jsx`

```
import { useState } from 'react';
import { Link } from 'react-router-dom';
import { FiUsers, FiUserCheck, FiBookOpen, FiClock } from 'react-icons/fi';
import Hero from '../../components/Hero';
import StatCard from '../../components/StatCard';
import Card from '../../components/Card';
import Badge from '../../components/Badge';
import BarChart from '../../components/BarChart';
```

```
import MeterRow from '../../components/MeterRow';
import Avatar from '../../components/Avatar';
import DataTable from '../../components/DataTable';
import { useAuth } from '../../context/AuthContext';
import { firstName, ROLE_LABELS } from '../../utils/format';
import { adminStats, users } from '../../data/demoData';

export default function AdminDashboard() {
  const { user } = useAuth();
  const [pending, setPending] = useState(users.filter((u) => u.status === 'PENDING'));
  const decide = (id) => setPending((list) => list.filter((u) => u.id !== id)); // PATCH /api/admin/users/{id}/approve | reject

  return (
    <>
      <Hero title={`Welcome back, ${firstName(user?.name)}`} subtitle="Manage users, approvals, events and platform health." action={<Link className="btn btn-light"
to="/admin/users">Review approvals</Link>} />
      <div className="stat-grid">
        <StatCard icon={FiUsers} label="Total users" value="2,450" hint="Up 42 this month" />
        <StatCard icon={FiUserCheck} tone="success" label="Active students" value="1,240" hint="Signed in this week" />
        <StatCard icon={FiBookOpen} label="Active faculty" value="182" hint="Across 6 departments" trend="flat" />
        <StatCard icon={FiClock} tone="warning" label="Pending approvals" value={pending.length} hint="Faculty and placement accounts" trend="down" />
      </div>

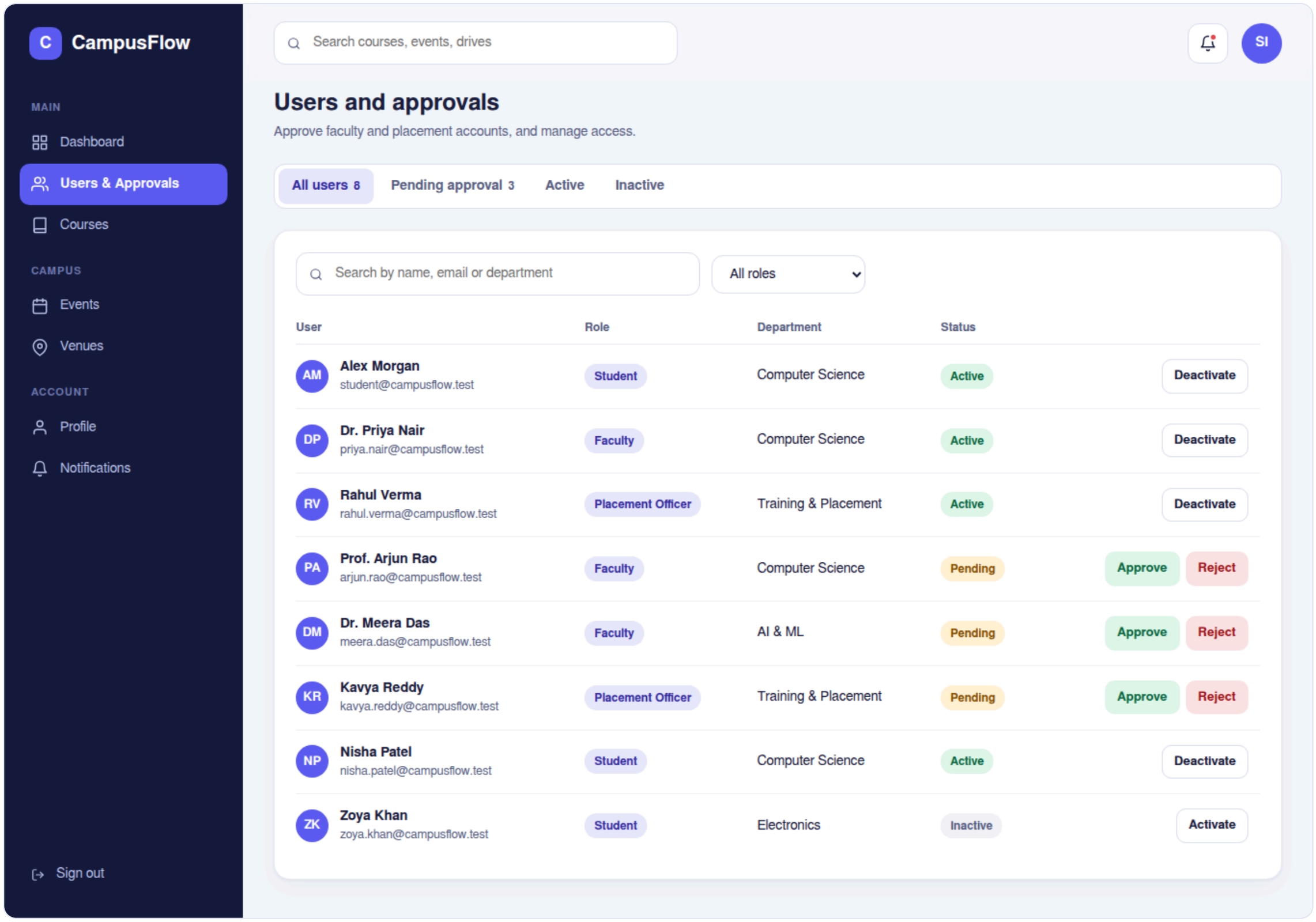
      <div className="grid-main">
        <Card title="Platform usage (sessions per day)"><BarChart data={adminStats.usage} /></Card>
        <Card title="Department usage">
          {adminStats.departmentUsage.map((d) => <MeterRow key={d.label} label={d.label} value={d.value} warnBelow={50} />)}
        </Card>
      </div>

      <div className="grid-2">
        <Card title="Account approvals" action={<Link className="btn btn-outline btn-sm" to="/admin/users">All users</Link>}>
          {pending.length === 0 && <p className="muted">No accounts waiting for approval.</p>}
          {pending.map((u) => (
            <div className="list-row" key={u.id}>
              <Avatar name={u.name} size="sm" />
              <div className="grow"><div className="fw-700">{u.name}</div><div className="muted text-sm">{ROLE_LABELS[u.role]} · {u.department}</div></div>
              <button type="button" className="btn btn-success btn-sm" onClick={() => decide(u.id)}>Approve</button>
              <button type="button" className="btn btn-danger btn-sm" onClick={() => decide(u.id)}>Reject</button>
            </div>
          ))}
        </Card>
        <Card title="Recent alerts">
          {adminStats.alerts.map((a) => (
            <div className="list-row" key={a.id}><div className="grow fw-600">{a.text}</div><Badge tone={a.tone}>{a.tone === 'danger' ? 'Action needed' : a.tone ===
'warning' ? 'Watch' : 'Info'}</Badge></div>
          ))}
        </Card>
      </div>

      <Card title="System activity">
        <DataTable
          columns={[
            { key: 'time', header: 'Time' },
            { key: 'who', header: 'User', render: (r) => <span className="fw-700">{r.who}</span> },
            { key: 'action', header: 'Activity' },
          ]}
          rows={adminStats.activity}
        />
      </Card>
    </>
  );
}
```

Users and approvals

Route	`/admin/users`
Role	ADMIN
Page file	`frontend/src/pages/admin/UserManagement.jsx`
Data needed	`GET /api/admin/users?status=&role=&search=` `PATCH /api/admin/users/{id}/status`
Behaviour	Tabs: All, Pending approval, Active, Inactive.



Source code

`frontend/src/pages/admin/UserManagement.jsx`

```
import { useMemo, useState } from 'react';
import PageHeader from '../../components/PageHeader';
import Card from '../../components/Card';
import Badge from '../../components/Badge';
import Tabs from '../../components/Tabs';
import Avatar from '../../components/Avatar';
import DataTable from '../../components/DataTable';
import SearchInput from '../../components/SearchInput';
import { ROLE_LABELS } from '../../utils/format';
import { users as demoUsers } from '../../data/demoData';

const STATUS_TONE = { ACTIVE: 'success', PENDING: 'warning', INACTIVE: 'neutral' };

export default function UserManagement() {
  const [rows, setRows] = useState(demoUsers); // replace with GET /api/admin/users
  const [tab, setTab] = useState('ALL');
  const [role, setRole] = useState('');
  const [query, setQuery] = useState('');

  const pendingCount = rows.filter((u) => u.status === 'PENDING').length;
  const filtered = useMemo(
    () => rows.filter((u) =>
      (tab === 'ALL' || u.status === tab) &&
      (!role || u.role === role) &&
      `${u.name} ${u.email} ${u.department}`.toLowerCase().includes(query.toLowerCase())),
    [rows, tab, role, query]
  );

  const setStatus = (id, status) => setRows((list) => list.map((u) => (u.id === id ? { ...u, status } : u))); // PATCH /api/admin/users/{id}/status

  return (
```

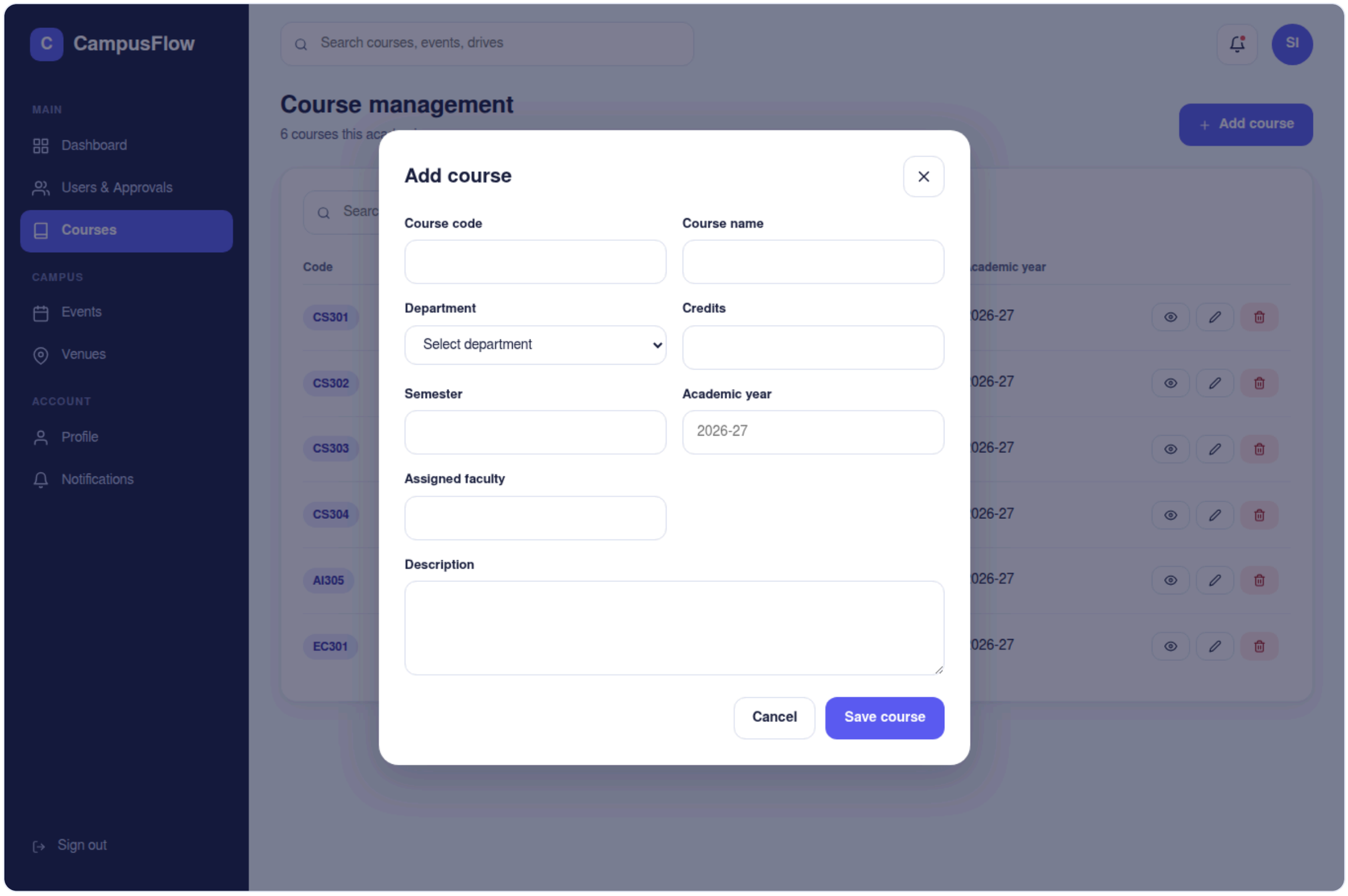
```
    <>
      <PageHeader title="Users and approvals" subtitle="Approve faculty and placement accounts, and manage access." />
      <Tabs value={tab} onChange={setTab} tabs={[
        { value: 'ALL', label: 'All users', count: rows.length },
        { value: 'PENDING', label: 'Pending approval', count: pendingCount },
        { value: 'ACTIVE', label: 'Active' },
        { value: 'INACTIVE', label: 'Inactive' },
      ]} />
      <Card>
        <div className="toolbar" style={{ marginBottom: 16 }}>
          <SearchInput value={query} onChange={setQuery} placeholder="Search by name, email or department" label="Search users" />
          <select className="select" value={role} onChange={(e) => setRole(e.target.value)} aria-label="Filter by role">
            <option value="">All roles</option>
            {Object.entries(ROLE_LABELS).map(([value, label]) => <option key={value} value={value}>{label}</option>)}
          </select>
        </div>
        <DataTable
          columns={[
            { key: 'name', header: 'User', render: (r) => <div className="cell-user"><Avatar name={r.name} size="sm" /><div><div className="fw-700">{r.name}</div><div
 className="muted text-sm">{r.email}</div></div></div> },
            { key: 'role', header: 'Role', render: (r) => <Badge tone="info">{ROLE_LABELS[r.role]}</Badge> },
            { key: 'department', header: 'Department' },
            { key: 'status', header: 'Status', render: (r) => <Badge tone={STATUS_TONE[r.status]}>{r.status.charAt(0) + r.status.slice(1).toLowerCase()}</Badge> },
            { key: 'actions', header: '', className: 'actions', render: (r) => r.status === 'PENDING' ? (
              <><button type="button" className="btn btn-success btn-sm" onClick={() => setStatus(r.id, 'ACTIVE')}>Approve</button><button type="button" className="btn
 btn-danger btn-sm" onClick={() => setStatus(r.id, 'INACTIVE')}>Reject</button></>
            ) : r.status === 'ACTIVE' ? (
              <button type="button" className="btn btn-outline btn-sm" onClick={() => setStatus(r.id, 'INACTIVE')}>Deactivate</button>
            ) : (
              <button type="button" className="btn btn-outline btn-sm" onClick={() => setStatus(r.id, 'ACTIVE')}>Activate</button>
            ) },
          ]}
          rows={filtered}
          emptyTitle="No users found"
          emptyText="Try a different filter or search."
        />
      </Card>
    </>
  );
}
```

Course management (add course dialog)

Route	`/admin/courses`
Role	ADMIN, FACULTY (manage mode)
Page file	`frontend/src/pages/shared/Courses.jsx`

Data needed	`GET /api/courses`
	`POST /api/courses`
	`PUT /api/courses/{id}`
	`DELETE /api/courses/{id}`

Behaviour	Page file shared with the student screen; manage={true} shows Add, Edit and Delete. The screenshot shows the Add course dialog open.



Source code

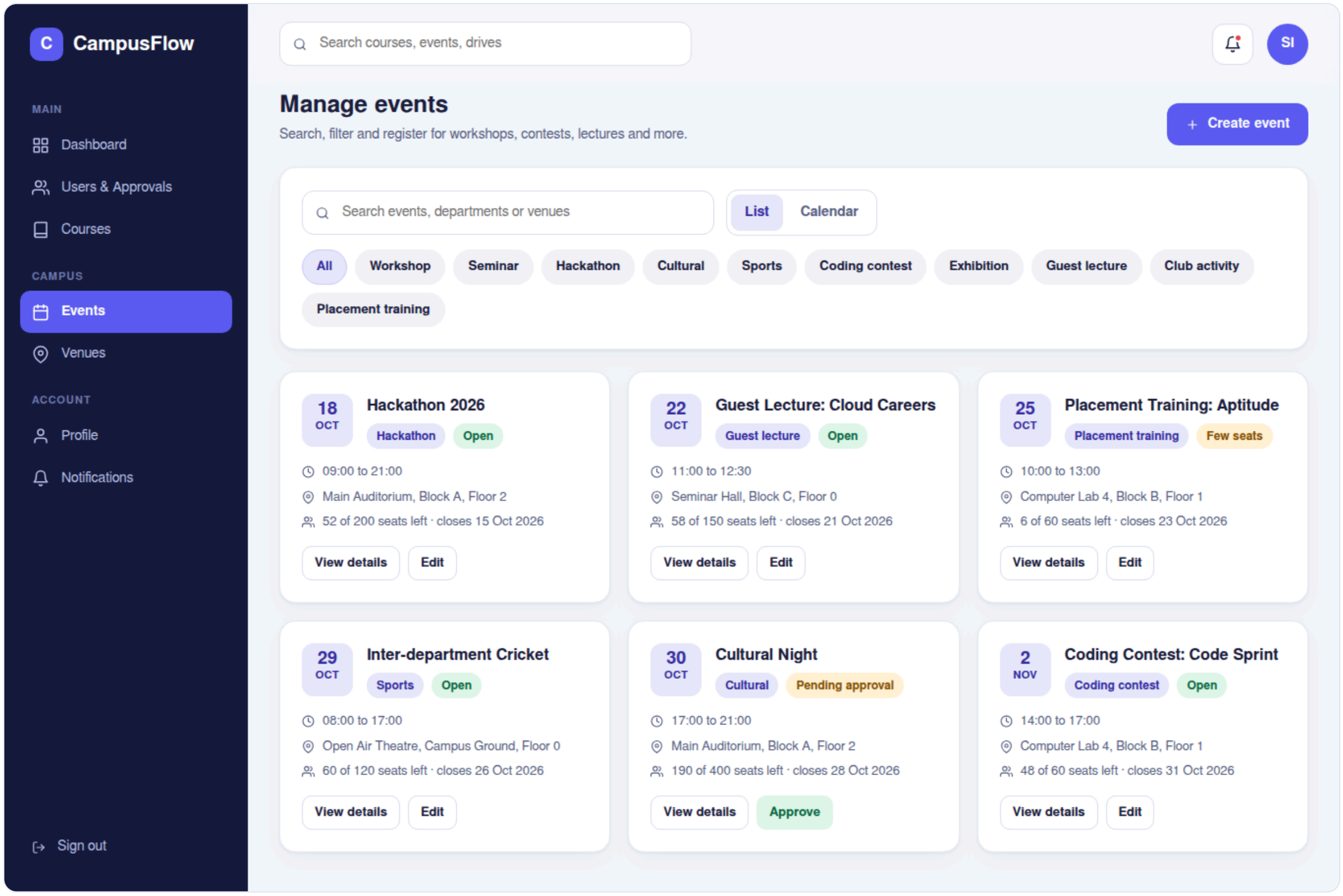
Same file as an earlier interface: `frontend/src/pages/shared/Courses.jsx` (rendered with a different role, props or state).

Manage events (admin)

Route	`/admin/events`
Role	ADMIN, FACULTY (manage mode)
Page file	`frontend/src/pages/shared/Events.jsx`

Data needed	`POST /api/events`
	`PUT /api/events/{id}`
	`PATCH /api/events/{id}/approve`
	`PATCH /api/events/{id}/cancel`

Behaviour	Same page file as the student events screen, rendered with manage={true}: shows Create event, Approve and Edit.

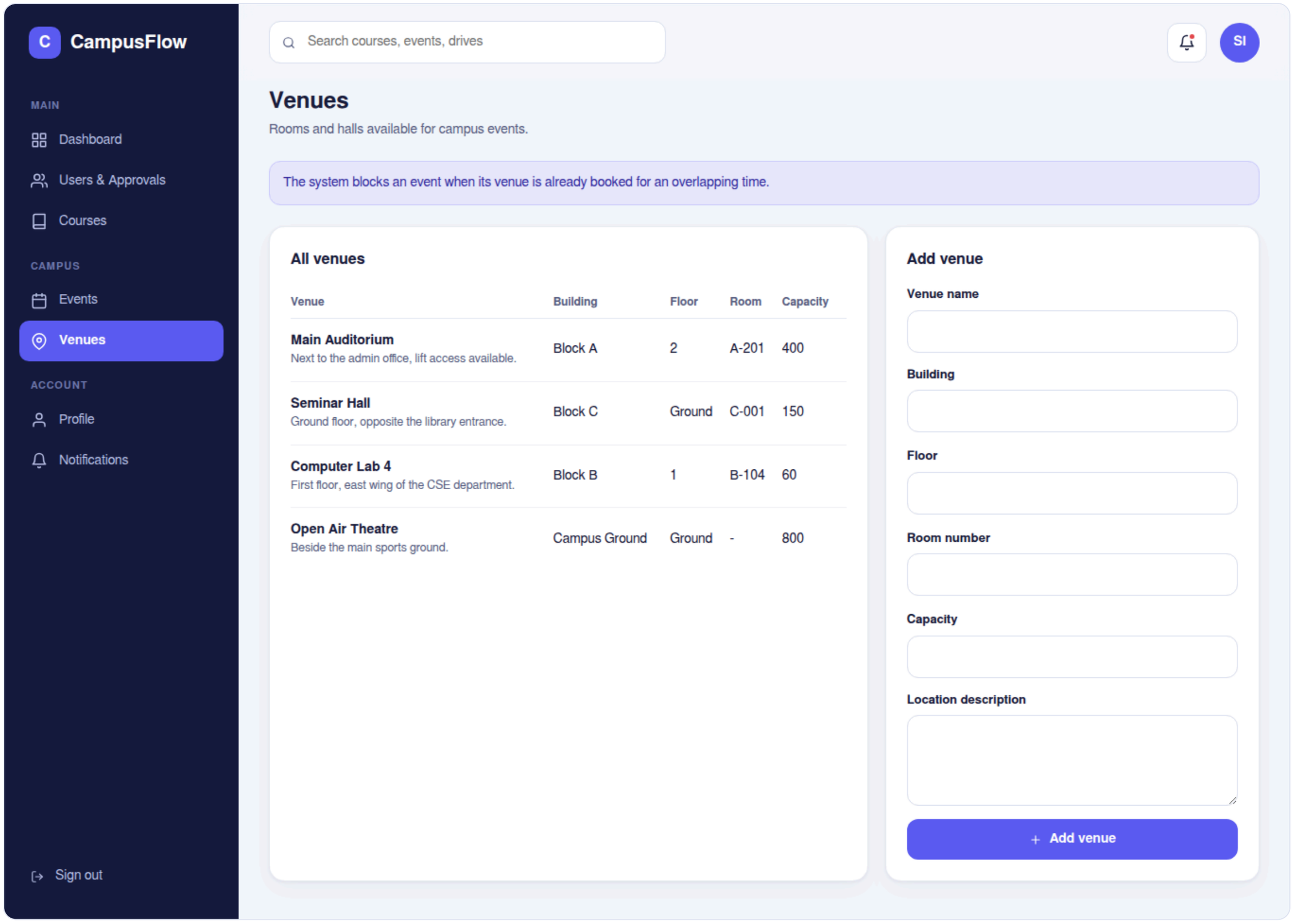


Source code

Same file as an earlier interface: `frontend/src/pages/shared/Events.jsx` (rendered with a different role, props or state).

Venue management

Route	`/admin/venues`
Role	ADMIN
Page file	`frontend/src/pages/admin/Venues.jsx`
Data needed	`GET /api/venues` `POST /api/venues`
Behaviour	Backend must reject an event whose venue is already booked for an overlapping time.



Source code

`frontend/src/pages/admin/Venues.jsx`

```
import { useState } from 'react';
import { FiPlus } from 'react-icons/fi';
import PageHeader from '../../components/PageHeader';
import Card from '../../components/Card';
import Field from '../../components/Field';
import Alert from '../../components/Alert';
import DataTable from '../../components/DataTable';
import { venues as demoVenues } from '../../data/demoData';

const EMPTY = { name: '', building: '', floor: '', room: '', capacity: '', description: '' };

export default function Venues() {
  const [rows, setRows] = useState(demoVenues); // replace with GET /api/venues
  const [form, setForm] = useState(EMPTY);
  const [errors, setErrors] = useState({});
  const update = (key) => (e) => setForm((f) => ({ ...f, [key]: e.target.value }));

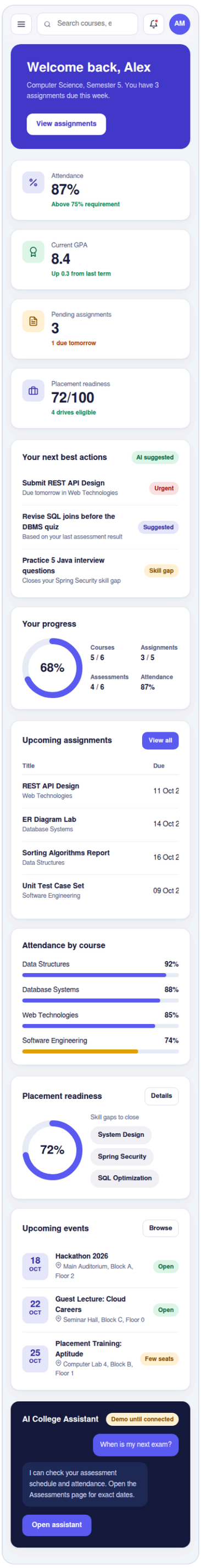
  const add = (e) => {
    e.preventDefault();
    const next = {};
    if (!form.name.trim()) next.name = 'Venue name is required.';
    if (!form.building.trim()) next.building = 'Building is required.';
    if (!(Number(form.capacity) > 0)) next.capacity = 'Enter the capacity.';
    setErrors(next);
    if (Object.keys(next).length) return;
    setRows((r) => [...r, { ...form, id: Date.now(), floor: Number(form.floor) || 0, capacity: Number(form.capacity) }]); // POST /api/venues
    setForm(EMPTY);
  };

  return (
```

```
    <>
      <PageHeader title="Venues" subtitle="Rooms and halls available for campus events." />
      <Alert tone="info">The system blocks an event when its venue is already booked for an overlapping time.</Alert>
      <div className="grid-main">
        <Card title="All venues">
          <DataTable
            columns={[
              { key: 'name', header: 'Venue', render: (r) => <div><div className="fw-700">{r.name}</div><div className="muted text-sm">{r.description}</div></div> },
              { key: 'building', header: 'Building' },
              { key: 'floor', header: 'Floor', render: (r) => (r.floor === 0 ? 'Ground' : r.floor) },
              { key: 'room', header: 'Room' },
              { key: 'capacity', header: 'Capacity' },
            ]}
            rows={rows}
          />
        </Card>
        <Card title="Add venue">
          <form className="stack" style={{ gap: 14 }} onSubmit={add} noValidate>
            <Field label="Venue name" id="v-name" error={errors.name}><input className="input" value={form.name} onChange={update('name')} /></Field>
            <div className="form-grid">
              <Field label="Building" id="v-building" error={errors.building}><input className="input" value={form.building} onChange={update('building')} /></Field>
              <Field label="Floor" id="v-floor"><input className="input" type="number" min="0" value={form.floor} onChange={update('floor')} /></Field>
              <Field label="Room number" id="v-room"><input className="input" value={form.room} onChange={update('room')} /></Field>
              <Field label="Capacity" id="v-cap" error={errors.capacity}><input className="input" type="number" min="1" value={form.capacity} onChange=
{update('capacity')} /></Field>
            </div>
            <Field label="Location description" id="v-desc"><textarea className="textarea" value={form.description} onChange={update('description')} /></Field>
            <button type="submit" className="btn btn-primary btn-block"><FiPlus /> Add venue</button>
          </form>
        </Card>
      </div>
    </>
  );
}
```

Student dashboard on a phone (responsive)

Route	`/student`
Role	STUDENT
Page file	`frontend/src/pages/student/StudentDashboard.jsx`
Data needed	`Same as student dashboard`
Behaviour	Responsive layout under 900px: the sidebar becomes a slide-in drawer opened from the menu button, and cards stack in one column.



Source code

Same file as an earlier interface: `frontend/src/pages/student/StudentDashboard.jsx` (rendered with a different role, props or state).

Appendix A. Shared source code

Every remaining file of the frontend, in full.

Routing, entry and auth

frontend/src/App.jsx

```
import { useRoutes } from 'react-router-dom';
import routes from './routes/routes';

export default function App() {
  return useRoutes(routes);
}
```

frontend/src/context/AuthContext.jsx

```
import { createContext, useCallback, useContext, useMemo, useState } from 'react';
import { loginRequest } from '../services/authService';
import { SESSION_KEY } from '../services/api';

const AuthContext = createContext(null);

function readSession() {
  try {
    return JSON.parse(localStorage.getItem(SESSION_KEY));
  } catch {
    return null;
  }
}

export function AuthProvider({ children }) {
  const [session, setSession] = useState(readSession);

  const login = useCallback(async (email, password) => {
    const data = await loginRequest(email, password);
    localStorage.setItem(SESSION_KEY, JSON.stringify(data));
    setSession(data);
    return data.user;
  }, []);

  const logout = useCallback(() => {
    localStorage.removeItem(SESSION_KEY);
    setSession(null);
  }, []);

  const value = useMemo(
    () => ({
      user: session?.user ?? null,
      token: session?.token ?? null,
      isAuthenticated: Boolean(session?.token),
      login,
      logout,
    }),
    [session, login, logout]
  );

  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

export const useAuth = () => useContext(AuthContext);
```

frontend/src/main.jsx

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import App from './App';
import { AuthProvider } from './context/AuthContext';
import './styles/global.css';
import './styles/layout.css';
import './styles/components.css';
import './styles/auth.css';
import './styles/pages.css';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <BrowserRouter>
      <AuthProvider>
        <App />
      </AuthProvider>
    </BrowserRouter>
  </React.StrictMode>
);
```

frontend/src/routes/ProtectedRoute.jsx

```
import { Navigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
import { ROLE_HOME } from '../utils/format';

// Frontend guard only. The backend must enforce the same roles on every API.
export default function ProtectedRoute({ roles, children }) {
  const { isAuthenticated, user } = useAuth();
  if (!isAuthenticated) return <Navigate to="/login" replace />;
  if (roles && !roles.includes(user.role)) return <Navigate to={ROLE_HOME[user.role]} replace />;
  return children;
}
```

frontend/src/routes/routes.jsx

```
import { Navigate } from 'react-router-dom';
import AppLayout from '../layouts/AppLayout';
import AuthLayout from '../layouts/AuthLayout';
import ProtectedRoute from './ProtectedRoute';
import Login from '../pages/auth/Login';
import Register from '../pages/auth/Register';
import StudentDashboard from '../pages/student/StudentDashboard';
import Attendance from '../pages/student/Attendance';
import Results from '../pages/student/Results';
import Skills from '../pages/student/Skills';
import Placements from '../pages/student/Placements';
import FacultyDashboard from '../pages/faculty/FacultyDashboard';
import ManageAttendance from '../pages/faculty/ManageAttendance';
import PlacementDashboard from '../pages/placement/PlacementDashboard';
import DriveManagement from '../pages/placement/DriveManagement';
import AdminDashboard from '../pages/admin/AdminDashboard';
import UserManagement from '../pages/admin/UserManagement';
import Venues from '../pages/admin/Venues';
import Courses from '../pages/shared/Courses';
import Assignments from '../pages/shared/Assignments';
import Events from '../pages/shared/Events';
import EventDetails from '../pages/shared/EventDetails';
import Notifications from '../pages/shared/Notifications';
import AIAssistant from '../pages/shared/AIAssistant';
import Profile from '../pages/shared/Profile';
import SearchResults from '../pages/shared/SearchResults';

const shared = (manageEvents = false) => [
  { path: 'events', element: <Events manage={manageEvents} /> },
  { path: 'events/:id', element: <EventDetails /> },
  { path: 'notifications', element: <Notifications /> },
  { path: 'profile', element: <Profile /> },
  { path: 'search', element: <SearchResults /> },
];

const guarded = (role, path, children) => ({
  path,
  element: (
    <ProtectedRoute roles={[role]}>
      <AppLayout role={role} />
    </ProtectedRoute>
  ),
  children,
});

const routes = [
  { path: '/', element: <Navigate to="/login" replace /> },
  {
    element: <AuthLayout />,
    children: [
      { path: '/login', element: <Login /> },
      { path: '/register', element: <Register /> },
    ],
  },
  guarded('STUDENT', '/student', [
    { index: true, element: <StudentDashboard /> },
    { path: 'courses', element: <Courses /> },
    { path: 'assignments', element: <Assignments mode="student" /> },
    { path: 'attendance', element: <Attendance /> },
    { path: 'results', element: <Results /> },
    { path: 'placements', element: <Placements /> },
    { path: 'skills', element: <Skills /> },
    { path: 'assistant', element: <AIAssistant /> },
    ...shared(),
  ]),
  guarded('FACULTY', '/faculty', [
    { index: true, element: <FacultyDashboard /> },
    { path: 'courses', element: <Courses manage /> },
    { path: 'assignments', element: <Assignments mode="faculty" /> },
    { path: 'attendance', element: <ManageAttendance /> },
    { path: 'assistant', element: <AIAssistant /> },
    ...shared(true),
  ]),
  guarded('PLACEMENT_OFFICER', '/placement', [
    { index: true, element: <PlacementDashboard /> },
    { path: 'drives', element: <DriveManagement /> },
    { path: 'assistant', element: <AIAssistant /> },
    ...shared(),
  ]),
  guarded('ADMIN', '/admin', [
    { index: true, element: <AdminDashboard /> },
    { path: 'users', element: <UserManagement /> },
    { path: 'courses', element: <Courses manage /> },
    { path: 'venues', element: <Venues /> },
    ...shared(true),
  ]),
  { path: '*', element: <Navigate to="/login" replace /> },
];

export default routes;
```

Layouts

frontend/src/layouts/AppLayout.jsx

```
import { useState } from 'react';
import { Outlet, useNavigate } from 'react-router-dom';
import Sidebar from './Sidebar';
import Topbar from './Topbar';
import { useAuth } from '../context/AuthContext';

export default function AppLayout({ role }) {
  const { user, logout } = useAuth();
  const navigate = useNavigate();
  const [open, setOpen] = useState(false);

  const signOut = () => {
    logout();
    navigate('/login', { replace: true });
  };

  return (
    <div className="app">
      <Sidebar role={role} open={open} onNavigate={() => setOpen(false)} onSignOut={signOut} />
      <div className={`scrim ${open ? 'show' : ''}`} onClick={() => setOpen(false)} />
      <div className="main">
        <Topbar user={user} role={role} onMenu={() => setOpen(true)} onSignOut={signOut} />
        <main className="content">
          <Outlet />
        </main>
      </div>
    </div>
  );
}
```

frontend/src/layouts/AuthLayout.jsx

```
import { Outlet } from 'react-router-dom';
import { FiTarget, FiMessageSquare, FiCalendar } from 'react-icons/fi';

export default function AuthLayout() {
  return (
    <div className="auth">
      <section className="auth__brand" aria-label="About CampusFlow">
        <div className="auth__logo">
          <span className="brand-mark">C</span>
          CampusFlow
        </div>
        <div>
          <h2 className="auth__headline">Smarter learning. Personalized guidance. Better campus management.</h2>
          <p className="auth__sub">One platform for students, faculty, placement officers and administrators.</p>
          <div className="auth__points">
            <div className="auth__point">
              <span className="icon-tile"><FiTarget aria-hidden="true" /></span>
              <div><b>Role-based workspace</b><span>See only what matters for your role and department.</span></div>
            </div>
            <div className="auth__point">
              <span className="icon-tile"><FiMessageSquare aria-hidden="true" /></span>
              <div><b>AI College Assistant</b><span>Ask about academics, events, placements and services.</span></div>
            </div>
            <div className="auth__point">
              <span className="icon-tile"><FiCalendar aria-hidden="true" /></span>
              <div><b>Campus events and venues</b><span>Find, register for and locate every event on campus.</span></div>
            </div>
          </div>
        </div>
        <p className="auth__foot">CampusFlow · Smarter Learning. Personalized Guidance.</p>
      </section>
      <section className="auth__panel">
        <Outlet />
      </section>
    </div>
  );
}
```

frontend/src/layouts/Sidebar.jsx

```
import { NavLink } from 'react-router-dom';
import { FiLogOut } from 'react-icons/fi';
import { NAV } from '../utils/navConfig';

export default function Sidebar({ role, open, onNavigate, onSignOut }) {
  return (
    <aside className={`sidebar ${open ? 'open' : ''}`} aria-label="Main navigation">
      <div className="sidebar__brand">
        <span className="brand-mark">C</span>
        CampusFlow
      </div>
      <nav>
        {NAV[role].map((group) => (
          <div key={group.label}>
            <div className="nav-label">{group.label}</div>
            {group.items.map(({ to, label, icon: Icon, end }) => (
              <NavLink
                key={to}
                to={to}
                end={end}
                onClick={onNavigate}
                className={({ isActive }) => `nav-link ${isActive ? 'active' : ''}`}
              >
                <Icon aria-hidden="true" />
                {label}
              </NavLink>
            ))}
```

```
          </div>
        ))}
      </nav>
      <div className="sidebar__foot">
        <button type="button" className="sidebar__signout" onClick={onSignOut}>
          <FiLogOut aria-hidden="true" /> Sign out
        </button>
      </div>
    </aside>
  );
}
```

frontend/src/layouts/Topbar.jsx

```
import { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { FiMenu, FiBell, FiSearch, FiUser, FiLogOut } from 'react-icons/fi';
import Avatar from '../components/Avatar';
import { ROLE_LABELS } from '../utils/format';
import { BASE_PATH } from '../utils/navConfig';

export default function Topbar({ user, role, onMenu, onSignOut }) {
  const navigate = useNavigate();
  const [query, setQuery] = useState('');
  const [menuOpen, setMenuOpen] = useState(false);
  const base = BASE_PATH[role];

  const submit = (e) => {
    e.preventDefault();
    if (query.trim()) navigate(`${base}/search?q=${encodeURIComponent(query.trim())}`);
  };

  return (
    <header className="topbar">
      <button type="button" className="btn btn-icon menu-btn" onClick={onMenu} aria-label="Open menu">
        <FiMenu />
      </button>
      <form className="search" role="search" onSubmit={submit}>
        <FiSearch aria-hidden="true" />
        <input
          className="input"
          type="search"
          value={query}
          onChange={(e) => setQuery(e.target.value)}
          placeholder="Search courses, events, drives"
          aria-label="Search courses, assignments, events and placement drives"
        />
      </form>
      <div className="topbar__spacer" />
      <button type="button" className="btn btn-icon bell" onClick={() => navigate(`${base}/notifications`)} aria-label="Notifications, unread">
        <FiBell />
        <span className="bell__dot" />
      </button>
      <div className="menu-wrap">
        <button type="button" className="avatar-btn" onClick={() => setMenuOpen((v) => !v)} aria-haspopup="menu" aria-expanded={menuOpen} aria-label="Account menu">
          <Avatar name={user?.name} />
        </button>
        {menuOpen && (
          <div className="dropdown" role="menu">
            <div className="dropdown__who">
              <div className="fw-700">{user?.name}</div>
              <div className="muted text-sm">{ROLE_LABELS[role]}</div>
            </div>
            <button type="button" className="dropdown__item" role="menuitem" onClick={() => { setMenuOpen(false); navigate(`${base}/profile`); }}>
              <FiUser aria-hidden="true" /> Profile
            </button>
            <button type="button" className="dropdown__item" role="menuitem" onClick={onSignOut}>
              <FiLogOut aria-hidden="true" /> Sign out
            </button>
          </div>
        )}
      </div>
    </header>
  );
}
```

Reusable components

frontend/src/components/Alert.jsx

```
// tone: success | error | info | warning
export default function Alert({ tone = 'info', children }) {
  return (
    <div className={`alert alert-${tone}`} role={tone === 'error' ? 'alert' : 'status'}>
      {children}
    </div>
  );
}
```

frontend/src/components/Avatar.jsx

```
import { initials } from '../utils/format';

export default function Avatar({ name, size = '' }) {
  return (
    <span className={`avatar ${size ? `avatar-${size}` : ''}`} aria-hidden="true">
      {initials(name)}
    </span>
  );
}
```

frontend/src/components/Badge.jsx

```
// tone: success | warning | danger | info | neutral
export default function Badge({ tone = 'neutral', children }) {
  return <span className={`badge badge-${tone}`}>{children}</span>;
}
```

frontend/src/components/BarChart.jsx

```
// data: [{ label, value, tone? }]  tone: '' | 'alt' | 'warn'
export default function BarChart({ data, max, unit = '' }) {
  const top = max ?? Math.max(...data.map((d) => d.value), 1);
  return (
    <div className="bars" role="img" aria-label="Bar chart">
      {data.map((d) => (
        <div className="bars__col" key={d.label}>
          <span className="bars__val">{d.value}{unit}</span>
          <div className={`bars__fill ${d.tone || ''}`} style={{ height: `${(d.value / top) * 78}%` }} />
          <span className="bars__label">{d.label}</span>
        </div>
      ))}
    </div>
  );
}
```

frontend/src/components/Card.jsx

```
export default function Card({ title, action, children, className = '' }) {
  return (
    <section className={`card ${className}`}>
      {(title || action) && (
        <header className="card__head">
          <h3>{title}</h3>
          {action}
        </header>
      )}
      {children}
    </section>
  );
}
```

frontend/src/components/DataTable.jsx

```
import EmptyState from './EmptyState';

// columns: [{ key, header, render?(row), className? }]
export default function DataTable({ columns, rows, rowKey = 'id', emptyTitle, emptyText }) {
  return (
    <div className="table-wrap">
      <table className="table">
        <thead>
          <tr>
            {columns.map((c) => (
              <th key={c.key} scope="col" className={c.className}>{c.header}</th>
            ))}
          </tr>
        </thead>
        <tbody>
          {rows.length === 0 ? (
            <tr>
              <td colSpan={columns.length}>
                <EmptyState title={emptyTitle} text={emptyText} />
              </td>
            </tr>
          ) : (
            rows.map((row) => (
              <tr key={row[rowKey]}>
                {columns.map((c) => (
                  <td key={c.key} className={c.className}>{c.render ? c.render(row) : row[c.key]}</td>
                ))}
              </tr>
```

```
            ))
          )}
        </tbody>
      </table>
    </div>
  );
}
```

frontend/src/components/EmptyState.jsx

```
import { FiInbox } from 'react-icons/fi';

export default function EmptyState({ icon: Icon = FiInbox, title = 'Nothing here yet', text, action }) {
  return (
    <div className="empty">
      <Icon aria-hidden="true" />
      <h4>{title}</h4>
      {text && <p>{text}</p>}
      {action && <div className="mt-16">{action}</div>}
    </div>
  );
}
```

frontend/src/components/EventCard.jsx

```
import { Link } from 'react-router-dom';
import { FiMapPin, FiClock, FiUsers } from 'react-icons/fi';
import Badge from './Badge';
import { dayOf, monthShort, formatDate } from '../utils/format';

export function seatStatus(event) {
  const left = event.total - event.registered;
  if (left <= 0) return { tone: 'danger', label: 'Full', left };
  if (left <= 10) return { tone: 'warning', label: 'Few seats', left };
  return { tone: 'success', label: 'Open', left };
}

export default function EventCard({ event, basePath, manage, onRegister }) {
  const status = seatStatus(event);
  return (
    <article className="card event-card">
      <div className="event-card__top">
        <div className="date-tile">{dayOf(event.date)}<small>{monthShort(event.date)}</small></div>
        <div className="grow">
          <h3 style={{ fontSize: 16, fontWeight: 700 }}>{event.title}</h3>
          <div className="flex gap-8 wrap" style={{ marginTop: 6 }}>
            <Badge tone="info">{event.category}</Badge>
            {manage && event.approval === 'PENDING' && <Badge tone="warning">Pending approval</Badge>}
            {(!manage || event.approval === 'APPROVED') && <Badge tone={status.tone}>{status.label}</Badge>}
          </div>
        </div>
      </div>
      <div className="event-card__meta">
        <span><FiClock aria-hidden="true" /> {event.start} to {event.end}</span>
        <span><FiMapPin aria-hidden="true" /> {event.venue}, {event.building}, Floor {event.floor}</span>
        <span><FiUsers aria-hidden="true" /> {status.left} of {event.total} seats left · closes {formatDate(event.deadline)}</span>
      </div>
      <div className="event-card__foot">
        <Link className="btn btn-outline btn-sm" to={`${basePath}/events/${event.id}`}>View details</Link>
        {manage ? (
          event.approval === 'PENDING' ? <button type="button" className="btn btn-success btn-sm">Approve</button>
            : <button type="button" className="btn btn-outline btn-sm">Edit</button>
        ) : (
          <button type="button" className="btn btn-primary btn-sm" disabled={status.left <= 0} onClick={() => onRegister?.(event)}>
            Register
          </button>
        )}
      </div>
    </article>
  );
}
```

frontend/src/components/Field.jsx

```
import { cloneElement } from 'react';

// Wraps one input/select/textarea with label, hint and error. Pass the control as children.
export default function Field({ label, id, error, hint, children }) {
  const describedBy = error ? `${id}-error` : hint ? `${id}-hint` : undefined;
  return (
    <div className="field">
      <label htmlFor={id}>{label}</label>
      {cloneElement(children, {
        id,
        'aria-invalid': error ? 'true' : undefined,
        'aria-describedby': describedBy,
        className: `${children.props.className || ''} ${error ? 'error' : ''}`.trim(),
      })}
      {error && <p className="field-error" id={`${id}-error`} role="alert">{error}</p>}
      {!error && hint && <p className="field-hint" id={`${id}-hint`}>{hint}</p>}
    </div>
  );
}
```

frontend/src/components/Hero.jsx

```
export default function Hero({ title, subtitle, action }) {
  return (
    <section className="hero">
      <div>
        <h1>{title}</h1>
        <p>{subtitle}</p>
      </div>
      {action}
    </section>
```

```
  );
}
```

frontend/src/components/MeterRow.jsx

```
import ProgressBar from './ProgressBar';

export default function MeterRow({ label, value, display, tone, warnBelow }) {
  return (
    <div className="meter-row">
      <div className="meter-row__top">
        <span>{label}</span>
        <b>{display ?? `${value}%`}</b>
      </div>
      <ProgressBar value={value} tone={tone} warnBelow={warnBelow} label={label} />
    </div>
  );
}
```

frontend/src/components/Modal.jsx

```
import { useEffect } from 'react';
import { FiX } from 'react-icons/fi';

export default function Modal({ open, title, onClose, children, footer }) {
  useEffect(() => {
    if (!open) return undefined;
    const onKey = (e) => e.key === 'Escape' && onClose();
    document.addEventListener('keydown', onKey);
    return () => document.removeEventListener('keydown', onKey);
  }, [open, onClose]);

  if (!open) return null;
  return (
    <div className="modal-backdrop" onMouseDown={(e) => e.target === e.currentTarget && onClose()}>
      <div className="modal" role="dialog" aria-modal="true" aria-label={title}>
        <div className="modal__head">
          <h2>{title}</h2>
          <button type="button" className="btn btn-icon" onClick={onClose} aria-label="Close dialog">
            <FiX />
          </button>
        </div>
        {children}
        {footer && <div className="modal__foot">{footer}</div>}
      </div>
    </div>
  );
}
```

frontend/src/components/PageHeader.jsx

```
export default function PageHeader({ title, subtitle, actions }) {
  return (
    <div className="page-head">
      <div>
        <h1>{title}</h1>
        {subtitle && <p>{subtitle}</p>}
      </div>
      {actions && <div className="flex gap-8 wrap">{actions}</div>}
    </div>
  );
}
```

frontend/src/components/ProgressBar.jsx

```
// Turns amber below `warnBelow` (default 75) unless a tone is forced.
export default function ProgressBar({ value, tone, warnBelow = 75, label }) {
  const auto = value < warnBelow ? 'warning' : '';
  return (
    <div
      className={`progress ${tone || auto}`}
      role="progressbar"
      aria-valuenow={value}
      aria-valuemin={0}
      aria-valuemax={100}
      aria-label={label}
    >
      <span style={{ width: `${Math.min(100, Math.max(0, value))}%` }} />
    </div>
  );
}
```

frontend/src/components/RingChart.jsx

```
export default function RingChart({ value, size = 120, stroke = 12, label }) {
  const r = (size - stroke) / 2;
  const c = 2 * Math.PI * r;
  const dash = (Math.min(100, Math.max(0, value)) / 100) * c;
  return (
    <svg className="ring" width={size} height={size} viewBox={`0 0 ${size} ${size}`} role="img" aria-label={label || `${value} percent`}>
      <circle className="ring__track" cx={size / 2} cy={size / 2} r={r} fill="none" strokeWidth={stroke} />
      <circle
        className="ring__value"
        cx={size / 2}
        cy={size / 2}
        r={r}
        fill="none"
        strokeWidth={stroke}
        strokeLinecap="round"
        strokeDasharray={`${dash} ${c}`}
        transform={`rotate(-90 ${size / 2} ${size / 2})`}
      />
      <text className="ring__text" x="50%" y="50%" textAnchor="middle" dominantBaseline="central" fontSize={size / 5}>
```

```
        {value}%
      </text>
    </svg>
  );
}
```

frontend/src/components/SearchInput.jsx

```
import { FiSearch, FiX } from 'react-icons/fi';

export default function SearchInput({ value, onChange, placeholder = 'Search', label = 'Search' }) {
  return (
    <div className="search">
      <FiSearch aria-hidden="true" />
      <input
        className="input"
        type="search"
        value={value}
        onChange={(e) => onChange(e.target.value)}
        placeholder={placeholder}
        aria-label={label}
      />
      {value && (
        <button type="button" className="search__clear" onClick={() => onChange('')} aria-label="Clear search">
          <FiX />
        </button>
      )}
    </div>
  );
}
```

frontend/src/components/Skeleton.jsx

```
export default function Skeleton({ height = 16, width = '100%' }) {
  return <span className="skeleton" style={{ height, width }} aria-hidden="true" />;
}
```

frontend/src/components/StatCard.jsx

```
export default function StatCard({ label, value, hint, tone = 'primary', icon: Icon, trend = 'up' }) {
  return (
    <div className="card stat">
      {Icon && (
        <span className={`stat__icon tone-${tone}`}>
          <Icon aria-hidden="true" />
        </span>
      )}
      <div>
        <p className="stat__label">{label}</p>
        <p className="stat__value">{value}</p>
        {hint && <p className={`stat__hint ${trend}`}>{hint}</p>}
      </div>
    </div>
  );
}
```

frontend/src/components/Tabs.jsx

```
// tabs: [{ value, label, count? }]
export default function Tabs({ tabs, value, onChange }) {
  return (
    <div className="tabs" role="tablist">
      {tabs.map((t) => (
        <button
          key={t.value}
          type="button"
          role="tab"
          aria-selected={value === t.value}
          className={`tab ${value === t.value ? 'active' : ''}`}
          onClick={() => onChange(t.value)}
        >
          {t.label}
          {t.count !== undefined && <small>{t.count}</small>}
        </button>
      ))}
    </div>
  );
}
```

Services and utilities

frontend/src/services/api.js

```
import axios from 'axios';

export const SESSION_KEY = 'campusflow_session';

const api = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL || 'http://localhost:5454/api',
  headers: { 'Content-Type': 'application/json' },
});

api.interceptors.request.use((config) => {
  try {
    const session = JSON.parse(localStorage.getItem(SESSION_KEY));
    if (session?.token) config.headers.Authorization = `Bearer ${session.token}`;
  } catch {
    /* ignore corrupted session */
  }
  return config;
});

api.interceptors.response.use(
  (response) => response,
  (error) => {
    const isLoginCall = error.config?.url?.includes('/auth/login');
    if (error.response?.status === 401 && !isLoginCall) {
      localStorage.removeItem(SESSION_KEY);
      window.location.assign('/login?expired=1');
    }
    return Promise.reject(error);
  }
);

export default api;
```

frontend/src/services/assistantService.js

```
import api from './api';
import { isDemoMode } from './authService';

// Expected backend response: { reply: string, mode: 'LIVE' | 'DEMO' }
export async function askAssistant(message) {
  if (isDemoMode) {
    return {
      reply: 'Demo response: the AI service is not connected. This answer is not verified college information.',
      mode: 'DEMO',
    };
  }
  const { data } = await api.post('/assistant/chat', { message });
  return data;
}
```

frontend/src/services/authService.js

```
import api from './api';

export const isDemoMode = import.meta.env.VITE_DEMO_MODE === 'true';

const DEMO_USERS = {
  student: { id: 1, name: 'Alex Morgan', email: 'student@campusflow.test', role: 'STUDENT', department: 'Computer Science' },
  faculty: { id: 2, name: 'Dr. Priya Nair', email: 'faculty@campusflow.test', role: 'FACULTY', department: 'Computer Science' },
  placement: { id: 3, name: 'Rahul Verma', email: 'placement@campusflow.test', role: 'PLACEMENT_OFFICER', department: 'Training & Placement' },
  admin: { id: 4, name: 'Sana Iqbal', email: 'admin@campusflow.test', role: 'ADMIN', department: 'Administration' },
};

// Expected backend response: { token, user: { id, name, email, role, department } }
export async function loginRequest(email, password) {
  if (isDemoMode) {
    const key = Object.keys(DEMO_USERS).find((k) => email.toLowerCase().startsWith(k));
    if (!key) throw new Error('Demo mode: start the email with student, faculty, placement or admin.');
    return { token: 'demo-token', user: DEMO_USERS[key] };
  }
  const { data } = await api.post('/auth/login', { email, password });
  return data;
}

export async function registerRequest(payload) {
  if (isDemoMode) return { message: 'Demo mode: registration was not saved.' };
  const { data } = await api.post('/auth/register', payload);
  return data;
}
```

frontend/src/services/searchService.js

```
import api from './api';
import { isDemoMode } from './authService';
import { courses, assignments, events, drives } from '../data/demoData';

// Expected backend response:
// { courses: [{id,title,subtitle}], assignments: [...], events: [...], drives: [...] }
export async function searchAll(query) {
  const q = query.trim().toLowerCase();
  if (!q) return { courses: [], assignments: [], events: [], drives: [] };
  if (isDemoMode) {
    const has = (text) => text.toLowerCase().includes(q);
    return {
      courses: courses.filter((c) => has(c.name + c.code)).map((c) => ({ id: c.id, title: c.name, subtitle: `${c.code} · ${c.department}` })),
      assignments: assignments.filter((a) => has(a.title + a.course)).map((a) => ({ id: a.id, title: a.title, subtitle: a.course })),
      events: events.filter((e) => has(e.title + e.category)).map((e) => ({ id: e.id, title: e.title, subtitle: `${e.category} · ${e.venue}` })),
```

```
      drives: drives.filter((d) => has(d.company + d.role)).map((d) => ({ id: d.id, title: d.company, subtitle: d.role })),
    };
  }
  const { data } = await api.get('/search', { params: { q: query } });
  return data;
}
```

frontend/src/utils/format.js

```
export const ROLE_LABELS = {
  STUDENT: 'Student',
  FACULTY: 'Faculty',
  PLACEMENT_OFFICER: 'Placement Officer',
  ADMIN: 'Administrator',
};

export const ROLE_HOME = {
  STUDENT: '/student',
  FACULTY: '/faculty',
  PLACEMENT_OFFICER: '/placement',
  ADMIN: '/admin',
};

export const initials = (name = '') =>
  name
    .split(' ')
    .filter(Boolean)
    .slice(0, 2)
    .map((part) => part[0].toUpperCase())
    .join('');

export const firstName = (name = '') => name.split(' ')[0];

const MONTHS = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'];

export function parseDate(iso) {
  const [y, m, d] = iso.split('-').map(Number);
  return { year: y, month: m - 1, day: d };
}

export function formatDate(iso) {
  const { year, month, day } = parseDate(iso);
  return `${String(day).padStart(2, '0')} ${MONTHS[month]} ${year}`;
}

export const monthShort = (iso) => MONTHS[parseDate(iso).month].toUpperCase();
export const dayOf = (iso) => String(parseDate(iso).day);

export function getErrorMessage(error) {
  return (
    error?.response?.data?.message ||
    error?.message ||
    'Something went wrong. Please try again.'
  );
}
```

frontend/src/utils/navConfig.js

```
import {
  FiGrid, FiBook, FiFileText, FiCheckSquare, FiAward, FiCalendar, FiBriefcase,
  FiTrendingUp, FiMessageSquare, FiUser, FiBell, FiUsers, FiMapPin,
} from 'react-icons/fi';

const common = (base) => ({
  assistant: { to: `${base}/assistant`, label: 'AI Assistant', icon: FiMessageSquare },
  profile: { to: `${base}/profile`, label: 'Profile', icon: FiUser },
  notifications: { to: `${base}/notifications`, label: 'Notifications', icon: FiBell },
  events: { to: `${base}/events`, label: 'Events', icon: FiCalendar },
});

const s = common('/student');
const f = common('/faculty');
const p = common('/placement');
const a = common('/admin');

export const NAV = {
  STUDENT: [
    { label: 'Main', items: [
      { to: '/student', label: 'Dashboard', icon: FiGrid, end: true },
      { to: '/student/courses', label: 'Courses', icon: FiBook },
      { to: '/student/assignments', label: 'Assignments', icon: FiFileText },
      { to: '/student/attendance', label: 'Attendance', icon: FiCheckSquare },
      { to: '/student/results', label: 'Results', icon: FiAward },
    ] },
    { label: 'Campus', items: [
      s.events,
      { to: '/student/placements', label: 'Placements', icon: FiBriefcase },
      { to: '/student/skills', label: 'Skills & Path', icon: FiTrendingUp },
      s.assistant,
    ] },
    { label: 'Account', items: [s.profile, s.notifications] },
  ],
  FACULTY: [
    { label: 'Main', items: [
      { to: '/faculty', label: 'Dashboard', icon: FiGrid, end: true },
      { to: '/faculty/courses', label: 'My Courses', icon: FiBook },
      { to: '/faculty/assignments', label: 'Assignments', icon: FiFileText },
      { to: '/faculty/attendance', label: 'Attendance', icon: FiCheckSquare },
    ] },
    { label: 'Campus', items: [f.events, f.assistant] },
    { label: 'Account', items: [f.profile, f.notifications] },
  ],
  PLACEMENT_OFFICER: [
    { label: 'Main', items: [
      { to: '/placement', label: 'Dashboard', icon: FiGrid, end: true },
      { to: '/placement/drives', label: 'Placement Drives', icon: FiBriefcase },
    ] },
    { label: 'Campus', items: [p.events, p.assistant] },
    { label: 'Account', items: [p.profile, p.notifications] },
```

```
  ],
  ADMIN: [
    { label: 'Main', items: [
      { to: '/admin', label: 'Dashboard', icon: FiGrid, end: true },
      { to: '/admin/users', label: 'Users & Approvals', icon: FiUsers },
      { to: '/admin/courses', label: 'Courses', icon: FiBook },
    ] },
    { label: 'Campus', items: [a.events, { to: '/admin/venues', label: 'Venues', icon: FiMapPin }] },
    { label: 'Account', items: [a.profile, a.notifications] },
  ],
};

export const BASE_PATH = {
  STUDENT: '/student',
  FACULTY: '/faculty',
  PLACEMENT_OFFICER: '/placement',
  ADMIN: '/admin',
};
```

Stylesheets

frontend/src/styles/auth.css

```
/* Login and registration */
.auth { display: grid; grid-template-columns: minmax(0, 1.05fr) minmax(0, 1fr); min-height: 100vh; }
.auth__brand { background: var(--sidebar); color: #fff; padding: 48px 56px; display: flex; flex-direction: column; justify-content: space-between; position: relative;
overflow: hidden; }
.auth__brand::before, .auth__brand::after { content: ''; position: absolute; border-radius: 50%; background: var(--primary); opacity: 0.18; }
.auth__brand::before { width: 420px; height: 420px; right: -160px; top: -140px; }
.auth__brand::after { width: 300px; height: 300px; left: -120px; bottom: -110px; background: var(--primary-dark); opacity: 0.5; }
.auth__brand > * { position: relative; z-index: 1; }
.auth__logo { display: flex; align-items: center; gap: 12px; font-weight: 800; font-size: 24px; }
.auth__headline { font-size: 40px; line-height: 1.15; font-weight: 800; max-width: 460px; }
.auth__sub { color: var(--sidebar-text); margin-top: 14px; font-size: 16px; max-width: 440px; }
.auth__points { display: grid; gap: 14px; margin-top: 32px; max-width: 460px; }
.auth__point { display: flex; gap: 14px; align-items: flex-start; }
.auth__point .icon-tile { background: var(--sidebar-hover); color: #fff; }
.auth__point b { display: block; font-weight: 700; }
.auth__point span { color: var(--sidebar-text); font-size: 13px; }
.auth__foot { color: var(--sidebar-label); font-size: 13px; }
.auth__panel { display: grid; place-items: center; padding: 32px 20px; background: var(--bg); }
.auth-card { width: min(480px, 100%); background: #fff; border-radius: 20px; padding: 36px; box-shadow: var(--shadow-card); border: 1px solid rgba(221, 226, 240, 0.7);
}
.auth-card--wide { width: min(640px, 100%); }
.auth-card h1 { font-size: 26px; font-weight: 800; }
.auth-card .lead { color: var(--muted); margin: 6px 0 24px; }
.auth-form { display: grid; gap: 16px; }
.auth-switch { text-align: center; margin-top: 22px; color: var(--muted); }
.auth-switch a { color: var(--primary-dark); font-weight: 700; }
.role-picker { display: grid; grid-template-columns: repeat(auto-fit, minmax(130px, 1fr)); gap: 10px; }
.role-option { position: relative; border: 1px solid var(--border); border-radius: var(--radius-ctl); padding: 12px 14px; background: #fff; text-align: left; font-
weight: 600; display: flex; flex-direction: column; gap: 2px; }
.role-option small { color: var(--muted); font-weight: 500; }
.role-option:hover { border-color: #B9BAF7; }
.role-option.active { border-color: var(--primary); background: var(--primary-soft); box-shadow: 0 0 0 3px rgba(91, 91, 240, 0.15); }
.divider-label { display: flex; align-items: center; gap: 12px; color: var(--muted); font-size: 12px; font-weight: 700; text-transform: uppercase; letter-spacing:
0.06em; }
.divider-label::after { content: ''; flex: 1; height: 1px; background: var(--border); }
@media (max-width: 900px) {
  .auth { grid-template-columns: 1fr; }
  .auth__brand { display: none; }
  .auth-card { padding: 26px 20px; }
}
```

frontend/src/styles/components.css

```
/* Reusable components */
.card { background: var(--surface); border: 1px solid rgba(221, 226, 240, 0.7); border-radius: var(--radius-card); padding: 22px; box-shadow: var(--shadow-card); min-
width: 0; }
.card__head { display: flex; align-items: center; justify-content: space-between; gap: 12px; margin-bottom: 16px; }
.card__head h3 { font-size: 16px; font-weight: 700; }
.card--dark { background: var(--sidebar); color: #fff; border-color: transparent; }

/* Page header */
.page-head { display: flex; align-items: flex-end; justify-content: space-between; gap: 16px; flex-wrap: wrap; }
.page-head h1 { font-size: 24px; font-weight: 800; }
.page-head p { color: var(--muted); margin-top: 2px; }

/* Hero banner */
.hero { background: var(--primary-dark); color: #fff; border-radius: 20px; padding: 28px 32px; display: flex; justify-content: space-between; align-items: center; gap:
24px; flex-wrap: wrap; }
.hero h1 { font-size: 26px; font-weight: 800; margin-bottom: 6px; }
.hero p { color: #D6D9FF; }

/* Buttons */
.btn { display: inline-flex; align-items: center; justify-content: center; gap: 8px; border: 1px solid transparent; border-radius: var(--radius-ctl); padding: 10px
18px; font-weight: 700; transition: background 0.15s, border-color 0.15s, transform 0.05s; white-space: nowrap; }
.btn:active { transform: translateY(1px); }
.btn:disabled { opacity: 0.55; cursor: not-allowed; }
.btn-primary { background: var(--primary); color: #fff; }
.btn-primary:hover:not(:disabled) { background: var(--primary-hover); }
.btn-light { background: #fff; color: var(--primary-dark); }
.btn-light:hover { background: var(--primary-soft); }
.btn-outline { background: #fff; border-color: var(--border); color: var(--text); }
.btn-outline:hover { background: var(--divider); }
.btn-danger { background: var(--danger-soft); color: var(--danger-ink); }
.btn-danger:hover { background: #F9CFD1; }
.btn-success { background: var(--success-soft); color: var(--success-ink); }
.btn-success:hover { background: #C7EFDB; }
.btn-sm { padding: 7px 12px; font-size: 13px; border-radius: 10px; }
.btn-block { width: 100%; }
.btn-icon { width: 42px; height: 42px; padding: 0; background: #fff; border-color: var(--border); color: var(--text); font-size: 18px; }
.btn-icon:hover { background: var(--divider); }

/* Forms */
.field { display: flex; flex-direction: column; gap: 6px; min-width: 0; }
.field > label { font-weight: 600; font-size: 13px; }
.input, .select, .textarea { width: 100%; padding: 11px 14px; border: 1px solid var(--border); border-radius: var(--radius-ctl); background: #fff; transition: border-
color 0.15s, box-shadow 0.15s; }
.input:focus, .select:focus, .textarea:focus { outline: none; border-color: var(--primary); box-shadow: 0 0 0 3px rgba(91, 91, 240, 0.18); }
.input.error, .select.error { border-color: var(--danger); }
.textarea { min-height: 96px; resize: vertical; }
.field-error { color: var(--danger-ink); font-size: 12px; }
.field-hint { color: var(--muted); font-size: 12px; }
.form-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(min(220px, 100%), 1fr)); gap: 16px; }
.toolbar { display: flex; flex-wrap: wrap; gap: 12px; align-items: center; }
.toolbar .search { flex: 1 1 260px; }
.toolbar .select { width: auto; min-width: 160px; }

/* Alerts */
.alert { padding: 12px 14px; border-radius: var(--radius-ctl); font-weight: 500; border: 1px solid transparent; }
.alert-success { background: var(--success-soft); color: var(--success-ink); border-color: #BFE9D4; }
.alert-error { background: var(--danger-soft); color: var(--danger-ink); border-color: #F6BFC1; }
.alert-info { background: var(--primary-soft); color: var(--primary-ink); border-color: #CFCFFA; }
```

```
.alert-warning { background: var(--warning-soft); color: var(--warning-ink); border-color: #F4DBA1; }

/* Badges and chips */
.badge { display: inline-flex; align-items: center; padding: 4px 10px; border-radius: var(--radius-pill); font-size: 12px; font-weight: 700; white-space: nowrap; }
.badge-success { background: var(--success-soft); color: var(--success-ink); }
.badge-warning { background: var(--warning-soft); color: var(--warning-ink); }
.badge-danger { background: var(--danger-soft); color: var(--danger-ink); }
.badge-info { background: var(--primary-soft); color: var(--primary-ink); }
.badge-neutral { background: var(--divider); color: var(--muted); }
.chip { display: inline-flex; align-items: center; padding: 7px 14px; border-radius: var(--radius-pill); background: var(--divider); font-weight: 600; font-size: 13px;
border: 1px solid transparent; }
.chip-list { display: flex; flex-wrap: wrap; gap: 8px; }
button.chip:hover { background: var(--primary-soft); }
.chip.active { background: var(--primary-soft); color: var(--primary-ink); border-color: #CFCFFA; }

/* Stat cards */
.stat-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(min(210px, 100%), 1fr)); gap: 18px; }
.stat { display: flex; align-items: flex-start; gap: 14px; }
.stat__icon { width: 44px; height: 44px; border-radius: 12px; display: grid; place-items: center; font-size: 20px; flex: none; }
.tone-primary { background: var(--primary-soft); color: var(--primary-ink); }
.tone-success { background: var(--success-soft); color: var(--success-ink); }
.tone-warning { background: var(--warning-soft); color: var(--warning-ink); }
.tone-danger { background: var(--danger-soft); color: var(--danger-ink); }
.stat__label { color: var(--muted); font-size: 13px; font-weight: 500; }
.stat__value { font-size: 28px; font-weight: 800; line-height: 1.2; margin: 2px 0; }
.stat__hint { font-size: 12px; font-weight: 600; }
.stat__hint.up { color: var(--success); }
.stat__hint.down { color: #B54708; }
.stat__hint.flat { color: var(--muted); }

/* Tables */
.table-wrap { overflow-x: auto; }
.table { width: 100%; border-collapse: collapse; min-width: 520px; }
.table th { text-align: left; font-size: 12px; color: var(--muted); font-weight: 600; padding: 8px 12px 8px 0; border-bottom: 1px solid var(--track); white-space:
nowrap; }
.table td { padding: 13px 12px 13px 0; border-bottom: 1px solid var(--divider); font-weight: 500; vertical-align: middle; }
.table tbody tr:hover td { background: #FAFBFF; }
.table tr:last-child td { border-bottom: 0; }
.table .actions { text-align: right; white-space: nowrap; }
.table .actions .btn { margin-left: 6px; }
.cell-user { display: flex; align-items: center; gap: 12px; }

/* Progress, ring, bars */
.progress { height: 8px; background: var(--track); border-radius: 99px; overflow: hidden; }
.progress > span { display: block; height: 100%; border-radius: 99px; background: var(--primary); }
.progress.warning > span { background: var(--warning); }
.progress.success > span { background: var(--success); }
.progress.danger > span { background: var(--danger); }
.meter-row { margin-bottom: 16px; }
.meter-row:last-child { margin-bottom: 0; }
.meter-row__top { display: flex; justify-content: space-between; margin-bottom: 6px; font-weight: 500; }
.meter-row__top b { font-weight: 700; }
.ring { display: block; flex: none; }
.ring__track { stroke: var(--track); }
.ring__value { stroke: var(--primary); }
.ring__text { fill: var(--text); font-weight: 800; }
.bars { display: flex; align-items: flex-end; gap: 14px; height: 200px; padding-top: 8px; }
.bars__col { flex: 1; min-width: 0; height: 100%; display: flex; flex-direction: column; justify-content: flex-end; align-items: center; gap: 6px; }
.bars__val { font-size: 12px; font-weight: 700; }
.bars__fill { width: 100%; max-width: 46px; border-radius: 8px 8px 0 0; background: var(--primary); }
.bars__fill.alt { background: #A5A6F8; }
.bars__fill.warn { background: var(--warning); }
.bars__label { font-size: 12px; color: var(--muted); font-weight: 500; text-align: center; }

/* List rows and tiles */
.list-row { display: flex; align-items: center; gap: 14px; padding: 13px 0; border-bottom: 1px solid var(--divider); }
.list-row:last-child { border-bottom: 0; padding-bottom: 0; }
.list-row:first-child { padding-top: 0; }
.date-tile { width: 52px; height: 54px; flex: none; border-radius: 12px; background: var(--primary-soft); color: var(--primary-ink); text-align: center; font-weight:
800; font-size: 18px; padding-top: 7px; line-height: 1.1; }
.date-tile small { display: block; font-size: 11px; font-weight: 700; letter-spacing: 0.04em; }
.icon-tile { width: 40px; height: 40px; border-radius: 12px; display: grid; place-items: center; font-size: 18px; flex: none; }

/* Avatar */
.avatar { width: 42px; height: 42px; border-radius: 50%; background: var(--primary); color: #fff; display: inline-grid; place-items: center; font-weight: 700; flex:
none; }
.avatar-sm { width: 34px; height: 34px; font-size: 13px; }
.avatar-lg { width: 84px; height: 84px; font-size: 28px; }

/* Tabs */
.tabs { display: inline-flex; gap: 4px; background: #fff; border: 1px solid var(--border); border-radius: var(--radius-ctl); padding: 4px; flex-wrap: wrap; }
.tab { padding: 8px 14px; border-radius: 9px; border: 0; background: transparent; font-weight: 600; color: var(--muted); }
.tab:hover { color: var(--text); }
.tab.active { background: var(--primary-soft); color: var(--primary-ink); }
.tab small { margin-left: 6px; font-weight: 700; }

/* Modal */
.modal-backdrop { position: fixed; inset: 0; background: rgba(21, 26, 60, 0.55); display: grid; place-items: center; padding: 16px; z-index: 100; }
.modal { width: min(600px, 100%); max-height: 90vh; overflow-y: auto; background: #fff; border-radius: 20px; padding: 26px; box-shadow: var(--shadow-pop); }
.modal__head { display: flex; justify-content: space-between; align-items: center; margin-bottom: 18px; }
.modal__head h2 { font-size: 20px; font-weight: 800; }
.modal__foot { display: flex; justify-content: flex-end; gap: 10px; margin-top: 22px; }

/* Empty + skeleton */
.empty { text-align: center; padding: 36px 16px; color: var(--muted); }
.empty svg { font-size: 34px; color: #A5ABCB; margin-bottom: 8px; }
.empty h4 { color: var(--text); font-size: 15px; font-weight: 700; margin-bottom: 4px; }
.skeleton { display: block; border-radius: 10px; background: linear-gradient(90deg, var(--track) 25%, #F4F6FB 50%, var(--track) 75%); background-size: 200% 100%;
animation: shimmer 1.2s infinite linear; }
@keyframes shimmer { to { background-position: -200% 0; } }
@media (prefers-reduced-motion: reduce) { .skeleton { animation: none; } }
```

frontend/src/styles/global.css

```
/* CampusFlow design tokens and base styles */
:root {
  --bg: #F4F6FB;
  --surface: #FFFFFF;
  --text: #1B2140;
  --muted: #5B6385;
  --border: #DDE2F0;
```

```
  --divider: #F0F2F9;
  --track: #E8EBF5;

  --sidebar: #151A3C;
  --sidebar-hover: #222855;
  --sidebar-text: #B9C0E6;
  --sidebar-label: #6F78AD;

  --primary: #5B5BF0;
  --primary-hover: #4A4AD8;
  --primary-dark: #4338CA;
  --primary-soft: #E6E6FD;
  --primary-ink: #3730A3;

  --success: #0E8F5B;
  --success-soft: #DDF6EA;
  --success-ink: #0A6B43;
  --warning: #E5A100;
  --warning-soft: #FFF1D6;
  --warning-ink: #8A5300;
  --danger: #E5484D;
  --danger-soft: #FDE3E4;
  --danger-ink: #A4151B;

  --radius-card: 16px;
  --radius-ctl: 12px;
  --radius-pill: 999px;
  --shadow-card: 0 1px 3px rgba(27, 33, 64, 0.06), 0 8px 24px rgba(27, 33, 64, 0.04);
  --shadow-pop: 0 12px 40px rgba(21, 26, 60, 0.22);
  --font: 'Plus Jakarta Sans', system-ui, -apple-system, 'Segoe UI', Roboto, Arial, sans-serif;
}

*,
*::before,
*::after { box-sizing: border-box; }

html, body, #root { min-height: 100%; }

body {
  margin: 0;
  font-family: var(--font);
  font-size: 14px;
  line-height: 1.5;
  color: var(--text);
  background: var(--bg);
  -webkit-font-smoothing: antialiased;
}

h1, h2, h3, h4, p { margin: 0; }
a { color: inherit; text-decoration: none; }
button, input, select, textarea { font: inherit; color: inherit; }
button { cursor: pointer; }
ul { margin: 0; padding: 0; list-style: none; }

:focus-visible {
  outline: 3px solid rgba(91, 91, 240, 0.45);
  outline-offset: 2px;
}

.sr-only {
  position: absolute; width: 1px; height: 1px; padding: 0; margin: -1px;
  overflow: hidden; clip: rect(0, 0, 0, 0); white-space: nowrap; border: 0;
}

/* Utilities */
.muted { color: var(--muted); }
.text-sm { font-size: 12.5px; }
.fw-600 { font-weight: 600; }
.fw-700 { font-weight: 700; }
.flex { display: flex; }
.wrap { flex-wrap: wrap; }
.between { justify-content: space-between; }
.center { align-items: center; }
.gap-8 { gap: 8px; }
.gap-12 { gap: 12px; }
.gap-16 { gap: 16px; }
.grow { flex: 1; min-width: 0; }
.mt-16 { margin-top: 16px; }
.stack { display: flex; flex-direction: column; gap: 18px; }
.grid-2 { display: grid; grid-template-columns: repeat(auto-fit, minmax(min(380px, 100%), 1fr)); gap: 18px; }
.grid-3 { display: grid; grid-template-columns: repeat(auto-fit, minmax(min(300px, 100%), 1fr)); gap: 18px; }
.grid-main { display: grid; grid-template-columns: minmax(0, 1.6fr) minmax(0, 1fr); gap: 18px; }
@media (max-width: 1000px) { .grid-main { grid-template-columns: 1fr; } }
```

frontend/src/styles/layout.css

```
/* App shell: sidebar, topbar, content */
.app { display: flex; min-height: 100vh; }

.sidebar {
  width: 248px; flex: none; background: var(--sidebar); color: var(--sidebar-text);
  padding: 24px 16px; display: flex; flex-direction: column; gap: 2px;
  position: sticky; top: 0; height: 100vh; overflow-y: auto; z-index: 40;
}
.sidebar__brand { display: flex; align-items: center; gap: 10px; color: #fff; font-weight: 800; font-size: 20px; padding: 0 10px 22px; }
.brand-mark {
  width: 34px; height: 34px; border-radius: 10px; background: var(--primary); color: #fff;
  display: grid; place-items: center; font-size: 17px; font-weight: 800; flex: none;
}
.nav-label { font-size: 11px; letter-spacing: 0.08em; text-transform: uppercase; color: var(--sidebar-label); padding: 18px 12px 6px; font-weight: 700; }
.nav-link {
  display: flex; align-items: center; gap: 12px; padding: 11px 12px; border-radius: 10px;
  color: var(--sidebar-text); font-weight: 500; transition: background 0.15s, color 0.15s;
}
.nav-link svg { font-size: 18px; flex: none; }
.nav-link:hover { background: var(--sidebar-hover); color: #fff; }
.nav-link.active { background: var(--primary); color: #fff; font-weight: 600; }
.sidebar__foot { margin-top: auto; padding-top: 16px; }
.sidebar__signout {
  width: 100%; display: flex; align-items: center; gap: 12px; padding: 11px 12px; border-radius: 10px;
  background: transparent; border: 0; color: var(--sidebar-text); font-weight: 500; text-align: left;
```

```
}
.sidebar__signout:hover { background: var(--sidebar-hover); color: #fff; }
.scrim { display: none; }

.main { flex: 1; min-width: 0; display: flex; flex-direction: column; }
.topbar {
  display: flex; align-items: center; gap: 14px; padding: 18px 32px; position: sticky; top: 0; z-index: 30;
  background: rgba(244, 246, 251, 0.92); backdrop-filter: blur(8px);
}
.topbar__spacer { flex: 1; }
.btn.menu-btn { display: none; }
.search { position: relative; flex: 1; max-width: 420px; }
.search svg { position: absolute; left: 14px; top: 50%; transform: translateY(-50%); color: var(--muted); pointer-events: none; }
.search .input { padding-left: 40px; padding-right: 38px; }
.search__clear {
  position: absolute; right: 8px; top: 50%; transform: translateY(-50%); width: 26px; height: 26px;
  border-radius: 8px; border: 0; background: transparent; color: var(--muted); display: grid; place-items: center;
}
.search__clear:hover { background: var(--divider); }
.bell { position: relative; }
.bell__dot { position: absolute; top: 9px; right: 10px; width: 9px; height: 9px; border-radius: 50%; background: var(--danger); border: 2px solid #fff; }
.menu-wrap { position: relative; }
.avatar-btn { border: 0; padding: 0; background: transparent; border-radius: 50%; }
.dropdown {
  position: absolute; right: 0; top: 52px; width: 220px; background: #fff; border: 1px solid var(--border);
  border-radius: 14px; box-shadow: var(--shadow-pop); padding: 8px; z-index: 50;
}
.dropdown__who { padding: 10px 12px 12px; border-bottom: 1px solid var(--divider); margin-bottom: 6px; }
.dropdown__item {
  width: 100%; display: flex; align-items: center; gap: 10px; padding: 10px 12px; border-radius: 10px;
  border: 0; background: transparent; text-align: left; font-weight: 500;
}
.dropdown__item:hover { background: var(--divider); }

.content { padding: 4px 32px 40px; display: flex; flex-direction: column; gap: 22px; width: 100%; max-width: 1280px; }

@media (max-width: 900px) {
  .sidebar { position: fixed; left: 0; top: 0; bottom: 0; transform: translateX(-100%); transition: transform 0.2s ease; }
  .sidebar.open { transform: none; box-shadow: var(--shadow-pop); }
  .scrim.show { display: block; position: fixed; inset: 0; background: rgba(21, 26, 60, 0.5); z-index: 35; }
  .btn.menu-btn { display: inline-flex; }
  .topbar { padding: 14px 16px; gap: 10px; }
  .topbar__spacer { display: none; }
  .search { max-width: none; }
  .content { padding: 4px 16px 32px; }
}
```

frontend/src/styles/pages.css

```
/* Page-specific layout helpers */
.ai-card .bubble { background: var(--sidebar-hover); color: #DCE0FF; }
.ai-card .bubble.me { background: var(--primary); color: #fff; }
.ai-card .input { background: var(--sidebar-hover); border-color: #2F3770; color: #fff; }
.ai-card .input::placeholder { color: #8C94C8; }
.chat { display: flex; flex-direction: column; gap: 12px; }
.bubble { max-width: 80%; padding: 12px 14px; border-radius: 14px; line-height: 1.55; background: var(--divider); }
.bubble.me { align-self: flex-end; background: var(--primary); color: #fff; }
.bubble.error { background: var(--danger-soft); color: var(--danger-ink); }
.bubble small { display: block; margin-top: 6px; opacity: 0.75; font-weight: 600; }
.chat-page { display: flex; flex-direction: column; min-height: 560px; }
.chat-page .chat { flex: 1; overflow-y: auto; padding: 4px 0 16px; }
.chat-input { display: flex; gap: 10px; padding-top: 14px; border-top: 1px solid var(--divider); }

.kv { display: grid; grid-template-columns: repeat(auto-fit, minmax(190px, 1fr)); gap: 18px 24px; }
.kv dt { color: var(--muted); font-size: 12px; font-weight: 600; margin-bottom: 2px; }
.kv dd { margin: 0; font-weight: 600; }

.steps { display: grid; grid-template-columns: repeat(auto-fit, minmax(150px, 1fr)); gap: 14px; }
.step { border: 1px solid var(--border); border-radius: 14px; padding: 16px; position: relative; }
.step__no { width: 30px; height: 30px; border-radius: 50%; display: grid; place-items: center; font-weight: 800; margin-bottom: 10px; background: var(--track); color:
var(--muted); }
.step.done .step__no { background: var(--success); color: #fff; }
.step.current { border-color: var(--primary); background: var(--primary-soft); }
.step.current .step__no { background: var(--primary); color: #fff; }
.step h4 { font-size: 15px; margin-bottom: 2px; }

.event-grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(min(320px, 100%), 1fr)); gap: 18px; }
.event-card { display: flex; flex-direction: column; gap: 14px; }
.event-card__top { display: flex; gap: 14px; align-items: flex-start; }
.event-card__meta { display: grid; gap: 6px; color: var(--muted); font-size: 13px; }
.event-card__meta span { display: flex; align-items: center; gap: 8px; }
.event-card__foot { display: flex; gap: 8px; margin-top: auto; }

.calendar { display: grid; grid-template-columns: repeat(7, minmax(0, 1fr)); gap: 6px; }
.calendar__dow { font-size: 12px; font-weight: 700; color: var(--muted); padding: 4px 6px; }
.calendar__cell { min-height: 92px; border: 1px solid var(--divider); border-radius: 10px; padding: 6px; background: #fff; }
.calendar__cell.blank { background: transparent; border-color: transparent; }
.calendar__day { font-size: 12px; font-weight: 700; color: var(--muted); }
.calendar__event { display: block; margin-top: 4px; padding: 3px 6px; border-radius: 6px; background: var(--primary-soft); color: var(--primary-ink); font-size: 11px;
font-weight: 700; overflow: hidden; text-overflow: ellipsis; white-space: nowrap; }

.notif { display: flex; gap: 14px; padding: 16px 0; border-bottom: 1px solid var(--divider); align-items: flex-start; }
.notif:last-child { border-bottom: 0; }
.notif.unread { background: linear-gradient(90deg, rgba(230, 230, 253, 0.55), transparent 70%); margin: 0 -22px; padding-left: 22px; padding-right: 22px; }
.notif__title { font-weight: 700; }
.notif__time { margin-left: auto; color: var(--muted); font-size: 12px; white-space: nowrap; }

.result-group + .result-group { margin-top: 18px; }
.result-link { display: flex; justify-content: space-between; gap: 12px; padding: 12px 14px; border-radius: 12px; }
.result-link:hover { background: var(--divider); }

.attendance-input { display: inline-flex; gap: 6px; }
.radio-pill { position: relative; }
.radio-pill input { position: absolute; opacity: 0; inset: 0; cursor: pointer; }
.radio-pill span { display: inline-block; padding: 6px 12px; border-radius: var(--radius-pill); border: 1px solid var(--border); font-weight: 600; font-size: 13px;
background: #fff; }
.radio-pill input:focus-visible + span { outline: 3px solid rgba(91, 91, 240, 0.45); outline-offset: 2px; }
.radio-pill input:checked + span.p { background: var(--success-soft); border-color: #BFE9D4; color: var(--success-ink); }
```

```
.radio-pill input:checked + span.a { background: var(--danger-soft); border-color: #F6BFC1; color: var(--danger-ink); }
.radio-pill input:checked + span.l { background: var(--warning-soft); border-color: #F4DBA1; color: var(--warning-ink); }
```

Demo data

frontend/src/data/demoData.js

```
/*
 * DEMO DATA ONLY.
 * Every page imports its sample records from this file so the UI can be previewed.
 * When wiring the backend, replace each import with an API call (see docs/UI_SPEC.md,
 * "Data needed" per screen). Do not change the layout or CSS when doing so.
 * Dates are ISO strings (YYYY-MM-DD).
 */

export const student = {
  name: 'Alex Morgan', email: 'student@campusflow.test', phone: '+91 90000 00001',
  department: 'Computer Science', program: 'B.Tech Computer Science', studentId: 'CS21B042',
  academicYear: '2023 - 2027', semester: 5,
};

export const courses = [
  { id: 1, code: 'CS301', name: 'Data Structures', department: 'Computer Science', credits: 4, semester: 5, faculty: 'Dr. Priya Nair', academicYear: '2026-27',
description: 'Arrays, trees, graphs and algorithm analysis.' },
  { id: 2, code: 'CS302', name: 'Database Systems', department: 'Computer Science', credits: 4, semester: 5, faculty: 'Prof. Arjun Rao', academicYear: '2026-27',
description: 'Relational model, SQL, normalization and transactions.' },
  { id: 3, code: 'CS303', name: 'Web Technologies', department: 'Computer Science', credits: 3, semester: 5, faculty: 'Dr. Meera Das', academicYear: '2026-27',
description: 'React, REST APIs and full-stack development.' },
  { id: 4, code: 'CS304', name: 'Software Engineering', department: 'Computer Science', credits: 3, semester: 5, faculty: 'Prof. Kiran Shah', academicYear: '2026-27',
description: 'SDLC, testing, design patterns and project management.' },
  { id: 5, code: 'AI305', name: 'Machine Learning Basics', department: 'AI & ML', credits: 3, semester: 5, faculty: 'Dr. Neha Kulkarni', academicYear: '2026-27',
description: 'Supervised learning, evaluation and model selection.' },
  { id: 6, code: 'EC301', name: 'Digital Electronics', department: 'Electronics', credits: 3, semester: 5, faculty: 'Prof. Vikram Joshi', academicYear: '2026-27',
description: 'Logic design, flip-flops and counters.' },
];

export const assignments = [
  { id: 1, title: 'REST API Design', course: 'Web Technologies', due: '2026-10-11', maxMarks: 20, status: 'PENDING', marks: null, submitted: 31, total: 48 },
  { id: 2, title: 'ER Diagram Lab', course: 'Database Systems', due: '2026-10-14', maxMarks: 25, status: 'IN_PROGRESS', marks: null, submitted: 22, total: 48 },
  { id: 3, title: 'Sorting Algorithms Report', course: 'Data Structures', due: '2026-10-16', maxMarks: 30, status: 'IN_PROGRESS', marks: null, submitted: 18, total: 48
},
  { id: 4, title: 'Unit Test Case Set', course: 'Software Engineering', due: '2026-10-09', maxMarks: 20, status: 'SUBMITTED', marks: null, submitted: 45, total: 48 },
  { id: 5, title: 'SQL Joins Worksheet', course: 'Database Systems', due: '2026-10-02', maxMarks: 15, status: 'GRADED', marks: 13, submitted: 48, total: 48 },
  { id: 6, title: 'Linked List Implementation', course: 'Data Structures', due: '2026-09-28', maxMarks: 20, status: 'GRADED', marks: 17, submitted: 47, total: 48 },
];

export const attendanceByCourse = [
  { course: 'Data Structures', attended: 46, total: 50, percent: 92 },
  { course: 'Database Systems', attended: 44, total: 50, percent: 88 },
  { course: 'Web Technologies', attended: 34, total: 40, percent: 85 },
  { course: 'Software Engineering', attended: 31, total: 42, percent: 74 },
  { course: 'Machine Learning Basics', attended: 33, total: 40, percent: 83 },
];

export const attendanceHistory = [
  { id: 1, date: '2026-10-09', course: 'Data Structures', status: 'Present' },
  { id: 2, date: '2026-10-09', course: 'Software Engineering', status: 'Absent' },
  { id: 3, date: '2026-10-08', course: 'Database Systems', status: 'Present' },
  { id: 4, date: '2026-10-08', course: 'Web Technologies', status: 'Late' },
  { id: 5, date: '2026-10-07', course: 'Machine Learning Basics', status: 'Present' },
];

export const results = [
  { id: 1, assessment: 'Mid-term Exam', course: 'Data Structures', marks: 41, total: 50, grade: 'A' },
  { id: 2, assessment: 'Mid-term Exam', course: 'Database Systems', marks: 36, total: 50, grade: 'B+' },
  { id: 3, assessment: 'Quiz 2', course: 'Web Technologies', marks: 18, total: 20, grade: 'A+' },
  { id: 4, assessment: 'Internal Test', course: 'Software Engineering', marks: 27, total: 40, grade: 'B' },
  { id: 5, assessment: 'Lab Evaluation', course: 'Machine Learning Basics', marks: 22, total: 25, grade: 'A' },
];

export const semesterGpa = [
  { label: 'Sem 1', value: 7.8 }, { label: 'Sem 2', value: 8.0 }, { label: 'Sem 3', value: 8.1 },
  { label: 'Sem 4', value: 8.4 }, { label: 'Sem 5', value: 8.5, tone: 'alt' },
];

export const nextActions = [
  { id: 1, title: 'Submit REST API Design', hint: 'Due tomorrow in Web Technologies', badge: 'Urgent', tone: 'danger', to: 'assignments' },
  { id: 2, title: 'Revise SQL joins before the DBMS quiz', hint: 'Based on your last assessment result', badge: 'Suggested', tone: 'info', to: 'results' },
  { id: 3, title: 'Practice 5 Java interview questions', hint: 'Closes your Spring Security skill gap', badge: 'Skill gap', tone: 'warning', to: 'skills' },
];

export const progress = [
  { label: 'Courses', value: '5 / 6' }, { label: 'Assignments', value: '3 / 5' },
  { label: 'Assessments', value: '4 / 6' }, { label: 'Attendance', value: '87%' },
];

export const skills = [
  { name: 'Java', level: 4, target: 5 }, { name: 'SQL', level: 3, target: 5 },
  { name: 'React', level: 3, target: 4 }, { name: 'Spring Boot', level: 2, target: 4 },
  { name: 'Data Structures', level: 4, target: 5 }, { name: 'System Design', level: 1, target: 3 },
];

export const skillGaps = ['System Design', 'Spring Security', 'SQL Optimization'];

export const recommendedPractice = [
  { id: 1, title: 'DSA: Arrays', hint: '12 questions', badge: 'Recommended' },
  { id: 2, title: 'DBMS: SQL Joins', hint: '8 questions', badge: 'Recommended' },
  { id: 3, title: 'Java OOP Concepts', hint: '10 questions', badge: 'Optional' },
];

export const learningPath = [
  { step: 'Basics', topic: 'Java Fundamentals', state: 'done' },
  { step: 'Practice', topic: 'DSA - Arrays', state: 'current' },
  { step: 'Build', topic: 'DBMS - SQL', state: 'next' },
  { step: 'Master', topic: 'Project Building', state: 'next' },
];

export const readinessFactors = [
  { label: 'Skills', value: 70, weight: 35 }, { label: 'Academic performance', value: 80, weight: 25 },
  { label: 'Assessment performance', value: 65, weight: 20 }, { label: 'Attendance', value: 87, weight: 10 },
```

```
  { label: 'Placement interests', value: 60, weight: 10 },
];

export const drives = [
  { id: 1, company: 'TCS', role: 'Software Engineer', date: '2026-10-28', deadline: '2026-10-22', minCgpa: 7.0, minAttendance: 75, departments: 'All departments',
skills: ['Java', 'SQL', 'Aptitude'], registered: 64, eligible: true, reason: '' },
  { id: 2, company: 'Infosys', role: 'Systems Engineer', date: '2026-11-04', deadline: '2026-10-30', minCgpa: 6.5, minAttendance: 75, departments: 'CSE, AI & ML, ECE',
skills: ['Java', 'Problem Solving'], registered: 41, eligible: true, reason: '' },
  { id: 3, company: 'Zoho', role: 'Member Technical Staff', date: '2026-11-10', deadline: '2026-11-03', minCgpa: 8.5, minAttendance: 80, departments: 'CSE, AI & ML',
skills: ['DSA', 'System Design'], registered: 18, eligible: false, reason: 'Minimum CGPA 8.5 required' },
  { id: 4, company: 'Capgemini', role: 'Analyst', date: '2026-11-15', deadline: '2026-11-08', minCgpa: 6.0, minAttendance: 75, departments: 'All departments', skills:
['SQL', 'Communication'], registered: 5, eligible: true, reason: '' },
];

export const eligibleStudents = [
  { id: 1, name: 'Alex Morgan', roll: 'CS21B042', department: 'CSE', cgpa: 8.4, readiness: 72 },
  { id: 2, name: 'Nisha Patel', roll: 'CS21B018', department: 'CSE', cgpa: 8.9, readiness: 81 },
  { id: 3, name: 'Rohan Mehta', roll: 'AI21B007', department: 'AI & ML', cgpa: 7.6, readiness: 64 },
  { id: 4, name: 'Zoya Khan', roll: 'EC21B031', department: 'ECE', cgpa: 7.2, readiness: 58 },
];

export const venues = [
  { id: 1, name: 'Main Auditorium', building: 'Block A', floor: 2, room: 'A-201', description: 'Next to the admin office, lift access available.', capacity: 400 },
  { id: 2, name: 'Seminar Hall', building: 'Block C', floor: 0, room: 'C-001', description: 'Ground floor, opposite the library entrance.', capacity: 150 },
  { id: 3, name: 'Computer Lab 4', building: 'Block B', floor: 1, room: 'B-104', description: 'First floor, east wing of the CSE department.', capacity: 60 },
  { id: 4, name: 'Open Air Theatre', building: 'Campus Ground', floor: 0, room: '-', description: 'Beside the main sports ground.', capacity: 800 },
];

export const events = [
  { id: 1, title: 'Hackathon 2026', category: 'Hackathon', date: '2026-10-18', start: '09:00', end: '21:00', venue: 'Main Auditorium', building: 'Block A', floor: 2,
room: 'A-201', department: 'Computer Science', eligibility: 'All departments, teams of 3 to 4', deadline: '2026-10-15', total: 200, registered: 148, approval:
'APPROVED', description: 'A 12-hour build sprint where teams create working prototypes around campus problems. Mentors from faculty and industry guide each team.' },
  { id: 2, title: 'Guest Lecture: Cloud Careers', category: 'Guest lecture', date: '2026-10-22', start: '11:00', end: '12:30', venue: 'Seminar Hall', building: 'Block
C', floor: 0, room: 'C-001', department: 'Computer Science', eligibility: 'Semester 5 and above', deadline: '2026-10-21', total: 150, registered: 92, approval:
'APPROVED', description: 'An industry speaker walks through cloud engineering roles, certifications and how to prepare.' },
  { id: 3, title: 'Placement Training: Aptitude', category: 'Placement training', date: '2026-10-25', start: '10:00', end: '13:00', venue: 'Computer Lab 4', building:
'Block B', floor: 1, room: 'B-104', department: 'Training & Placement', eligibility: 'Final and pre-final year', deadline: '2026-10-23', total: 60, registered: 54,
approval: 'APPROVED', description: 'Timed aptitude practice with solutions and strategy sessions.' },
  { id: 4, title: 'Inter-department Cricket', category: 'Sports', date: '2026-10-29', start: '08:00', end: '17:00', venue: 'Open Air Theatre', building: 'Campus
Ground', floor: 0, room: '-', department: 'Physical Education', eligibility: 'Open to all students', deadline: '2026-10-26', total: 120, registered: 60, approval:
'APPROVED', description: 'League matches between departments with a final on the last day.' },
  { id: 5, title: 'Cultural Night', category: 'Cultural', date: '2026-10-30', start: '17:00', end: '21:00', venue: 'Main Auditorium', building: 'Block A', floor: 2,
room: 'A-201', department: 'Student Council', eligibility: 'Open to all students', deadline: '2026-10-28', total: 400, registered: 210, approval: 'PENDING',
description: 'Music, dance and drama performances by student clubs.' },
  { id: 6, title: 'Coding Contest: Code Sprint', category: 'Coding contest', date: '2026-11-02', start: '14:00', end: '17:00', venue: 'Computer Lab 4', building: 'Block
B', floor: 1, room: 'B-104', department: 'Computer Science', eligibility: 'All departments', deadline: '2026-10-31', total: 60, registered: 12, approval: 'APPROVED',
description: 'Individual algorithmic contest with three difficulty tiers.' },
];

export const eventCategories = ['Workshop', 'Seminar', 'Hackathon', 'Cultural', 'Sports', 'Coding contest', 'Exhibition', 'Guest lecture', 'Club activity', 'Placement
training'];

export const notifications = [
  { id: 1, type: 'assignment', title: 'Assignment due tomorrow', message: 'REST API Design in Web Technologies is due on 11 Oct.', time: '2 hours ago', unread: true },
  { id: 2, type: 'attendance', title: 'Attendance warning', message: 'Software Engineering attendance is 74%, below the 75% requirement.', time: '5 hours ago', unread:
true },
  { id: 3, type: 'event', title: 'Event reminder', message: 'Hackathon 2026 starts on 18 Oct at 09:00, Main Auditorium.', time: 'Yesterday', unread: true },
  { id: 4, type: 'placement', title: 'New placement drive', message: 'TCS has opened registrations for Software Engineer. You are eligible.', time: 'Yesterday', unread:
false },
  { id: 5, type: 'announcement', title: 'Announcement from Dr. Priya Nair', message: 'Data Structures lab moves to Computer Lab 4 this week.', time: '2 days ago',
unread: false },
];

export const users = [
  { id: 1, name: 'Alex Morgan', email: 'student@campusflow.test', role: 'STUDENT', department: 'Computer Science', status: 'ACTIVE' },
  { id: 2, name: 'Dr. Priya Nair', email: 'priya.nair@campusflow.test', role: 'FACULTY', department: 'Computer Science', status: 'ACTIVE' },
  { id: 3, name: 'Rahul Verma', email: 'rahul.verma@campusflow.test', role: 'PLACEMENT_OFFICER', department: 'Training & Placement', status: 'ACTIVE' },
  { id: 4, name: 'Prof. Arjun Rao', email: 'arjun.rao@campusflow.test', role: 'FACULTY', department: 'Computer Science', status: 'PENDING' },
  { id: 5, name: 'Dr. Meera Das', email: 'meera.das@campusflow.test', role: 'FACULTY', department: 'AI & ML', status: 'PENDING' },
  { id: 6, name: 'Kavya Reddy', email: 'kavya.reddy@campusflow.test', role: 'PLACEMENT_OFFICER', department: 'Training & Placement', status: 'PENDING' },
  { id: 7, name: 'Nisha Patel', email: 'nisha.patel@campusflow.test', role: 'STUDENT', department: 'Computer Science', status: 'ACTIVE' },
  { id: 8, name: 'Zoya Khan', email: 'zoya.khan@campusflow.test', role: 'STUDENT', department: 'Electronics', status: 'INACTIVE' },
];

export const classRoster = [
  { id: 1, name: 'Alex Morgan', roll: 'CS21B042' }, { id: 2, name: 'Nisha Patel', roll: 'CS21B018' },
  { id: 3, name: 'Rohan Mehta', roll: 'AI21B007' }, { id: 4, name: 'Zoya Khan', roll: 'EC21B031' },
  { id: 5, name: 'Dev Malhotra', roll: 'CS21B055' }, { id: 6, name: 'Ishita Roy', roll: 'CS21B023' },
];

export const facultyStats = {
  performance: [
    { label: 'Quiz 1', value: 72 }, { label: 'Quiz 2', value: 78 }, { label: 'Mid-term', value: 69, tone: 'alt' },
    { label: 'Lab', value: 84 }, { label: 'Assignment', value: 88, tone: 'alt' },
  ],
  insights: [
    { id: 1, text: '9 students are below 60% in Database Systems', tone: 'danger', badge: 'Low performance' },
    { id: 2, text: '12 submissions are waiting for review', tone: 'warning', badge: 'Review' },
    { id: 3, text: '3 students have attendance below 75%', tone: 'warning', badge: 'Attendance' },
  ],
  schedule: [
    { id: 1, time: '09:00', course: 'Data Structures', room: 'B-104' },
    { id: 2, time: '11:00', course: 'Database Systems', room: 'A-101' },
    { id: 3, time: '14:00', course: 'Web Technologies Lab', room: 'B-201' },
  ],
  toReview: [
    { id: 1, student: 'Alex Morgan', assignment: 'REST API Design', course: 'Web Technologies', submitted: '10 Oct, 08:40' },
    { id: 2, student: 'Nisha Patel', assignment: 'ER Diagram Lab', course: 'Database Systems', submitted: '09 Oct, 21:15' },
    { id: 3, student: 'Rohan Mehta', assignment: 'ER Diagram Lab', course: 'Database Systems', submitted: '09 Oct, 18:02' },
  ],
};

export const placementStats = {
  readinessBands: [
    { label: '0-40', value: 28, tone: 'warn' }, { label: '41-60', value: 74 }, { label: '61-80', value: 96, tone: 'alt' }, { label: '81-100', value: 42 },
  ],
  registrationsByDrive: [
    { label: 'TCS', value: 64, of: 120 }, { label: 'Infosys', value: 41, of: 100 },
    { label: 'Zoho', value: 18, of: 40 }, { label: 'Capgemini', value: 5, of: 80 },
  ],
  skillDemand: ['Java', 'SQL', 'DSA', 'Spring Boot', 'React', 'System Design', 'Communication'],
  activity: [
    { id: 1, text: 'TCS drive: 12 new registrations today', time: '1 hour ago' },
```

```
    { id: 2, text: 'Infosys eligibility updated to CGPA 6.5', time: 'Yesterday' },
    { id: 3, text: 'Zoya Khan cancelled Capgemini registration', time: '2 days ago' },
  ],
};

export const adminStats = {
  usage: [
    { label: 'Mon', value: 820 }, { label: 'Tue', value: 940 }, { label: 'Wed', value: 1010, tone: 'alt' },
    { label: 'Thu', value: 970 }, { label: 'Fri', value: 880, tone: 'alt' }, { label: 'Sat', value: 310 },
  ],
  departmentUsage: [
    { label: 'Computer Science', value: 92 }, { label: 'AI & ML', value: 78 }, { label: 'Electronics', value: 65 }, { label: 'Mechanical', value: 48 },
  ],
  alerts: [
    { id: 1, text: 'Low activity in Mechanical department', tone: 'warning' },
    { id: 2, text: 'Cultural Night is waiting for approval', tone: 'info' },
    { id: 3, text: 'Venue clash detected: Main Auditorium, 30 Oct', tone: 'danger' },
  ],
  activity: [
    { id: 1, who: 'Dr. Priya Nair', action: 'Created assignment "Sorting Algorithms Report"', time: '10:42' },
    { id: 2, who: 'Rahul Verma', action: 'Opened TCS placement drive', time: '10:15' },
    { id: 3, who: 'Sana Iqbal', action: 'Approved faculty account Prof. Arjun Rao', time: '09:50' },
    { id: 4, who: 'Alex Morgan', action: 'Registered for Hackathon 2026', time: '09:31' },
  ],
};

export const chatSeed = [
  { id: 1, from: 'me', text: 'What is the eligibility for campus placements?' },
  { id: 2, from: 'bot', text: 'To be eligible for campus placements you need a minimum aggregate set by each company, no active backlogs, and at least 75% attendance.
Each drive lists its own criteria.' },
  { id: 3, from: 'me', text: 'Show me upcoming placement drives' },
  { id: 4, from: 'bot', text: 'Upcoming drives: TCS on 28 Oct, Infosys on 4 Nov, Zoho on 10 Nov and Capgemini on 15 Nov. Open the Placements page to check your
eligibility.' },
];
```

Project configuration

frontend/.env.example

```
# Copy to .env
VITE_API_BASE_URL=http://localhost:5454/api
# true = UI preview without a backend (login by email prefix: student@, faculty@, placement@, admin@)
# false = real backend (default for the final project)
VITE_DEMO_MODE=false
```

frontend/.gitignore

```
node_modules
dist
.env
.env.local
```

frontend/index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>CampusFlow</title>
    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
    <link
      href="https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@400;500;600;700;800&display=swap"
      rel="stylesheet"
    />
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

frontend/package.json

```
{
  "name": "campusflow-frontend",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "axios": "^1.7.9",
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "react-icons": "^5.4.0",
    "react-router-dom": "^6.28.0"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.3.4",
    "vite": "^5.4.11"
  }
}
```

frontend/public/.gitkeep

```
```

frontend/vite.config.js

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  server: { port: 5173 },
});
```